```
//+------------------------------------------------------------------+
//|                          Remora_2_43_6_HardDirectional_Candidate.mq5 |
//| RÊMORA 2.434 — Rastreador de Fluxo Intraday (até 15:30) |
//|                                                                  |
//| FUSÃO: Defesas financeiras (Remora 3.6.0) + Cérebro de Fita      |
//| Quantitativa (Divergência de Fluxo + Weis Wave 1P).              |
//|                                                                  |
//| v2.43 — ABS: POC_FOLLOWING DIRECIONAL (cirurgia conservadora): |
//| [REA-1] MatrixSnapshot ganha absFollowingReaction e              |
//|      absFollowingExtension:                                      |
//|      REACTION = windowsValid && POC_FOLLOWING &&                 |
//|              displacementOk && moveDir == dir                    |
//|      EXTENSION = windowsValid && POC_FOLLOWING &&                |
//|              displacementOk && moveDir == -dir                   |
//|      SEM waveDir fechado como veto, SEM candleDir,               |
//|      displacementOk INALTERADO.                                  |
//| [REA-2] absPocContextOk = DIVERGENCE || STALL_STRICT ||          |
//|      FOLLOWING_REACTION. c1 = c1_effort_ok && absPocContextOk|
//|      (esforço WW SEMPRE obrigatório). FOLLOWING_EXTENSION e |
//|      POC_INVALID NUNCA validam ABS. BASE/TURBO/scoreEff          |
//|      INALTERADOS (C1 continua obrigatório).                      |
//| [REA-3] absCtxTag: + ABS_CONTEXT_REACTION_FOLLOWING.
|
//|      ABS_FOLLOWING_EXTENSION_BLOCK = log/shadow diagnóstico |
//|      (NÃO é gate novo).                                          |
//| [REA-4] FLOW_REGIME (FLOW_WEAK/FLOW_STALL/FLOW_REACTION/
|
//|      FLOW_EXTENSION), VolRatio, PriceEfficiency, PocResponse:|
//|      SOMENTE telemetria. Sem threshold. Não votam na matriz. |
//| [REA-5] POSITION_SHADOW: com posição aberta o gate               |
//|      HasOpenPosition() é PRESERVADO (return mantido); antes |
//|      do return, a nova M5 da sessão é avaliada READ-ONLY via |
//|      ComputeMatrixSnapshot(). NÃO arma, NÃO envia, NÃO altera|
//|      posição/BE/SL/TP/cota/gLastMatrixEvalBar/persistência. |
//| [REA-6] CSV principal: _Audit_v243.csv (44 colunas INALTERADAS).|
//|      CSV auxiliar: _Shadow_v243.csv (75 colunas v2.42 + 7 |
//|      colunas de fluxo ANEXADAS AO FINAL).                        |
//| CONT, Glass, Estágio 2, risco, gestão, WW, POC: idênticos v2.42.|
//|                                                                  |
//| v2.42 — OBSERVABILIDADE DO ARMED + M5 NÃO PERDIDA (cirurgia): |
//| [OBS-1] ProcessArmedSignal() retorna ENUM_ARMED_PROCESS_RESULT |
//|      (STILL_ARMED / ENDED_REEVAL_ALLOWED / ENDED_FAIL_CLOSED |
//|      / TRADE_OR_POSITION). TIMEOUT_BREAKOUT => REEVAL: a nova|
//|      M5 em que o sinal expirou segue pelos gates normais e |
//|      passa UMA ÚNICA vez por EvaluateMatrix(). Integridade, |
//|      referência temporal, fim de sessão, bloqueios               |
//|      protetivos e falha de ordem => FAIL_CLOSED.                 |
//| [OBS-2] Guarda gLastMatrixEvalBar: a mesma M5 nunca é avaliada |
```

```
//|      operacionalmente duas vezes.                                  |
//| [OBS-3] Cálculo da matriz EXTRAÍDO (sem alterar equações) para |
//|      ComputeMatrixSnapshot() — função PURA usada pela                  |
//|      EvaluateMatrix operacional e pela avaliação SHADOW. |
//| [OBS-4] SHADOW READ-ONLY: com Armed ativo, a cada nova M5 calcula|
//|      ABS e CONT e grava ARMED_SHADOW + flags diagnósticas |
//|      OPPOSITE_ROUTE_VALID / SAME_ROUTE_NEW_VALID /                                  |
//|      ARMED_CONTEXT_STILL_VALID / ROUTE_CONFLICT_SHADOW.
|
//|      NÃO arma, NÃO cancela, NÃO envia, NÃO persiste.               |
//| [OBS-5] STALL_RAW = windowsValid && POC_NO_DISPLACEMENT (sem |
//|      body_ok) — SOMENTE telemetria (ABS_STALL_RAW_ONLY). |
//|      C1 ABS operacional INALTERADO:                    |
//|      effort && (DIVERGENCE || STALL_STRICT).                   |
//| [OBS-6] ROUTE_CONFLICT_BLOCK preservado (bloqueia); detalhamento|
//|      das duas rotas em ROUTE_CONFLICT_SHADOW (CSV auxiliar). |
//| [OBS-7] CSV principal: _Audit_v242.csv (44 colunas INALTERADAS).|
//|      CSV auxiliar versionado: _Shadow_v242.csv.              |
//| ABS, CONT, Glass, Estágio 2, risco e gestão: idênticos à v2.41. |
//| Continua existindo UM ÚNICO ArmedSignal operacional.                |
//|                                  |
//| v2.41 — ABS POR CONTEXTO + GLASS POR ROTA (cirurgia mínima): |
//| [ABS-1] Glass NÃO executa mais RETURN global no Estágio 1.                  |
//|      C5_ABS = dir != 0. Glass = EXTREMO => ABS continua                |
//|      investigando absorção (ABS_GLASS_OBSERVE). Glass                       |
//|      sozinho NUNCA arma.                        |
//|      C5_CONT = dirCont != 0 && !glass (proteção anti-          |
//|      perseguição preservada para a CONT).                |
//|      Somente CONT habilitada + Glass => KAMIKAZE_GLASS e |
//|      return (comportamento idêntico ao v2.40).          |
//| [ABS-2] C1 ABS = esforço && (DIVERGENCE || STALL):                      |
//|      DIVERGENCE = windowsValid && POC_DIVERGENT                             |
//|      STALL      = windowsValid && POC_NO_DISPLACEMENT                        |
//|               && corpo travado (body_ok)           |
//|      BODY é obrigatório SOMENTE no STALL. Displacement,                  |
//|      moveDir e waveDir FECHADO deixam de ser VETO do C1 ABS |
//|      (continuam calculados e gravados com valor REAL).            |
//|      POC_FOLLOWING e POC_INVALID NUNCA validam ABS.                                 |
//|      (v2.43: FOLLOWING_REACTION passa a validar; ver REA-2.) |
//| [ABS-3] Estágio 2: Glass DIRECIONAL. CONT: LATE_GLASS_BLOCK |
//|      intacto. ABS: bloqueia somente Glass do lado OPOSTO ao |
//|      extremo original; Glass do MESMO lado =>              |
//|      ABS_LATE_GLASS_SAMESIDE_ALLOW (validação continua). |
//| [ABS-4] ONDA VIVA (waveDir == armed.dir) e esforço vivo               |
//|      (>= armed.breakoutEffortMin) continuam OBRIGATÓRIOS. |
//| [ABS-5] CSV renomeado para _Audit_v241.csv (44 colunas).                  |
//| CONT: equações matematicamente idênticas à v2.40.                   |
//| RISCO RESIDUAL CONHECIDO (v2.41): ArmedSignal ÚNICO — uma rota |
```

```
//| armada impede temporariamente a avaliação da outra (v2.42: agora|
//| OBSERVÁVEL via SHADOW; arquitetura NÃO reconstruída).          |
//|                                                                |
//| v2.40 — DUAS ROTAS PARALELAS (ADITIVO):                        |
//| ROTA A (ABS) = Absorção/Reversão (v2.41: contexto DIV/STALL). |
//| ROTA B (CONT) = Continuação/Direcional.                        |
//| Mesma infraestrutura: VWAP, Sigma, POC, C2/C3/C4, região,      |
//| breakout, risco e tiro diário único COMPARTILHADO.             |
//|                                                                |
//| CORREÇÃO CIRÚRGICA v2.40 (histórico):                          |
//| [FIX-1] Novo input Inp_Cont_Min_Displ_Sigma (deslocamento      |
//|      mínimo PRÓPRIO da CONT, independente da ABS).             |
//| [FIX-2] CONT com gate de deslocamento próprio, contMoveDir e   |
//|      contPocState (mesmas janelas, mesma fórmula               |
//|      Inp_POC_FollowRatio).                                     |
//| [FIX-3] ROUTE_CONFLICT_BLOCK: ABS e CONT válidas juntas =>     |
//|      entrada BLOQUEADA (sem prioridade arbitrária).            |
//| [FIX-4] CSV versionado (44 colunas).                           |
//| [FIX-5] Validação de inputs por rota no OnInit.                |
//|                                                                |
//| FLUXO OPERACIONAL (2 ESTÁGIOS) — REGIÃO DE ANOMALIA:           |
//| ESTÁGIO 1 (M5 - "ARMAR"): Avalia a MATRIZ HONESTA no           |
//| fechamento do M5. Responde: existe CONTEXTO institucional?     |
//| Se válido, NÃO executa: ARMA o gatilho e mapeia a MÁXIMA e a   |
//| MÍNIMA absolutas de toda a REGIÃO (Inp_Region_Candles).         |
//| ESTÁGIO 2 (TICK - "DISPARAR"): Monitora tick a tick.           |
//| - Janela: até Inp_Breakout_Max_Bars p/ romper a região.        |
//| - VENDA: last <= trigger (regionLow - margem, floor tick).     |
//| - COMPRA: last >= trigger (regionHigh + margem, ceil tick).    |
//| - Confirma FLUXO a favor (ONDA VIVA Weis Wave, cache O(1))     |
//| E FORÇA da ONDA VIVA >= armed.breakoutEffortMin                |
//| (ABS: Inp_Breakout_Effort | CONT: Inp_Cont_Breakout_Effort)    |
//|                                                                |
//| MATRIZ HONESTA — ROTA A (ABS, v2.43):                          |
//| C5_ABS = PORTÃO (dir != 0). Glass NÃO invalida ABS. NÃO vota.  |
//| C1 = PILAR MESTRE OBRIGATÓRIO = esforço (Inp_Vol_Effort) &&    |
//|    (DIVERGENCE || STALL || FOLLOWING_REACTION).                |
//|    FOLLOWING_EXTENSION NÃO valida. Corpo/deslocamento/waveDir  |
//|    fechado = TELEMETRIA (corpo obrigatório só no STALL).       |
//| C23 = (C2 || C3) e C4 = confirmações.                          |
//| BASE = c1 && (c23 || c4) | TURBO = c1 && c23 && c4.            |
//|                                                                |
//| MATRIZ HONESTA — ROTA B (CONT, v2.40 preservada):              |
//| dirCont segue a extensão (acima => BUY | abaixo => SELL).       |
//| C5_CONT = dirCont != 0 && !glass.                              |
//| C1_CONT = esforço (Inp_Cont_Vol_Effort) + deslocamento PRÓPRIO |
//| (Inp_Cont_Min_Displ_Sigma) + contPocState==POC_FOLLOWING +     |
//| contMoveDir==dirCont + waveDir==dirCont + vela na direção.     |
```

```
//| SEM teto de corpo (MaxBody pertence só à absorção).              |
//| C23_CONT / C4_CONT espelhados. Mesma lógica 2/3 e 3/3.           |
//| v2.41: as rotas NÃO são mais mutuamente exclusivas por           |
//| construção (ABS STALL x CONT FOLLOWING podem coexistir quando |
//| Cont_Min_Displ <= |PriceMove| < Min_Displ). O conflito           |
//| simultâneo é BLOQUEADO (ROUTE_CONFLICT_BLOCK), sem prioridade. |
//| v2.43: ABS FOLLOWING_REACTION (moveDir==dir) e CONT
|
//| (contMoveDir==dirCont==-dir) são mutuamente exclusivas por       |
//| construção (mesmo sinal de PriceMove).                           |
//|                                                                  |
//| TELEMETRIA DE ROTA: o campo MODE do CSV/ordem/deals carrega
|
//| "BASE"/"TURBO" (ABS) ou "BASE_CONT"/"TURBO_CONT" (CONT).
|
//| CSV: 44 colunas fixas. Para ABS o snapshot C1 usa:               |
//| C1_EFFORT_OK       = c1_effort_ok                                |
//| C1_BODY_OK         = corpo <= MaxBody (REAL; veto só no STALL) |
//| C1_DISPLACEMENT_OK = displacementOk (REAL; telemetria)
|
//| C1_POC_OK          = absPocContextOk                             |
//|          (DIVERGENCE || STALL || FOLLOWING_REACTION)|
//| C1_MOVEDIR_OK      = moveDir == -dir (REAL; telemetria)          |
//| C1_WAVEDIR_OK      = waveDir == moveDir (REAL; telemetria)       |
//| Contexto: C1_POC_STATE=POC_DIVERGENT => DIVERGENCE;
|
//|        POC_NO_DISPLACEMENT + BODY_OK=1 => STALL;
|
//|        POC_FOLLOWING + C1_POC_OK=1 => REACTION_FOLLOWING. |
//| Para CONT o snapshot C1 usa:                                     |
//| C1_EFFORT_OK       = cont_effort_ok                              |
//| C1_BODY_OK         = cont_candle_dir_ok (vela na direção)        |
//| C1_DISPLACEMENT_OK = contDisplacementOk                          |
//| C1_POC_OK          = contPocState == POC_FOLLOWING               |
//| C1_MOVEDIR_OK      = contMoveDir == dirCont                      |
//| C1_WAVEDIR_OK      = waveDir == dirCont                          |
//| Corpo em sigma = CANDLE_BODY / SIGMA (colunas existentes).
|
//|                                                                  |
//| ENDURECIMENTO OPERACIONAL (E1..E7):                              |
//| E1 = D-1 LITERAL: VWAP_D1/H1/L1 vêm EXCLUSIVAMENTE do último |
//|    pregão concluído imediatamente anterior. PROIBIDO usar |
//|    D-2/D-3 como estrutura e PROIBIDO o fallback OHLC             |
//|    (H+L+C)/3 como VWAP oficial. D-1 inválido = sem entradas. |
//| E2 = Separação de módulos: CalcD1Structure() x CalcSigmaHistory |
//| E3 [2.435] = D-1/Sigma obrigatorios; retry sem aborto horario. |
//| E4 = iBarShift()<0 => ABORT_TIME_REF_INVALID.                    |
//| E5 = Tiro diário: DONE/PLACED persistido imediatamente e         |
```

```
//|     restaurado no OnInit como max(histórico, persistido).                    |
//| E6 = Sinal armado NUNCA é persistido: reinício => reset.                     |
//| E7 = Revalidação final dentro de SendOrder() imediatamente                   |
//|     antes de PositionOpen (LATE_*_BLOCK).                                    |
//+------------------------------------------------------------------+
#property copyright "Remora 2.0 Híbrida"
#property version "2.436"
#property description "REMORA 2.436 - candidata Hard Directional; VWAP 2.435
preservada; MT5 PENDENTE"
#include <Trade\Trade.mqh>
#include <Trade\PositionInfo.mqh>

//+------------------------------------------------------------------+
// INPUTS — IDENTIDADE / INFRA
// DEFAULTS OFICIAIS: WIN/IND. Para WDO/DOL usar arquivo .set:
// MagicCore=20001; SpreadMaxPts=1.5; Inp_Max_SL_Pts=25.0;
// Inp_MaxDeviationPts=2; Inp_Breakout_Margin=2.0
//+------------------------------------------------------------------+
input int    MagicCore = 50001;         // Magic Number (WIN=50001 | WDO=20001 via .set)
input string CSVFileName = "Remora20"; // Prefixo do CSV de auditoria
input datetime BacktestStart = D'2025.01.01';

//--- SESSÃO ---
input int  Inp_StartHHMM = 900;     // Início das entradas (HHMM)
input int  Inp_StopEntryHHMM = 1530; // Fim das entradas (até 15:30)
input int  Inp_EOD_CloseHHMM = 1750; // Zeragem compulsória (HHMM)

//+------------------------------------------------------------------+
// [RISCO E GESTÃO] — herdado da Remora 01 (3.6.0)
//+------------------------------------------------------------------+
input int  Inp_MaxTradesDay = 1;      // TIRO ÚNICO DIÁRIO (V1: acertou/errou, encerra)
input double MargemPorContrato = 350.0; // Margem por contrato (sizing MarginOk)
input int  Inp_MaxLotes   = 300;   // Lote máximo permitido

//--- RISCO E GESTÃO DINÂMICA (Multiplicadores de Sigma) ---
input double Inp_Mult_SL    = 1.0;    // Mult. Sigma p/ Stop Loss
input double Inp_Max_SL_Pts = 500.0; // TETO MÁXIMO do SL (WIN=500 | WDO=25 via
.set)
input double Inp_Mult_TP_Base = 2.0;      // Mult. Sigma p/ Take Profit BASE
input double Inp_Mult_TP_Turbo = 3.5;     // Mult. Sigma p/ Take Profit TURBO

//--- GERENCIAMENTO DE PROTEÇÃO EM DEGRAUS (Escada de Lucro) ---
input double Inp_Mult_BE     = 1.0; // 1. Mult. Sigma p/ Gatilho Zero a Zero
input double Inp_Mult_Lock1_Trig = 2.0; // 2. Mult. Sigma p/ Gatilho Lock 1 (Ex: Andou 2x)
input double Inp_Mult_Lock1_Prof = 1.0; // 2. Mult. Sigma p/ Lucro Lock 1 (Ex: Protege 1x)
input double Inp_Mult_Lock2_Trig = 3.0; // 3. Mult. Sigma p/ Gatilho Lock 2 (Ex: Andou 3x)
input double Inp_Mult_Lock2_Prof = 2.0; // 3. Mult. Sigma p/ Lucro Lock 2 (Ex: Protege 2x)
input double SpreadMaxPts      = 5.0; // Spread máximo (WIN=5.0 | WDO=1.5 via .set)
```

```
//====================================================================
// [FLUXO E ABSORÇÃO — WW1P / DIVERGÊNCIA]
//====================================================================
input ENUM_TIMEFRAMES Inp_Timeframe = PERIOD_M5; // Estrutura + Fluxo
input double Inp_Vol_Effort      = 1.5; // Mult. Esforço WW1P (Onda > Média*X)
input double Inp_Min_Displ_Sigma = 0.20; // Mult. Sigma: piso do deslocamento (classifica POC ABS: NO_DISPLACEMENT x DIVERGENT/FOLLOWING)
input double Inp_Max_Body_Sigma = 0.20; // Mult. Sigma: teto do corpo da M5 (C1 ABS: obrigatório só no contexto STALL)
input int    Inp_Inertia_Timeout = 180; // Seg. (janela p/ módulo divergência POC)
input int    Inp_Breakout_Max_Bars = 3; // Máx. velas p/ confirmar rompimento
input double Inp_POC_AnchorMult = 1.5; // Tolerância ancoragem da POC ao preço (telemetria)
input double Inp_POC_FollowRatio = 0.5; // % que a POC deve seguir o preço

//====================================================================
// [REGIÃO DE ANOMALIA — CLUSTER DE ABSORÇÃO / VALIDAÇÃO DE ROMPIMENTO]
//====================================================================
input int  Inp_Region_Candles       = 3; // Quantidade de candles p/ mapear a região de anomalia
input double Inp_Breakout_Margin      = 5.0; // Margem em PONTOS além do extremo (WIN=5.0 | WDO=2.0 via .set)
input double Inp_Max_Trigger_Overshoot = 50.0; // [BLINDAGEM E2] Ultrapassagem máxima permitida além do trigger (WIN/IND=50 | WDO/DOL=3 via painel/.set)
input double Inp_Breakout_Effort     = 1.5; // Mult. esforço mínimo da ONDA VIVA (WW1P) exigido no rompimento (ROTA ABS)

//====================================================================
// [ROTAS — ABSORÇÃO x CONTINUAÇÃO] (v2.40, aditivo)
//====================================================================
input bool Inp_EnableAbsorption       = true; // ROTA A: Absorção/Reversão habilitada
input bool Inp_EnableContinuation      = true; // ROTA B: Continuação/Direcional habilitada
input double Inp_Cont_Vol_Effort      = 1.25; // CONT: Mult. Esforço WW1P mínimo (C1_CONT)
input double Inp_Cont_Min_Displ_Sigma      = 0.15; // CONT: Mult. Sigma: piso MINIMO do deslocamento do preço (C1_CONT) [FIX-1]
input bool Inp_Cont_RequireCandleDirection = true; // CONT: vela M5 fechada deve ter a direção da operação
input double Inp_Cont_Breakout_Effort    = 1.50; // CONT: esforço mínimo da ONDA VIVA no rompimento

//====================================================================
// [GEOMETRIA — VWAP D-1 E BANDS]
//====================================================================
input double BandMultiplier = 1.0; // Sigma * mult => meia-banda (C5)
input double Inp_Rule_40_60 = 0.40; // Filtro Extremo Range D-1 (C4)
```

```
//--- DISTÂNCIAS FIXAS DA VWAP D-1 ---
input double Inp_Pts_C2_Longa = 700.0; // C2: distância longa mínima em pontos (preset
WIN/IND; WDO/DOL=25 via .set)
input double Inp_Pts_C3_Min = 125.0; // C3: distância curta mínima em pontos (preset
WIN/IND; WDO/DOL=5 via .set)
input double Inp_Pts_C3_Max = 400.0; // C3: distância curta máxima em pontos (preset
WIN/IND; WDO/DOL=15 via .set)
input double Inp_VWAP_Inner_Pct = 0.60; // Entrada (Banda 1)
input double Inp_VWAP_Outer_Pct = 1.50; // Limite Kamikaze (Banda 2)

//+------------------------------------------------------------------+
// SIZING / SIGMA (herdado 3.6.0)
//+------------------------------------------------------------------+
input int   SigmaLookbackDays = 5;      // Dias para média do Sigma D1
input double Inp_SigmaFloor_WIN = 200.0; // Piso mínimo do Sigma para WIN/IND
input double Inp_SigmaFloor_WDO = 7.0; // Piso mínimo do Sigma para WDO/DOL
input int   Inp_MaxDeviationPts = 10; // Slippage máx. (WIN=10 | WDO=2 via .set)
input int   Inp_CooldownBars = 3;     // Cooldown (barras) após trade

//+------------------------------------------------------------------+
// RECONSTRUÇÃO D-1 VIA M1 (núcleo impecável intocado)
//+------------------------------------------------------------------+
const int MAX_TRADES_HARD = 50;
const int D1_SESSION_START = 900;
const int D1_SESSION_END = 1825;
const int D1_MIN_M1_BARS = 480;
const int MAX_SEARCH_DAYS = 40;

//+------------------------------------------------------------------+
// OBJETOS / GLOBAIS
//+------------------------------------------------------------------+
CTrade      trade;
CPositionInfo posInfo;
int      csvHandle = INVALID_HANDLE;
int      shadowHandle = INVALID_HANDLE; // [v2.43] CSV auxiliar _Shadow_v243.csv

double H1 = 0.0, L1 = 0.0, VWAP_D1 = 0.0, SigmaD1 = 0.0;
bool VWAP_D1_Official = false;
double UpperBand1 = 0.0, LowerBand1 = 0.0;

int    ordersEnteredToday = 0;
// [2.43.1/S1] Estado da evidência separado da quantidade reconhecida.
enum ENUM_QUOTA_HISTORY { HISTORICO_OK, HISTORICO_INDISPONIVEL };
enum ENUM_QUOTA_STATE { COTA_CONFIRMADA, COTA_DESCONHECIDA };
ENUM_QUOTA_HISTORY gQuotaHistoryState = HISTORICO_INDISPONIVEL;
ENUM_QUOTA_STATE gQuotaState = COTA_DESCONHECIDA;
datetime gQuotaNextRetry = 0;
```

```
bool gQuotaPersistPending = false;
bool gQuotaUnknownLogged = false;
datetime lastBarTime = 0;
datetime lastEntryTime = 0;
datetime gLastMatrixEvalBar = 0;          // [v2.42] guarda: 1 EvaluateMatrix por M5

//--- Identidade do ativo / conta (Cirurgias 11/12)
bool gIsIndexFamily = false; // WIN/IND
bool gIsDollarFamily = false; // WDO/DOL
bool gNetting       = false; // Netting/Exchange => uma Rêmora por símbolo por conta
bool foreignAlertActive = false;
double gTickSize       = 1.0;

//--- Reset diário por chave (Cirurgia 13.2)
int gDayKey = -1;

//--- D-1 resiliente (Cirurgia 13.3) + D-1 LITERAL (E1)
bool d1Ready = false;
datetime nextD1Retry = 0;
datetime gD1ExpectedDate = 0; // [E1] meia-noite do D-1 literal identificado

//--- EOD no OnTimer (Cirurgia 13.5)
bool eodDone       = false;
bool eodStuckLogged = false;
datetime eodNextTry = 0;

//--- Break Even (Cirurgia 13.4)
struct BERecord { ulong ticket; bool applied; datetime backoffUntil; };
BERecord beList[64];
int     beCount = 0;
bool beSyncPending = false;

//--- Cache VWAP vigente incremental (Cirurgia 7)
double vwapSumPV       = 0.0;
double vwapSumVol      = 0.0;
ulong vwapNextFromMsc = 0;
bool vwapReady       = false;
int vwapDayKey      = -1;
double gVwapValue     = 0.0; // publicado por nova M5
bool gVwapOk        = false; // validade nesta M5

//--- Cache Weis Wave fechado O(1) (Cirurgia 8)
bool wwValid      = false;
bool wwUseReal       = false;
int wwDirClosed = 0;
double wwAccumClosed = 0.0;
double wwAvgClosed = 0.0;
double wwAvgIfFlip = 0.0;
```

```
double wwCloseRef = 0.0;
double wwEffortClosed = 0.0;
int wwWaveDirClosed = 0;

//+------------------------------------------------------------------+
//| [v2.42] RESULTADO OPERACIONAL DO ESTÁGIO 2                       |
//| APS_STILL_ARMED                : sinal segue aguardando confirmação. |
//| APS_ENDED_REEVAL_ALLOWED : terminou SEM operação e sem condição |
//|                  fail-closed (TIMEOUT_BREAKOUT). Em |
//|                  nova M5, segue p/ gates normais e |
//|                  EvaluateMatrix() UMA única vez.          |
//| APS_ENDED_FAIL_CLOSED : integridade/referência/sessão/           |
//|                  bloqueio protetivo/falha de ordem. |
//|                  NÃO reavalia neste evento.               |
//| APS_TRADE_OR_POSITION : ordem aceita / posição / cota.           |
//|                  NÃO reavalia.                    |
//+------------------------------------------------------------------+
enum ENUM_ARMED_PROCESS_RESULT
{
   APS_STILL_ARMED = 0,
   APS_ENDED_REEVAL_ALLOWED,
   APS_ENDED_FAIL_CLOSED,
   APS_TRADE_OR_POSITION
};

//+------------------------------------------------------------------+
//| [v2.42] STATUS DO SNAPSHOT PURO DA MATRIZ                        |
//+------------------------------------------------------------------+
enum ENUM_MATRIX_SNAP_STATUS
{
  MSS_NOT_COMPUTED = 0,
  MSS_OK,
  MSS_VWAP_D1_INVALID,
  MSS_SIGMA_INVALID,
  MSS_STRUCT_INVALID,                // H1/L1, VWAP vigente, range ou preço inválidos
  MSS_STOP_KAMIKAZE_CONT_ONLY, // Glass com ABS desabilitada (v2.41:
KAMIKAZE_GLASS)
  MSS_STOP_NO_C5_TRIGGER,            // dir==0 && dirCont==0
  MSS_SHADOW_DATA_INVALID            // shadow: D-1/VWAP/Weis indisponíveis
nesta M5
};

//+------------------------------------------------------------------+
//| CIRURGIA 5 — SNAPSHOT DE TELEMETRIA C1                           |
//| Para a rota CONT, os campos assumem a semântica descrita no      |
//| cabeçalho (body_ok = vela na direção da operação).               |
//| v2.41 ABS: poc_ok = absPocContextOk; body/displacement/moveDir/ |
//| waveDir carregam o valor REAL (telemetria honesta).              |
```

```
//+------------------------------------------------------------------+
struct C1Audit
{
   bool valid;
   int effort_ok;
   int body_ok;
   int displacement_ok;
   int poc_ok;
   int movedir_ok;
   int wavedir_ok;
   double candleBody;
   double maxDisplThreshold;
   double priceMove;
   double pocMove;
   int moveDir;
   int waveDir;
   string pocState;
   double volNow;
   double volPrev;
   double pocNow;
   double pocPrev;
   double lastPriceNow;
   double lastPricePrev;
   int anchored;
   int c23;
   int scoreEff;
};
C1Audit gC1Audit;

void ResetC1AuditSnapshot(C1Audit &a)
{
  a.valid       = false;
  a.effort_ok    = 0;
  a.body_ok        = 0;
  a.displacement_ok = 0;
  a.poc_ok        = 0;
  a.movedir_ok      = 0;
  a.wavedir_ok      = 0;
  a.candleBody       = 0.0;
  a.maxDisplThreshold = 0.0;
  a.priceMove       = 0.0;
  a.pocMove         = 0.0;
  a.moveDir        = 0;
  a.waveDir       = 0;
  a.pocState       = "NA";
  a.volNow        = 0.0;
  a.volPrev      = 0.0;
  a.pocNow         = 0.0;
```

```
  a.pocPrev        = 0.0;
  a.lastPriceNow     = 0.0;
    a.lastPricePrev = 0.0;
    a.anchored        = 0;
    a.c23          = 0;
    a.scoreEff      = 0;
}

//+------------------------------------------------------------------+
//| [v2.42] SNAPSHOT PURO DA MATRIZ (ABS + CONT)                     |
//| Produzido por ComputeMatrixSnapshot() — MESMAS equações v2.41. |
//| Consumido pela EvaluateMatrix operacional e pela SHADOW.         |
//| [v2.43] + absFollowingReaction/absFollowingExtension (ABS) e |
//| telemetria de fluxo (flowRegime/volRatio/priceEfficiency/        |
//| pocResponse — SOMENTE diagnóstico).                              |
//+------------------------------------------------------------------+
struct MatrixSnapshot
{
   ENUM_MATRIX_SNAP_STATUS status;
   bool bandsComputed;
   // Geometria / C5
   double vwapVig;
   double upper;
   double lower;
   double bandRange;
   double price;
   double open1;
   double candleBody;
   double posRel;
   bool glass;
   int dir;
   int dirCont;
   bool c5AbsOk;
   bool c5ContOk;
   // C2/C3/C4
   double distAbs;
   double minDispl;
   double maxBody;
   double contMinDispl;
   bool c2;
   bool c3;
   bool c23;
   bool c2Cont;
   bool c3Cont;
   bool c23Cont;
   double posRange;
   bool c4;
   bool c4Cont;
```

```
   // POC / janelas
    double pocNow;
   double pocPrev;
   double vA;
   double vB;
   double lastPriceNow;
   double lastPricePrev;
   bool winNowOk;
   bool winPrevOk;
   bool windowsValid;
   double priceMove;
   double pocMove;
   bool displacementOk;
   int moveDir;
   string pocState;
   bool contDisplacementOk;
   int contMoveDir;
   string contPocState;
   bool anchored;
   // Fluxo fechado
   bool wwok;
   double effort;
   int waveDir;
   // C1 ABS
   bool c1_effort_ok;
   bool c1_body_ok;
   bool c1_displacement_ok;
   bool c1_movedir_ok;
   bool c1_wavedir_ok;
   bool absDivergence;            // DIVERGENCE
   bool absStallStrict;       // STALL_STRICT (operacional)
   bool absStallRaw;            // STALL_RAW (SOMENTE telemetria)
   bool absFollowingReaction; // [v2.43] FOLLOWING na direção da reversão (operacional)
   bool absFollowingExtension; // [v2.43] FOLLOWING aprofundando o extremo (NÃO valida)
   bool absPocContextOk;
   bool c1_poc_ok;
   bool c1;
   bool c1RawHyp;               // effort && (DIV || STALL_RAW || REACTION) — SOMENTE
telemetria
   string absCtxTag;
   // [v2.43] Telemetria de fluxo — SOMENTE diagnóstico (NÃO vota)
   string flowRegime;
   double volRatio;
   double priceEfficiency;
   double pocResponse;
   // C1 CONT
   int candleDir;
   bool cont_effort_ok;
```

```
    bool cont_displacement_ok;
    bool cont_poc_ok;
    bool cont_movedir_ok;
    bool cont_wavedir_ok;
    bool cont_candle_dir_ok;
    bool c1Cont;
    // Classificação
    bool turboAbs;
    bool baseAbs;
    int scoreEff;
    bool turboCont;
    bool baseCont;
    int scoreContEff;
    bool absValid;
    bool contValid;
};

void ResetMatrixSnapshot(MatrixSnapshot &s)
{
  s.status = MSS_NOT_COMPUTED;
  s.bandsComputed = false;
  s.vwapVig = 0.0; s.upper = 0.0; s.lower = 0.0; s.bandRange = 0.0;
  s.price = 0.0; s.open1 = 0.0; s.candleBody = 0.0; s.posRel = 0.0;
  s.glass = false; s.dir = 0; s.dirCont = 0; s.c5AbsOk = false; s.c5ContOk = false;
  s.distAbs = 0.0; s.minDispl = 0.0; s.maxBody = 0.0; s.contMinDispl = 0.0;
  s.c2 = false; s.c3 = false; s.c23 = false;
  s.c2Cont = false; s.c3Cont = false; s.c23Cont = false;
  s.posRange = 0.0; s.c4 = false; s.c4Cont = false;
  s.pocNow = 0.0; s.pocPrev = 0.0; s.vA = 0.0; s.vB = 0.0;
  s.lastPriceNow = 0.0; s.lastPricePrev = 0.0;
  s.winNowOk = false; s.winPrevOk = false; s.windowsValid = false;
  s.priceMove = 0.0; s.pocMove = 0.0; s.displacementOk = false; s.moveDir = 0;
  s.pocState = "NA";
  s.contDisplacementOk = false; s.contMoveDir = 0; s.contPocState = "NA";
  s.anchored = false;
  s.wwok = false; s.effort = 0.0; s.waveDir = 0;
  s.c1_effort_ok = false; s.c1_body_ok = false; s.c1_displacement_ok = false;
  s.c1_movedir_ok = false; s.c1_wavedir_ok = false;
  s.absDivergence = false; s.absStallStrict = false; s.absStallRaw = false;
  s.absFollowingReaction = false; s.absFollowingExtension = false; // [v2.43]
  s.absPocContextOk = false; s.c1_poc_ok = false; s.c1 = false; s.c1RawHyp = false;
  s.absCtxTag = "NA";
  s.flowRegime = "NA"; s.volRatio = 0.0; s.priceEfficiency = 0.0; s.pocResponse = 0.0; //
[v2.43]
  s.candleDir = 0;
  s.cont_effort_ok = false; s.cont_displacement_ok = false; s.cont_poc_ok = false;
  s.cont_movedir_ok = false; s.cont_wavedir_ok = false; s.cont_candle_dir_ok = false;
  s.c1Cont = false;
```

```
   s.turboAbs = false; s.baseAbs = false; s.scoreEff = 0;
   s.turboCont = false; s.baseCont = false; s.scoreContEff = 0;
   s.absValid = false; s.contValid = false;
}

//+------------------------------------------------------------------+
//| ESTÁGIO 2 — ESTRUTURA DO SINAL ARMADO (Região de Anomalia)
|
//| [E6] Este estado é VOLÁTIL por política: NUNCA é persistido. |
//| Reinício do EA => ResetArmedSignal() no OnInit, sem reconstrução.|
//| v2.40: UMA ÚNICA estrutura para as duas rotas (route/effortMin). |
//| v2.41: flag anti-spam do log ABS_LATE_GLASS_SAMESIDE_ALLOW.                    |
//+------------------------------------------------------------------+
struct ArmedSignal
{
  bool active;             // Sinal armado aguardando reação
  int    dir;          // +1 compra / -1 venda
  double regionHigh;              // Máxima absoluta calculada de toda a região de anomalia
  double regionLow;               // Mínima absoluta calculada de toda a região de anomalia
  double activeSL;              // Stop Loss ativo (Fixo)
  double activeTP;              // Take Profit ativo (BASE ou TURBO)
  datetime regionEndTime; // ABERTURA da última barra fechada usada na região (Cirurgia
10/14)
  string mode;               // "BASE" / "TURBO" (ABS) | "BASE_CONT" / "TURBO_CONT"
(CONT)
  int    lots;          // Lotes fixos calculados por FullSizeLot()
  string route;             // "ABS" ou "CONT"
  double breakoutEffortMin; // Esforço mínimo da ONDA VIVA no Estágio 2 (por rota)
  bool sameSideGlassLogged; // [v2.41] ABS_LATE_GLASS_SAMESIDE_ALLOW já
registrado neste sinal
  // Snapshots de auditoria
  double posRel, distD1, effort, vwapVig;
  int    c1, c2, c3, c4, c5;
  C1Audit audit;              // Snapshot C1 congelado no ARMED (Cirurgia 5)
};
ArmedSignal armed;

//+------------------------------------------------------------------+
//| HELPERS                                                          |
//+------------------------------------------------------------------+
string DirStr(int d){ return d==1 ? "BUY" : (d==-1 ? "SELL" : "NONE"); }

bool IsFokSupported()
{
 ENUM_SYMBOL_TRADE_EXECUTION exec =
  (ENUM_SYMBOL_TRADE_EXECUTION)SymbolInfoInteger(Symbol(),
SYMBOL_TRADE_EXEMODE);
```

```
   // Instant e Request aceitam FOK independentemente do bitmask.
   if(exec == SYMBOL_TRADE_EXECUTION_INSTANT ||
      exec == SYMBOL_TRADE_EXECUTION_REQUEST)
   {
      return true;
   }

   // Market/Exchange: depende da política configurada no símbolo.
   long filling = SymbolInfoInteger(Symbol(), SYMBOL_FILLING_MODE);
   return ((filling & SYMBOL_FILLING_FOK) == SYMBOL_FILLING_FOK);
}

void ResetArmedSignal()
{
  armed.active=false;
  armed.dir=0;
  armed.regionHigh=0.0;
  armed.regionLow=0.0;
  armed.activeSL=0.0;
  armed.activeTP=0.0;
  armed.regionEndTime=0;
  armed.mode="NONE";
  armed.lots=0;
  armed.route="NONE";
  armed.breakoutEffortMin=0.0;
  armed.sameSideGlassLogged=false;
  armed.posRel=0; armed.distD1=0; armed.effort=0; armed.vwapVig=0;
  armed.c1=0; armed.c2=0; armed.c3=0; armed.c4=0; armed.c5=0;
  ResetC1AuditSnapshot(armed.audit); // Cirurgia 5 — limpa snapshot do sinal
}

//+------------------------------------------------------------------+
//| [E3/E5] PERSISTÊNCIA DIÁRIA EM VARIÁVEIS GLOBAIS DO TERMINAL |
//| Chaves específicas da Rêmora 2.0 (NÃO copiar as chaves da 1.0): |
//| [2.435] D1A legado e ignorado, sem apagar variaveis globais. |
//| R2H_<conta>_<símbolo>_<magic>_SHOT_<AAAAMMDD> => tiros aceitos |
//| Persiste SOMENTE a quantidade de tiros aceitos pelo servidor. |
//| DONE/PLACED inalterado. E PROIBIDO persistir sinal               |
//| armado, NO_CONFLUENCE, timeout de região, spread ou rejeições. |
//| Chaves são por data: as do dia anterior são simplesmente          |
//| ignoradas na virada (sem GlobalVariablesDeleteAll amplo).         |
//+------------------------------------------------------------------+
string GvDateToday()
{
   MqlDateTime dt; TimeToStruct(TimeTradeServer(), dt);
   return StringFormat("%04d%02d%02d", dt.year, dt.mon, dt.day);
}
string GvKeyShots()
```

```
{
  return StringFormat("R2H_%I64d_%s_%d_SHOT_%s",
            (long)AccountInfoInteger(ACCOUNT_LOGIN), Symbol(), MagicCore,
GvDateToday());
}




// A presença de um valor inteiro válido é evidência; ausência/erro não é zero.
bool ReadPersistedShotCount(int &count)
{
   double value = 0.0;
   if(!GlobalVariableGet(GvKeyShots(), value)) return false;
   if(!MathIsValidNumber(value) || value < 0.0 || value > 2147483647.0 ||
      value != MathFloor(value)) return false;
   count = (int)value;
   return true;
}

bool PersistShotCount()
{
  if(gQuotaState == COTA_DESCONHECIDA) return false;
  string key = GvKeyShots();
  int previous = 0;
  if(ReadPersistedShotCount(previous))
     ordersEnteredToday = MathMax(ordersEnteredToday, previous);
  datetime r = GlobalVariableSet(key, (double)ordersEnteredToday);
  gQuotaPersistPending = (r == 0);
  if(r == 0)
  {
     PrintFormat("[REMORA2.0][PERSIST_FAIL] Falha ao persistir cota=%d (chave=%s).
Cota preservada em memória; retry pelo timer.", ordersEnteredToday, key);
     return false;
  }
  GlobalVariablesFlush();
  return true;
}

void ReconcileDailyQuota()
{
  int countHistory = 0, countPersisted = 0;
  gQuotaHistoryState = CountTodayTrades(countHistory);
  bool persistenceOK = ReadPersistedShotCount(countPersisted);
  int recognized = ordersEnteredToday;
  if(gQuotaHistoryState == HISTORICO_OK)
     recognized = MathMax(recognized, countHistory);
  if(persistenceOK) recognized = MathMax(recognized, countPersisted);
```

```
  ordersEnteredToday = recognized; // Nunca reduz evidência já reconhecida neste dia.
  // Persistência confirma um mínimo, não a disponibilidade de slots após queda.
  // Sem histórico, um valor abaixo do limite pode estar defasado por falha de IO.
  // Cota já esgotada é segura; slots livres exigem histórico ou memória desta sessão.
  bool known = (gQuotaState == COTA_CONFIRMADA ||
                gQuotaHistoryState == HISTORICO_OK ||
                (persistenceOK && countPersisted >= Inp_MaxTradesDay));
  gQuotaState = known ? COTA_CONFIRMADA : COTA_DESCONHECIDA;
  if(!known)
  {
     if(!gQuotaUnknownLogged)
        Print("[REMORA2.0][QUOTA_UNKNOWN] Cota sem evidência suficiente; somente
novo envio bloqueado. Reconciliação pelo timer.");
     gQuotaUnknownLogged = true;
  }
  else
  {
     if(gQuotaUnknownLogged)
        PrintFormat("[REMORA2.0][QUOTA_RECOVERED] Cota reconciliada=%d. Regras
originais retomadas.", ordersEnteredToday);
     gQuotaUnknownLogged = false;
     if(ordersEnteredToday > 0 || persistenceOK)
        PersistShotCount(); // Não fabrica SHOT=0 na primeira inicialização.
  }
  gQuotaNextRetry = TimeTradeServer() + 5;
}

void RestoreDailyPersistence()
{
  // [2.435/F1] D1A legado nao e lido, recriado nem apagado. SHOT permanece intacto.
  // Dados D-1/Sigma continuam obrigatorios; somente a politica de aborto foi removida.
  ReconcileDailyQuota();
}

//+------------------------------------------------------------------+
//| CIRURGIA 11 — NORMALIZAÇÃO DIRECIONAL POR TICK SIZE              |
//| Snap para múltiplo exato dentro de tolerância; caso contrário    |
//| ceil/floor direcional. NUNCA MathRound no trigger.               |
//+------------------------------------------------------------------+
double NormalizePriceDigits(double p)
{
   return NormalizeDouble(p, (int)SymbolInfoInteger(Symbol(), SYMBOL_DIGITS));
}
double RoundUpToTick(double p)
{
   double r = p / gTickSize;
   double n = MathRound(r);
 if(MathAbs(r - n) < 1e-6) return NormalizePriceDigits(n * gTickSize);
```

```
 return NormalizePriceDigits(MathCeil(r) * gTickSize);
}
double RoundDownToTick(double p)
{
   double r = p / gTickSize;
   double n = MathRound(r);
   if(MathAbs(r - n) < 1e-6) return NormalizePriceDigits(n * gTickSize);
   return NormalizePriceDigits(MathFloor(r) * gTickSize);
}
// TRIGGER: BUY = ceil(regionHigh+margem) | SELL = floor(regionLow-margem)
double ComputeTrigger(int dir, double regionHigh, double regionLow)
{
   if(dir == 1) return RoundUpToTick(regionHigh + Inp_Breakout_Margin);
   return RoundDownToTick(regionLow - Inp_Breakout_Margin);
}
// SL/TP de ENTRADA: preservam ao menos a distância estratégica
double NormEntrySL(int dir, double sl){ return (dir==1) ? RoundDownToTick(sl) :
RoundUpToTick(sl); }
double NormEntryTP(int dir, double tp){ return (dir==1) ? RoundUpToTick(tp) :
RoundDownToTick(tp); }
// SL de PROTEÇÃO (BE/Locks): BUY floor | SELL ceil
double NormProtSL (int dir, double sl){ return (dir==1) ? RoundDownToTick(sl) :
RoundUpToTick(sl); }
long TickIndex(double price){ return (long)MathRound(price / gTickSize); }

//+------------------------------------------------------------------+
//| INTEGRIDADE DE TICKS — FAIL-CLOSED                               |
//| Rejeita erro, timeout ou sincronização parcial.                  |
//+------------------------------------------------------------------+
// [DIAGNOSTICO 4407] Instrumentacao incorporada; nao corrige a recuperacao.
// Somente observacao; sem novos inputs, retries ou alteracao de acumuladores.
// MetaEditor e testes MT5 ainda pendentes.
struct R2436TickDiagStats
{
   ulong calls;
   ulong rejected;
   ulong empty;
   ulong elapsedUsTotal;
   ulong elapsedUsMax;
   ulong suppressed;
   ulong lastLogMs;
   bool logged;
};

// Separate API non-error/rejection buckets: at most one line per minute,
// per call site and outcome. The existing failure log is preserved separately.
R2436TickDiagStats gR2436TickDiagStats[10];
```

```
int R2436DiagSiteIndex(const string callSite)
{
  if(callSite == "VWAP_BOOTSTRAP") return 0;
  if(callSite == "VWAP_INCREMENTAL") return 1;
  if(callSite == "VWAP_LIVE") return 2;
  if(callSite == "POC_WINDOW") return 3;
  return 4; // OTHER: future/unclassified calls.
}

void R2436DiagObserveQuery(const string callSite,
                           const ulong from_msc, const ulong to_msc,
                           const MqlTick &ticks[], const int copied,
                           const int apiError, const ulong elapsedUs,
                           const bool dynamicBefore, const int sizeBefore,
                           const long mqlMemoryBefore,
                           const long terminalMemoryBefore)
{
  bool rejected = (copied < 0 || apiError != 0);
  int slot = 2 * R2436DiagSiteIndex(callSite) + (rejected ? 1 : 0);
  gR2436TickDiagStats[slot].calls++;
  if(rejected) gR2436TickDiagStats[slot].rejected++;
  if(copied == 0) gR2436TickDiagStats[slot].empty++;
  gR2436TickDiagStats[slot].elapsedUsTotal += elapsedUs;
  if(elapsedUs > gR2436TickDiagStats[slot].elapsedUsMax)
    gR2436TickDiagStats[slot].elapsedUsMax = elapsedUs;

  ulong nowMs = GetTickCount64();
  if(gR2436TickDiagStats[slot].logged &&
    nowMs - gR2436TickDiagStats[slot].lastLogMs < 60000)
  {
    gR2436TickDiagStats[slot].suppressed++;
    return;
  }
  gR2436TickDiagStats[slot].logged = true;
  gR2436TickDiagStats[slot].lastLogMs = nowMs;

  int sizeAfter = ArraySize(ticks);
  int readable = 0;
  if(copied > 0 && sizeAfter > 0)
    readable = (copied < sizeAfter) ? copied : sizeAfter;
  long firstMsc = 0, lastMsc = 0;
  if(readable > 0)
  {
    firstMsc = ticks[0].time_msc;
    lastMsc = ticks[readable - 1].time_msc;
  }
  // Endpoints and synchronization are observations, NOT completeness proofs.
  // No scan, sorting, deduplication or acceptance of partial ticks is added.
```

```
   PrintFormat("[REMORA2.436][TICKS_DIAG] Site=%s Symbol=%s Magic=%d TF=%s "
               "From=%I64u To=%I64u Copied=%d Error=%d Disposition=%s "
               "DynamicBefore=%d DynamicAfter=%d SizeBefore=%d SizeAfter=%d "
               "Readable=%d FirstMsc=%I64d LastMsc=%I64d ApiUs=%I64u "
               "MqlMemBeforeMB=%I64d MqlMemAfterMB=%I64d "
               "TerminalMemBeforeMB=%I64d TerminalMemAfterMB=%I64d "
               "Build=%I64d Connected=%d SymbolSynchronized=%d "
               "CacheReadyBeforeCaller=%d NextBeforeCaller=%I64u "
               "PVBeforeCaller=%.17g VolBeforeCaller=%.17g "
               "PublishedOkBeforeCaller=%d PublishedVwapBeforeCaller=%.17g "
               "CallsBucket=%I64u RejectedBucket=%I64u EmptyBucket=%I64u "
               "ApiUsTotalBucket=%I64u ApiUsMaxBucket=%I64u SuppressedBucket=%I64u",
               callSite, Symbol(), MagicCore, EnumToString(Inp_Timeframe),
               from_msc, to_msc, copied, apiError,
               rejected ? "REJECT" : "API_NO_ERROR",
               (int)dynamicBefore, (int)ArrayIsDynamic(ticks), sizeBefore, sizeAfter,
               readable, firstMsc, lastMsc, elapsedUs,
               mqlMemoryBefore, (long)MQLInfoInteger(MQL_MEMORY_USED),
               terminalMemoryBefore, (long)TerminalInfoInteger(TERMINAL_MEMORY_USED),
               (long)TerminalInfoInteger(TERMINAL_BUILD),
               (int)TerminalInfoInteger(TERMINAL_CONNECTED),
               (int)SymbolIsSynchronized(Symbol()),
               (int)vwapReady, vwapNextFromMsc, vwapSumPV, vwapSumVol,
               (int)gVwapOk, gVwapValue,
               gR2436TickDiagStats[slot].calls,
               gR2436TickDiagStats[slot].rejected,
               gR2436TickDiagStats[slot].empty,
               gR2436TickDiagStats[slot].elapsedUsTotal,
               gR2436TickDiagStats[slot].elapsedUsMax,
               gR2436TickDiagStats[slot].suppressed);
}

// Called ONLY after OnTick's original gVwapOk/gVwapValue assignments, on newBar.
// Logs actual official sums; never recomputes or modifies them.
void R2436DiagPublishedCache(const datetime barTime)
{
   PrintFormat("[REMORA2.436][VWAP_CACHE_DIAG] Symbol=%s Magic=%d TF=%s "
               "Bar=%s BarMsc=%I64u DayKey=%d Ready=%d "
               "NextExclusive=%I64u PV=%.17g Vol=%.17g "
               "PublishedOk=%d PublishedVwap=%.17g",
               Symbol(), MagicCore, EnumToString(Inp_Timeframe),
               TimeToString(barTime, TIME_DATE|TIME_SECONDS),
               (ulong)barTime * 1000, vwapDayKey, (int)vwapReady,
               vwapNextFromMsc, vwapSumPV, vwapSumVol,
               (int)gVwapOk, gVwapValue);
}

int CopyTradeTicksStrict(ulong from_msc, ulong to_msc, MqlTick &ticksOut[],
```

```
                    const string callSite = "OTHER")
{
  if(to_msc < from_msc)
    return -1;

  bool dynamicBefore = ArrayIsDynamic(ticksOut);
  int sizeBefore = ArraySize(ticksOut);
  long mqlMemoryBefore = MQLInfoInteger(MQL_MEMORY_USED);
  long terminalMemoryBefore = TerminalInfoInteger(TERMINAL_MEMORY_USED);
  ulong startedUs = GetMicrosecondCount();

  ResetLastError();
  int copied = CopyTicksRange(Symbol(), ticksOut, COPY_TICKS_TRADE,
                        from_msc, to_msc);
  int err = GetLastError(); // Capture immediately, before ANY diagnostics.
  ulong elapsedUs = GetMicrosecondCount() - startedUs;

  if(copied < 0 || err != 0)
  {
    // Same existing failure line, parameters and return policy.
    PrintFormat("[REMORA2.0][TICKS_PARTIAL_BLOCK] From=%I64u | To=%I64u |
Copied=%d | Error=%d | Site=%s",
                from_msc, to_msc, copied, err, callSite);
  }

  R2436DiagObserveQuery(callSite, from_msc, to_msc, ticksOut, copied, err,
                    elapsedUs, dynamicBefore, sizeBefore,
                    mqlMemoryBefore, terminalMemoryBefore);

  if(copied < 0 || err != 0)
    return -1;
  return copied;
}

//+------------------------------------------------------------------+
//| CIRURGIA 7 — VWAP VIGENTE INCREMENTAL                            |
//| Âncora: meia-noite do dia da barra. Fechada em t0_msc-1.         |
//| Bootstrap ALL-OR-NOTHING; incremental sem duplicar fronteira.    |
//| (Substitui a legada GetNativeVwapVigente reconstruída a cada M5.)|
//+------------------------------------------------------------------+
void ResetVwapCache()
{
  vwapSumPV          = 0.0;
  vwapSumVol         = 0.0;
  vwapNextFromMsc = 0;
  vwapReady          = false;
  vwapDayKey         = -1;
  gVwapValue         = 0.0;
```

```
   gVwapOk             = false;
}

bool UpdateVwapIncremental(datetime t0)
{
  MqlDateTime d; TimeToStruct(t0, d);
  int key = d.year*1000 + d.day_of_year;
  if(key != vwapDayKey)
  {
     // Novo dia (safety-net; o reset diário oficial ocorre no OnTimer)
     vwapSumPV=0.0; vwapSumVol=0.0; vwapNextFromMsc=0; vwapReady=false;
     vwapDayKey = key;
  }

    ulong t0_msc = (ulong)t0 * 1000;

    //--- BOOTSTRAP ALL-OR-NOTHING ------------------------------------
    if(!vwapReady)
    {
       d.hour=0; d.min=0; d.sec=0;
       datetime dayStart = StructToTime(d);
       ulong from = (ulong)dayStart * 1000;

     if(t0_msc <= from)
     {
        vwapSumPV=0.0; vwapSumVol=0.0; vwapReady=false; vwapNextFromMsc=0;
        return false;
     }

     ulong to = t0_msc - 1;
     MqlTick tk[];
     int copied = CopyTradeTicksStrict(from, to, tk, "VWAP_BOOTSTRAP");
    if(copied <= 0)
    {
       vwapSumPV=0.0; vwapSumVol=0.0; vwapReady=false; vwapNextFromMsc=0;
       return false; // próxima nova M5 repete o bootstrap completo desde 00:00
    }

    double pv=0.0, vv=0.0; bool any=false;
    for(int i=0; i<copied; i++)
    {
       if(tk[i].last > 0.0 && tk[i].volume > 0)
       {
          pv += tk[i].last * (double)tk[i].volume;
          vv += (double)tk[i].volume;
          any = true;
       }
    }
```

```
   if(!any || vv <= 0.0)
   {
      vwapSumPV=0.0; vwapSumVol=0.0; vwapReady=false; vwapNextFromMsc=0;
      return false;
   }

   // Sucesso integral: só agora copia para os acumuladores globais
   vwapSumPV        = pv;
   vwapSumVol       = vv;
   vwapReady        = true;
   vwapNextFromMsc = t0_msc;
   return true;
}

//--- INCREMENTAL ---------------------------------------------------
if(vwapNextFromMsc >= t0_msc)
  return (vwapSumVol > 0.0); // sem intervalo novo: retorna VWAP atual

ulong from = vwapNextFromMsc;
ulong to = t0_msc - 1;
MqlTick tk[];
int copied = CopyTradeTicksStrict(from, to, tk, "VWAP_INCREMENTAL");
if(copied < 0)
{
  // Erro de cópia: não altera somas, não avança fronteira.
  // Próxima M5 tentará TODO o intervalo pendente.
  return false;
}
if(copied == 0)
{
  // Intervalo legítimo sem negócios
  vwapNextFromMsc = t0_msc;
      return (vwapSumVol > 0.0);
   }

   double pv=0.0, vv=0.0;
   for(int i=0; i<copied; i++)
   {
     if(tk[i].last > 0.0 && tk[i].volume > 0)
     {
        pv += tk[i].last * (double)tk[i].volume;
        vv += (double)tk[i].volume;
     }
   }
   vwapSumPV += pv;
   vwapSumVol += vv;
   vwapNextFromMsc = t0_msc; // avança somente após processamento completo
   return (vwapSumVol > 0.0);
```

```
}

//+------------------------------------------------------------------+
//| [BLINDAGEM E2] SNAPSHOT DA VWAP VIVA PARA O ESTÁGIO 2            |
//| Estende LOCALMENTE o cache oficial até o negócio atual (to_msc). |
//| Usa cópias locais de vwapSumPV/vwapSumVol e da fronteira         |
//| vwapNextFromMsc. NÃO altera vwapSumPV, vwapSumVol, vwapNextFromMsc|
//| gVwapValue nem gVwapOk. Invariante do cache preservado: o        |
//| acumulador cobre msc < vwapNextFromMsc; a cópia adiciona         |
//| [vwapNextFromMsc, to_msc] sem lacuna, sobreposição ou duplicação.|
//+------------------------------------------------------------------+
bool GetLiveVwapSnapshot(ulong to_msc, double &liveVwap)
{
  liveVwap = 0.0;

   if(!vwapReady ||
      vwapSumVol <= 0.0 ||
      vwapNextFromMsc == 0)
   {
      return false;
   }

   double localPV = vwapSumPV;
   double localVol = vwapSumVol;

   if(to_msc >= vwapNextFromMsc)
   {
      MqlTick ticks[];
      int copied = CopyTradeTicksStrict(vwapNextFromMsc, to_msc, ticks, "VWAP_LIVE");

       // [BLINDAGEM E2+] Cópia vazia (copied==0) é falha conservadora: o
       // helper só é chamado após confirmar um negócio real, então zero
       // ticks no intervalo indica indisponibilidade momentânea do histórico
       // de ticks. Cair no cache anterior devolveria uma VWAP não atualizada
       // até o negócio rompedor, contrariando o propósito do snapshot vivo.
       if(copied <= 0)
          return false;

       for(int i = 0; i < copied; i++)
       {
         if(ticks[i].last > 0.0 &&
            ticks[i].volume > 0)
         {
            localPV += ticks[i].last * (double)ticks[i].volume;
            localVol += (double)ticks[i].volume;
         }
       }
   }
```

```
   if(localVol <= 0.0)
      return false;

   liveVwap = localPV / localVol;
   return liveVwap > 0.0;
}

//+------------------------------------------------------------------+
//| CIRURGIA 2 — MOTOR NATIVO 1: VOLUME PROFILE POR JANELA (POC)
|
//| Limite ÚNICO de níveis: 100000. Sem truncar, sem clampar, sem |
//| concentrar excedentes. Desempate preservado (primeiro máximo). |
//+------------------------------------------------------------------+
bool ComputePOCWindow(ulong from_msc, ulong to_msc,
                      double &poc, double &totalVolume, double &lastValidPrice)
{
   poc = 0.0; totalVolume = 0.0; lastValidPrice = 0.0;
   if(to_msc < from_msc) return false;                         // intervalo inválido

   MqlTick t[];
   int copied = CopyTradeTicksStrict(from_msc, to_msc, t, "POC_WINDOW");
   if(copied <= 0) return false;            // sem negócios (ou erro)

   double ts = SymbolInfoDouble(Symbol(), SYMBOL_TRADE_TICK_SIZE);
   if(ts <= 0.0) ts = SymbolInfoDouble(Symbol(), SYMBOL_POINT);
   if(ts <= 0.0) return false;              // tick size inválido

   double pmin = DBL_MAX, pmax = -DBL_MAX;
   bool any = false;
   for(int i=0; i<copied; i++)
   {
     if(t[i].last > 0.0 && t[i].volume > 0)
     {
        any = true;
        if(t[i].last < pmin) pmin = t[i].last;
        if(t[i].last > pmax) pmax = t[i].last;
        lastValidPrice = t[i].last;             // ordem cronológica: fica o último válido
     }
   }
   if(!any) return false;                      // nenhum negócio válido
   if(pmax < pmin) return false;                       // range inválido

   // Quantidade de níveis em tipo largo ANTES do cast
   long levelsL = (long)MathRound((pmax - pmin) / ts) + 1;
   if(levelsL < 1 || levelsL > 100000) return false; // não aloca, não trunca, não clampa
   int levels = (int)levelsL;
```

```
   double vol[];
   ArrayResize(vol, levels);
   ArrayInitialize(vol, 0.0);

   for(int i=0; i<copied; i++)
   {
     if(t[i].last > 0.0 && t[i].volume > 0)
     {
       int idx = (int)MathRound((t[i].last - pmin) / ts);
       if(idx < 0 || idx >= levels) continue;       // defensivo (matematicamente inatingível)
       vol[idx] += (double)t[i].volume;
       totalVolume += (double)t[i].volume;
     }
   }
   if(totalVolume <= 0.0) return false;                // volume total zero

   // Desempate preservado: '>' estrito mantém o PRIMEIRO máximo encontrado
   int best=0; double bestV=-1.0;
   for(int i=0; i<levels; i++)
     if(vol[i] > bestV){ bestV = vol[i]; best = i; }

   poc = pmin + best * ts;
   return true;
}

//+------------------------------------------------------------------+
//| CIRURGIA 8 — MOTOR NATIVO 2: WEIS WAVE 1P COM CACHE FECHADO      |
//| Rebuild na NOVA M5 (até 150 barras FECHADAS, lógica original     |
//| preservada: cronologia, dojis herdam direção, RealVolume com     |
//| fallback TickVolume, média das 10 ondas concluídas).             |
//| Estágio 2 usa extensão O(1) — proibido varrer 150 barras.        |
//+------------------------------------------------------------------+
void RebuildWeisCache()
{
   wwValid = false;
   int need = 150;
   double close[]; long rvol[], tvol[];
   ArraySetAsSeries(close, true);
   ArraySetAsSeries(rvol , true);
   ArraySetAsSeries(tvol , true);

   int n = CopyClose(Symbol(), Inp_Timeframe, 0, need, close);
   if(n < 12) return;

   bool useReal = (CopyRealVolume(Symbol(), Inp_Timeframe, 0, need, rvol) == n);
   if(!useReal && CopyTickVolume(Symbol(), Inp_Timeframe, 0, need, tvol) != n) return;
```

```
   double waves[]; int wc=0;
   int curDir=0; double accum=0.0;

   // LÓGICA ORIGINAL — somente barras FECHADAS (equivale a startIndex=1)
   for(int i=n-2; i>=1; i--)
   {
     int bd = (close[i] > close[i+1]) ? 1 : (close[i] < close[i+1] ? -1 : curDir);
     if(bd==0) bd = (curDir==0 ? 1 : curDir);
     double v = useReal ? (double)rvol[i] : (double)tvol[i];

      if(curDir==0)    { curDir=bd; accum=v; }
      else if(bd==curDir) { accum += v; }
      else { ArrayResize(waves, wc+1); waves[wc++]=accum; curDir=bd; accum=v; }
   }

   // Média das 10 últimas ondas CONCLUÍDAS (estado fechado)
   int cnt = MathMin(10, wc);
   double s=0.0;
   for(int k=wc-1; k>=wc-cnt && k>=0; k--) s += waves[k];
   double avgClosed = (cnt>0) ? s/cnt : 0.0;

   // Média SE houver inversão na barra viva: a onda ativa fechada
   // torna-se a mais nova onda CONCLUÍDA
   int cntF = MathMin(10, wc+1);
   double sF = accum;
   for(int k=wc-1; k>=wc-(cntF-1) && k>=0; k--) sF += waves[k];
   double avgFlip = (cntF>0) ? sF/cntF : 0.0;

   wwUseReal    = useReal;
   wwDirClosed = curDir;
   wwAccumClosed = accum;
   wwAvgClosed = avgClosed;
   wwAvgIfFlip = avgFlip;
   wwCloseRef   = close[1];
   wwWaveDirClosed = curDir;
   wwEffortClosed = (avgClosed>0.0) ? accum/avgClosed : 0.0;
   wwValid     = true;
}

// Extensão O(1) da onda para a barra VIVA — reproduz ComputeWeisWave(startIndex=0)
void ComputeLiveWaveO1(double &effort, double &curVol, double &avgVol, int &waveDir)
{
  double v0 = wwUseReal ? (double)iRealVolume(Symbol(), Inp_Timeframe, 0)
            : (double)iVolume(Symbol(), Inp_Timeframe, 0);
  double c0 = iClose(Symbol(), Inp_Timeframe, 0);

   int bd0 = (c0 > wwCloseRef) ? 1 : ((c0 < wwCloseRef) ? -1 : wwDirClosed);
   if(bd0 == 0) bd0 = wwDirClosed;
```

```
   if(bd0 == wwDirClosed)
   {
      // Continuidade da onda fechada
      curVol = wwAccumClosed + v0;
      waveDir = wwDirClosed;
      avgVol = wwAvgClosed;
   }
   else
   {
      // Inversão: a onda fechada conclui; a barra viva inicia nova onda
      curVol = v0;
      waveDir = bd0;
      avgVol = wwAvgIfFlip;
   }
   effort = (avgVol > 0.0) ? curVol / avgVol : 0.0;
}

//+------------------------------------------------------------------+
//| [E1] ESTRUTURA D-1 LITERAL — VWAP_D1 / H1 / L1                   |
//| Fonte ÚNICA: o último pregão concluído imediatamente anterior ao |
//| dia atual, identificado pelo histórico diário do próprio símbolo:|
//| - iTime(PERIOD_D1,0) >= meia-noite de hoje => sessão atual já |
//| criada => D-1 literal = iTime(PERIOD_D1,1);                      |
//| - caso contrário (pré-abertura) => D-1 literal = a própria       |
//| última barra diária concluída, iTime(PERIOD_D1,0).               |
//| PROIBIDO: substituir por D-2/D-3 ou pelo primeiro dia M1 válido; |
//| PROIBIDO: fallback OHLC (H+L+C)/3 marcado como VWAP oficial. |
//| Falha => VWAP_D1_Official permanece false e entradas bloqueadas. |
//+------------------------------------------------------------------+
void CalcD1Structure()
{
   H1=0.0; L1=0.0; VWAP_D1=0.0; VWAP_D1_Official=false; gD1ExpectedDate=0;
 // Meia-noite do dia atual no servidor
 MqlDateTime td; TimeToStruct(TimeTradeServer(), td);
 td.hour=0; td.min=0; td.sec=0;
 datetime todayStart = StructToTime(td);

  // Identificação do D-1 literal via PERIOD_D1 do próprio símbolo
  datetime d1_0 = iTime(Symbol(), PERIOD_D1, 0);
  if(d1_0 <= 0)
  {
     Print("[REMORA2.0][D1_LITERAL] Histórico diário indisponível (iTime D1 shift 0
falhou).");
     return;
  }

  datetime expected = 0;
```

```
 if(d1_0 >= todayStart)
 {
    // Sessão de hoje já existe no PERIOD_D1: D-1 literal é o shift 1
    expected = iTime(Symbol(), PERIOD_D1, 1);
    if(expected <= 0)
    {
       Print("[REMORA2.0][D1_LITERAL] Histórico diário indisponível (iTime D1 shift 1
falhou).");
       return;
    }
 }
 else
 {
    // Pré-abertura: a última barra diária concluída É o D-1 literal
    expected = d1_0;
 }
 gD1ExpectedDate = expected;

// Janela M1 do dia esperado
MqlDateTime ed; TimeToStruct(expected, ed);
ed.hour=0; ed.min=0; ed.sec=0;
datetime dayStart = StructToTime(ed);
datetime dayEnd = dayStart + 86400 - 1;

datetime timeBuf[];
int nBars = CopyTime(Symbol(), PERIOD_M1, dayStart, dayEnd, timeBuf);
if(nBars < D1_MIN_M1_BARS)
{
   PrintFormat("[REMORA2.0][D1_LITERAL] D-1=%s com M1 insuficiente (%d < %d).
Estrutura INVÁLIDA; nenhum dia anterior substitui. Retry em 60s.",
         TimeToString(dayStart, TIME_DATE), nBars, D1_MIN_M1_BARS);
   return;
}
 // Data carregada deve ser exatamente a data esperada
 MqlDateTime b0; TimeToStruct(timeBuf[0], b0);
 if(b0.year != ed.year || b0.mon != ed.mon || b0.day != ed.day)
 {
    PrintFormat("[REMORA2.0][D1_LITERAL] Data carregada (%s) difere da data esperada
(%s). Estrutura INVÁLIDA.",
          TimeToString(timeBuf[0], TIME_DATE), TimeToString(dayStart, TIME_DATE));
    return;
 }

double highBuf[], lowBuf[], closeBuf[];
if(CopyHigh (Symbol(),PERIOD_M1,dayStart,dayEnd,highBuf )!=nBars) return;
if(CopyLow (Symbol(),PERIOD_M1,dayStart,dayEnd,lowBuf )!=nBars) return;
if(CopyClose(Symbol(),PERIOD_M1,dayStart,dayEnd,closeBuf)!=nBars) return;
```

```
  long realVol[], tickVol[];
  bool useRealVol =
(CopyRealVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,realVol)==nBars);
  if(!useRealVol &&
     CopyTickVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,tickVol)!=nBars) return;

 // Sessão 09:00-18:25: H/L e VWAP ponderada por volume do MESMO dia
 double dayHigh=0.0, dayLow=DBL_MAX;
 double sumTPVol=0.0, sumVol=0.0;
 int sessBar=0;
 for(int i=0;i<nBars;i++)
 {
    MqlDateTime b; TimeToStruct(timeBuf[i], b);
    int hhmm = b.hour*100 + b.min;
    if(hhmm>=D1_SESSION_START && hhmm<D1_SESSION_END)
    {
       if(highBuf[i]>dayHigh) dayHigh=highBuf[i];
       if(lowBuf[i] <dayLow ) dayLow =lowBuf[i];
       double tp = (highBuf[i]+lowBuf[i]+closeBuf[i])/3.0;
       double vol = useRealVol ? (double)realVol[i] : (double)tickVol[i];
       if(vol>0.0){ sumTPVol += tp*vol; sumVol += vol; }
       sessBar++;
    }
 }

  if(dayHigh<=0.0 || dayLow>=DBL_MAX || dayHigh<=dayLow || sessBar==0)
  {
     PrintFormat("[REMORA2.0][D1_LITERAL] Sessão do D-1=%s sem H/L válidos
(sessBar=%d). Estrutura INVÁLIDA.",
            TimeToString(dayStart, TIME_DATE), sessBar);
     return;
  }
  if(sumVol<=0.0)
  {
     PrintFormat("[REMORA2.0][D1_LITERAL] D-1=%s sem volume acumulado. Estrutura
INVÁLIDA.",
            TimeToString(dayStart, TIME_DATE));
     return;
  }
  double dayVWAP = sumTPVol/sumVol;
  if(dayVWAP<=0.0) return;

  // Aprovação: SOMENTE aqui a estrutura recebe valores oficiais
  H1 = dayHigh; L1 = dayLow; VWAP_D1 = dayVWAP; VWAP_D1_Official = true;
  PrintFormat("[REMORA2.0][D1_LITERAL] D-1 (%s): VWAP=%.2f H1=%.2f L1=%.2f |
Barras M1=%d SessBar=%d | Vol=%s",
         TimeToString(dayStart,TIME_DATE), VWAP_D1, H1, L1,
         nBars, sessBar, useRealVol?"RealVolume":"TickVolume");
```

```
}

//+------------------------------------------------------------------+
//| [E2] SIGMA HISTÓRICO — VWSD médio dos últimos dias válidos       |
//| A busca por dias anteriores (até MAX_SEARCH_DAYS) é permitida    |
//| SOMENTE para o Sigma. A VWAP interna de cada dia serve apenas ao |
//| VWSD: NÃO é VWAP_D1, NÃO altera H1/L1, NÃO marca oficialidade.   |
//| Semântica intacta: lookback, RealVolume/TickVolume, média dos    |
//| dias válidos e piso por família.                                 |
//+------------------------------------------------------------------+
void CalcSigmaHistory()
{
   SigmaD1 = 0.0;

 MqlDateTime todayDT; TimeToStruct(TimeTradeServer(), todayDT);
 todayDT.hour=0; todayDT.min=0; todayDT.sec=0;
 datetime searchDay = StructToTime(todayDT);

 int targetDays = MathMax(1, MathMin(SigmaLookbackDays, 20));
 double sigmaArray[20];
 int validDaysFound=0;

  for(int attempt=0; attempt<MAX_SEARCH_DAYS && validDaysFound<targetDays;
attempt++)
  {
    searchDay -= 86400;
    MqlDateTime sdt; TimeToStruct(searchDay, sdt);
    if(sdt.day_of_week==0 || sdt.day_of_week==6) continue;

   sdt.hour=0; sdt.min=0; sdt.sec=0;
   datetime dayStart = StructToTime(sdt);
   datetime dayEnd = dayStart + 86400 - 1;
   datetime timeBuf[];
   int nBars = CopyTime(Symbol(), PERIOD_M1, dayStart, dayEnd, timeBuf);
   if(nBars < D1_MIN_M1_BARS) continue;

   double highBuf[], lowBuf[];
   if(CopyHigh(Symbol(),PERIOD_M1,dayStart,dayEnd,highBuf)!=nBars) continue;
   if(CopyLow (Symbol(),PERIOD_M1,dayStart,dayEnd,lowBuf )!=nBars) continue;

   double dayHigh=0.0, dayLow=DBL_MAX; int sessBar=0;
   for(int i=0;i<nBars;i++)
   {
      MqlDateTime b; TimeToStruct(timeBuf[i], b);
      int hhmm = b.hour*100 + b.min;
      if(hhmm>=D1_SESSION_START && hhmm<D1_SESSION_END)
      {
         if(highBuf[i]>dayHigh) dayHigh=highBuf[i];
```

```
        if(lowBuf[i] <dayLow ) dayLow =lowBuf[i];
        sessBar++;
      }
    }
    if(dayHigh<=0.0 || dayLow>=DBL_MAX || dayHigh<=dayLow || sessBar==0) continue;

    double closeBuf[]; long realVol[], tickVol[];
    int copiedClose = CopyClose(Symbol(),PERIOD_M1,dayStart,dayEnd,closeBuf);
    bool useRealVol =
(CopyRealVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,realVol)==nBars);
    int copiedTV=0;
    if(!useRealVol) copiedTV =
CopyTickVolume(Symbol(),PERIOD_M1,dayStart,dayEnd,tickVol);
    if(!((copiedClose==nBars) && (useRealVol || copiedTV==nBars))) continue;

    // VWAP INTERNA do dia — insumo exclusivo do VWSD (NÃO é VWAP_D1)
    double sumTPVol=0.0, sumVol=0.0;
    for(int i=0;i<nBars;i++)
    {
      MqlDateTime b; TimeToStruct(timeBuf[i], b);
      int hhmm = b.hour*100 + b.min;
      if(hhmm>=D1_SESSION_START && hhmm<D1_SESSION_END)
      {
        double tp = (highBuf[i]+lowBuf[i]+closeBuf[i])/3.0;
        double vol = useRealVol ? (double)realVol[i] : (double)tickVol[i];
        if(vol>0.0){ sumTPVol += tp*vol; sumVol += vol; }
      }
    }
    if(sumVol<=0.0) continue;
    double dayVWAP = sumTPVol/sumVol;
     double sumWVar=0.0, sumVolB=0.0;
     for(int j=0;j<nBars;j++)
     {
       MqlDateTime b2; TimeToStruct(timeBuf[j], b2);
       int hhmm2 = b2.hour*100 + b2.min;
       if(hhmm2>=D1_SESSION_START && hhmm2<D1_SESSION_END)
       {
         double tp = (highBuf[j]+lowBuf[j]+closeBuf[j])/3.0;
         double vol = useRealVol ? (double)realVol[j] : (double)tickVol[j];
         if(vol>0.0){ double dd=tp-dayVWAP; sumWVar += vol*dd*dd; sumVolB += vol; }
       }
     }
     double daySigma = (sumVolB>0.0) ? MathSqrt(sumWVar/sumVolB) : 0.0;
     sigmaArray[validDaysFound++] = daySigma;
  }

  if(validDaysFound==0)
  {
```

```
   Print("[REMORA2.0][SIGMA] CRÍTICO: nenhum dia válido para o Sigma histórico.");
   return;
 }

 double sumSigma = 0.0;
 for(int k = 0; k < validDaysFound; k++)
   sumSigma += sigmaArray[k];

 SigmaD1 = sumSigma / (double)validDaysFound;

 // Piso por família (gIsIndexFamily já classificado e validado no OnInit)
 double pisoSigma = gIsIndexFamily ? Inp_SigmaFloor_WIN : Inp_SigmaFloor_WDO;

 // Aplica somente o piso mínimo. O Sigma permanece livre para subir acima desse valor.
 if(SigmaD1 < pisoSigma)
 {
   PrintFormat("[%s] SIGMA ABAIXO DO PISO | Calculado=%.2f | PisoConfigurado=%.2f |
Familia=%s",
         Symbol(), SigmaD1, pisoSigma, gIsIndexFamily ? "WIN/IND" : "WDO/DOL");
   SigmaD1 = pisoSigma;
 }

  PrintFormat("[%s] SIGMA HISTÓRICO CONSOLIDADO | SigmaD1=%.4f |
DiasVálidos=%d/%d",
       Symbol(), SigmaD1, validDaysFound, targetDays);
}

//+------------------------------------------------------------------+
//| CIRURGIA 13.3 + [E2] — ORQUESTRADOR D-1 RESILIENTE               |
//| Recalcula APENAS o módulo pendente: a falha da estrutura não     |
//| apaga um Sigma já válido, e o retry não refaz todo o histórico. |
//+------------------------------------------------------------------+
void TryCalcD1()
{
  if(!VWAP_D1_Official) CalcD1Structure(); // [E1] estrutura D-1 literal
  if(SigmaD1 <= 0.0) CalcSigmaHistory(); // [E2] Sigma histórico separado

   d1Ready = (VWAP_D1_Official && VWAP_D1>0.0 && SigmaD1>0.0 &&
       H1>0.0 && L1>0.0 && H1>L1);

   if(!d1Ready)
     nextD1Retry = TimeTradeServer() + 60;
}

// [2.43.1/S2] Telemetria auxiliar. Nenhuma funcao abaixo veta entradas,
// altera cota, cooldown, BE ou o resultado operacional de SendOrder.
int g2431ExecHandle = INVALID_HANDLE;
ulong g2431AcceptedOrders[];
```

```
ulong g2431AcceptedDeals[];
ulong g2431ConfirmedDeals[];
ulong g2431PendingOrders[];
ulong g2431ReconciledOrders[];
datetime g2431ExecNextReconcile = 0;
bool g2431ExecIoWarned = false;

// 2.435 DIAGNOSTICS ONLY. Embed after original globals; no external dependency.
// All trading state referenced here is READ ONLY. No diagnostic result is a gate.
enum ENUM_DIAG2435_REASON
{
 DIAG2435_NONE=0,
 DIAG2435_OUTSIDE_SESSION,
 DIAG2435_POSITION_GATE,
 DIAG2435_DAILY_QUOTA,
 DIAG2435_COOLDOWN,
 DIAG2435_D1_INVALID,
 DIAG2435_VWAP_INVALID,
 DIAG2435_WEIS_INVALID,
 DIAG2435_MATRIX_ALREADY_EVALUATED,
 DIAG2435_ARMED_TICK_UNAVAILABLE,
 DIAG2435_ARMED_NO_REAL_TRADE,
 DIAG2435_ARMED_LAST_INVALID,
 DIAG2435_ARMED_BREAKOUT_NOT_CONFIRMED,
 DIAG2435_ARMED_EXECUTABLE_INVALID,
 DIAG2435_ARMED_WEIS_INVALID,
 DIAG2435_ARMED_WAVE_DIRECTION,
 DIAG2435_ARMED_EFFORT_INSUFFICIENT,
 DIAG2435_ARMED_HOLD_SPREAD,
 DIAG2435_ARMED_POSITION_CANCEL,
 DIAG2435_ARMED_QUOTA_CANCEL,
 DIAG2435_LATE_POSITION_BLOCK,
 DIAG2435_QUOTA_UNKNOWN,
 DIAG2435_LATE_QUOTA_BLOCK,
 DIAG2435_LATE_PENDING_BLOCK,
 DIAG2435_FINAL_TICK_UNAVAILABLE,
 DIAG2435_FINAL_EXECUTABLE_INVALID,
 DIAG2435_LATE_BREAKOUT_LOST,
 DIAG2435_LATE_OVERSHOOT_BLOCK,
 DIAG2435_STOPS_LEVEL_BLOCK,
 DIAG2435_NO_MARGIN,
 DIAG2435_ORDER_FAILED,
 DIAG2435_ORDER_ACCEPTED
};

struct Diag2435Row
{
 ulong sequence;
```

```
 int stage,reason;
 datetime bar,serverTime;
 int hhmm,direction,route,mode,lots;
 int shots,quota,quotaHistory,persistPending,pendingOrders,acceptedDeals;
 int netting,d1,vwap,weis;
 double trigger,last,effort;
 int waveDirection;
 long firstTickMsc,finalTickMsc;
 double finalBid,finalAsk,finalExecutable;
 int sent,accepted;
 uint ret;
 double requestPrice,sl,tp;
 ulong order,deal;
};

struct Diag2435Trace
{
 int reason,direction,lots;
 double trigger,last;
 long firstTickMsc,finalTickMsc;
 double finalBid,finalAsk,finalExecutable;
 int sent,accepted;
 uint ret;
 double requestPrice,sl,tp;
 ulong order,deal;
};

Diag2435Row gDiag2435Queue[128];
Diag2435Trace gDiag2435Trace;
datetime gDiag2435SeenBar[64];
int gDiag2435Head=0,gDiag2435Count=0;
ulong gDiag2435Sequence=0,gDiag2435Drops=0,gDiag2435IoFailures=0;
ulong gDiag2435LastFlushUs=0,gDiag2435MaxFlushUs=0,gDiag2435RowsWritten=0;
int gDiag2435Handle=INVALID_HANDLE;
string gDiag2435Symbol="";
long gDiag2435Magic=0;
bool gDiag2435IoWarned=false;

string Diag2435ReasonText(int reason)
{
 switch(reason)
 {
  case DIAG2435_OUTSIDE_SESSION: return "OUTSIDE_ENTRY_SESSION";
  case DIAG2435_POSITION_GATE: return "POSITION_GATE";
  case DIAG2435_DAILY_QUOTA: return "DAILY_QUOTA_REACHED";
  case DIAG2435_COOLDOWN: return "COOLDOWN";
  case DIAG2435_D1_INVALID: return "D1_SIGMA_NOT_READY";
  case DIAG2435_VWAP_INVALID: return "VWAP_CACHE_INVALID";
```

```
  case DIAG2435_WEIS_INVALID: return "WW_CACHE_INVALID";
  case DIAG2435_MATRIX_ALREADY_EVALUATED: return "MATRIX_DUP_GUARD";
  case DIAG2435_ARMED_TICK_UNAVAILABLE: return "ARMED_TICK_UNAVAILABLE";
  case DIAG2435_ARMED_NO_REAL_TRADE: return "WAIT_REAL_TRADE_EVENT";
  case DIAG2435_ARMED_LAST_INVALID: return "WAIT_VALID_LAST";
  case DIAG2435_ARMED_BREAKOUT_NOT_CONFIRMED: return
"WAIT_LAST_BREAKOUT";
  case DIAG2435_ARMED_EXECUTABLE_INVALID: return "WAIT_VALID_EXECUTABLE";
  case DIAG2435_ARMED_WEIS_INVALID: return "WAIT_WEIS_CACHE";
  case DIAG2435_ARMED_WAVE_DIRECTION: return "WAIT_LIVE_WAVE_DIRECTION";
  case DIAG2435_ARMED_EFFORT_INSUFFICIENT: return
"WAIT_LIVE_WAVE_EFFORT";
  case DIAG2435_ARMED_HOLD_SPREAD: return "HOLD_SPREAD";
  case DIAG2435_ARMED_POSITION_CANCEL: return "ARMED_POSITION_CANCEL";
  case DIAG2435_ARMED_QUOTA_CANCEL: return "ARMED_QUOTA_CANCEL";
  case DIAG2435_LATE_POSITION_BLOCK: return "LATE_POSITION_BLOCK";
  case DIAG2435_QUOTA_UNKNOWN: return "QUOTA_UNKNOWN";
  case DIAG2435_LATE_QUOTA_BLOCK: return "LATE_QUOTA_BLOCK";
  case DIAG2435_LATE_PENDING_BLOCK: return "LATE_PENDING_BLOCK";
  case DIAG2435_FINAL_TICK_UNAVAILABLE: return "FINAL_TICK_UNAVAILABLE";
  case DIAG2435_FINAL_EXECUTABLE_INVALID: return
"FINAL_EXECUTABLE_INVALID";
  case DIAG2435_LATE_BREAKOUT_LOST: return "LATE_BREAKOUT_LOST";
  case DIAG2435_LATE_OVERSHOOT_BLOCK: return "LATE_OVERSHOOT_BLOCK";
  case DIAG2435_STOPS_LEVEL_BLOCK: return "STOPS_LEVEL_BLOCK";
  case DIAG2435_NO_MARGIN: return "NO_MARGIN";
  case DIAG2435_ORDER_FAILED: return "ORDER_FAILED";
  case DIAG2435_ORDER_ACCEPTED: return "ORDER_ACCEPTED";
 }
 return "UNSPECIFIED";
}

string Diag2435StageText(int stage)
{
 if(stage==1) return "MATRIX_NOT_EVALUATED";
 if(stage==2) return "ARMED_WAITING";
 if(stage==3) return "ARMED_CANCELLED";
 if(stage==4) return "FINAL_REVALIDATION";
 if(stage==5) return "REQUEST_OUTCOME";
 return "UNSPECIFIED";
}

int Diag2435RouteCode(string route)
{
 if(route=="ABS") return 1;
 if(route=="CONT") return 2;
 return 0;
}
```

```
int Diag2435ModeCode(string mode)
{
 if(mode=="BASE" || mode=="BASE_CONT") return 1;
 if(mode=="TURBO" || mode=="TURBO_CONT") return 2;
 return 0;
}

// No terminal/account/tick/history/position query; scalar context only.
void Diag2435Context(Diag2435Row &r,datetime bar,datetime serverTime,int hhmm)
{
 r.bar=bar; r.serverTime=serverTime; r.hhmm=hhmm;
 r.shots=ordersEnteredToday;
 r.quota=(int)gQuotaState; r.quotaHistory=(int)gQuotaHistoryState;
 r.persistPending=(int)gQuotaPersistPending;
 r.pendingOrders=ArraySize(g2431PendingOrders);
 r.acceptedDeals=ArraySize(g2431AcceptedDeals);
 r.netting=(int)gNetting;
 r.d1=(int)d1Ready; r.vwap=(int)gVwapOk; r.weis=(int)wwValid;
}

void Diag2435Enqueue(Diag2435Row &r,bool dedup)
{
 if(dedup && r.reason>0 && r.reason<64)
 {
  if(gDiag2435SeenBar[r.reason]==r.bar) return;
  gDiag2435SeenBar[r.reason]=r.bar;
 }
 if(gDiag2435Count>=128) { gDiag2435Drops++; return; }
 r.sequence=++gDiag2435Sequence;
 int tail=(gDiag2435Head+gDiag2435Count)%128;
 gDiag2435Queue[tail]=r;
 gDiag2435Count++;
}

void Diag2435M5(int reason,datetime bar,datetime serverTime,int hhmm)
{
 Diag2435Row r; ZeroMemory(r);
 r.stage=1; r.reason=reason;
 Diag2435Context(r,bar,serverTime,hhmm);
 Diag2435Enqueue(r,true);
}

// First observed nonterminal wait per bar/reason; never writes per tick.
// Terminal=true only in an existing final branch, once before original reset.
void Diag2435Armed(int reason,datetime bar,datetime serverTime,int hhmm,
                   double trigger,double last=0.0,double effort=0.0,
                   int waveDirection=0,bool terminal=false)
```

```
{
 if(!terminal && reason>0 && reason<64 && gDiag2435SeenBar[reason]==bar) return;
 Diag2435Row r; ZeroMemory(r);
 r.stage=terminal?3:2; r.reason=reason;
 Diag2435Context(r,bar,serverTime,hhmm);
 r.direction=armed.dir;
 r.lots=armed.lots;
 r.route=Diag2435RouteCode(armed.route); r.mode=Diag2435ModeCode(armed.mode);
 r.trigger=trigger; r.last=last; r.effort=effort; r.waveDirection=waveDirection;
 Diag2435Enqueue(r,!terminal);
}

// These trace helpers are called ONLY after an original refusal or AFTER
// PositionOpen and all its original result treatment. There is NO new helper
// on a successful path before PositionOpen. No clock, strings, formatting,
// allocation, file or terminal query.
// Call SendBegin in ProcessArmedSignal AFTER SendOrder returns; do not reset
// fields copied from the final tick by the already-completed SendOrder.
void Diag2435SendBegin(int direction,int lots,double trigger,double last,long tickMsc)
{
 gDiag2435Trace.direction=direction; gDiag2435Trace.lots=lots;
 gDiag2435Trace.trigger=trigger; gDiag2435Trace.last=last;
 gDiag2435Trace.firstTickMsc=tickMsc;
}

void Diag2435SendTick(double bid,double ask,long tickMsc,double executable)
{
 gDiag2435Trace.finalBid=bid; gDiag2435Trace.finalAsk=ask;
 gDiag2435Trace.finalTickMsc=tickMsc; gDiag2435Trace.finalExecutable=executable;
}
// First call in every SendOrder return branch: reset stale final/outcome data.
// Call SendTick and SendOutcome only AFTER this reset.
void Diag2435SendReason(int reason)
{
 ZeroMemory(gDiag2435Trace);
 gDiag2435Trace.reason=reason;
}

// Call only AFTER original PositionOpen result treatment and quota persistence.
void Diag2435SendOutcome(bool sent,bool accepted,uint ret,double price,double sl,
              double tp,ulong order,ulong deal)
{
 gDiag2435Trace.sent=(int)sent; gDiag2435Trace.accepted=(int)accepted;
 gDiag2435Trace.ret=ret; gDiag2435Trace.requestPrice=price;
 gDiag2435Trace.sl=sl; gDiag2435Trace.tp=tp;
 gDiag2435Trace.order=order; gDiag2435Trace.deal=deal;
}
```

```
// Call after SendOrder returns and before the original ResetArmedSignal.
void Diag2435SendFinish(datetime bar,datetime serverTime,int hhmm)
{
 Diag2435Row r; ZeroMemory(r);
 r.reason=gDiag2435Trace.reason;
 r.stage=(r.reason==DIAG2435_ORDER_ACCEPTED ||
r.reason==DIAG2435_ORDER_FAILED)?5:4;
 Diag2435Context(r,bar,serverTime,hhmm);
 r.direction=gDiag2435Trace.direction;
 r.lots=gDiag2435Trace.lots;
 r.route=Diag2435RouteCode(armed.route); r.mode=Diag2435ModeCode(armed.mode);
 r.trigger=gDiag2435Trace.trigger; r.last=gDiag2435Trace.last;
 r.firstTickMsc=gDiag2435Trace.firstTickMsc;
 r.finalTickMsc=gDiag2435Trace.finalTickMsc;
 r.finalBid=gDiag2435Trace.finalBid; r.finalAsk=gDiag2435Trace.finalAsk;
 r.finalExecutable=gDiag2435Trace.finalExecutable;
 r.sent=gDiag2435Trace.sent; r.accepted=gDiag2435Trace.accepted;
 r.ret=gDiag2435Trace.ret; r.requestPrice=gDiag2435Trace.requestPrice;
 r.sl=gDiag2435Trace.sl; r.tp=gDiag2435Trace.tp;
 r.order=gDiag2435Trace.order; r.deal=gDiag2435Trace.deal;
 Diag2435Enqueue(r,false);
}

string Diag2435Ulong(ulong value) { return StringFormat("%I64u",value); }

// Serialization is exclusively Timer/Deinit work. Tick timestamps are NOT
// wall-clock latency; negative/missing market deltas are explicitly unavailable.
string Diag2435Serialize(Diag2435Row &r)
{
string delta="NA",rawDelta="NA",orientedDelta="NA";
if(r.firstTickMsc>0 && r.finalTickMsc>=r.firstTickMsc)
 delta=IntegerToString(r.finalTickMsc-r.firstTickMsc);
if(r.finalExecutable>0.0 && r.trigger>0.0)
{
 rawDelta=DoubleToString(r.finalExecutable-r.trigger,8);
 orientedDelta=DoubleToString(r.direction==1 ? r.finalExecutable-r.trigger :
                 r.trigger-r.finalExecutable,8);
}
string direction=r.direction==1?"BUY":(r.direction==-1?"SELL":"NONE");
string route=r.route==1?"ABS":(r.route==2?"CONT":"NONE");
string mode=r.mode==1?"BASE":(r.mode==2?"TURBO":"NONE");
return Diag2435Ulong(r.sequence)+","+IntegerToString((long)r.serverTime)+","+
 gDiag2435Symbol+","+IntegerToString(gDiag2435Magic)+","+
 Diag2435StageText(r.stage)+","+Diag2435ReasonText(r.reason)+","+
 IntegerToString((long)r.bar)+","+IntegerToString(r.hhmm)+","+
 direction+","+route+","+mode+","+IntegerToString(r.lots)+","+
 IntegerToString(r.shots)+","+IntegerToString(r.quota)+","+
 IntegerToString(r.quotaHistory)+","+IntegerToString(r.persistPending)+","+
```

```
 IntegerToString(r.pendingOrders)+","+IntegerToString(r.acceptedDeals)+","+
 IntegerToString(r.netting)+","+IntegerToString(r.d1)+","+
 IntegerToString(r.vwap)+","+IntegerToString(r.weis)+","+
 DoubleToString(r.trigger,8)+","+DoubleToString(r.last,8)+","+
 DoubleToString(r.effort,8)+","+IntegerToString(r.waveDirection)+","+
 IntegerToString(r.firstTickMsc)+","+IntegerToString(r.finalTickMsc)+","+
 DoubleToString(r.finalBid,8)+","+DoubleToString(r.finalAsk,8)+","+
 DoubleToString(r.finalExecutable,8)+","+rawDelta+","+orientedDelta+","+delta+","+
 IntegerToString(r.sent)+","+IntegerToString(r.accepted)+","+
 IntegerToString((long)r.ret)+","+DoubleToString(r.requestPrice,8)+","+
 DoubleToString(r.sl,8)+","+DoubleToString(r.tp,8)+","+
 Diag2435Ulong(r.order)+","+Diag2435Ulong(r.deal)+","+
 Diag2435Ulong(gDiag2435Drops)+","+Diag2435Ulong(gDiag2435IoFailures)+","+

Diag2435Ulong(gDiag2435LastFlushUs)+","+Diag2435Ulong(gDiag2435MaxFlushUs)+"\r\n"
;
}

void Diag2435IoWarning()
{
 if(gDiag2435IoWarned) return;
 gDiag2435IoWarned=true;
 Print("[REMORA2.435][DIAGNOSTICS_IO] Arquivo auxiliar indisponivel. "
     "Diagnosticos podem ser perdidos; fluxo operacional inalterado.");
}

// Init only: no inputs added; operational filenames are never changed.
void Diag2435Init()
{
gDiag2435Symbol=Symbol(); gDiag2435Magic=(long)MagicCore;
string name=CSVFileName+"_"+gDiag2435Symbol+"_"+
       IntegerToString(gDiag2435Magic)+"_Diagnostics_v2435.csv";

gDiag2435Handle=FileOpen(name,FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI|FILE_S
HARE_READ);
 if(gDiag2435Handle==INVALID_HANDLE)
 { gDiag2435IoFailures++; Diag2435IoWarning(); return; }
 string
header="SEQUENCE,SERVER_TIME,SYMBOL,MAGIC,STAGE,REASON,M5_BAR,HHMM,
DIR,ROUTE,MODE,LOTS,SHOTS,QUOTA_STATE,QUOTA_HISTORY,PERSIST_PENDING
,PENDING_ORDERS,ACCEPTED_DEALS,NETTING,D1_READY,VWAP_OK,WEIS_OK,TR
IGGER,CONFIRMED_LAST,LIVE_EFFORT,LIVE_WAVE_DIR,CONFIRM_TICK_MSC,FINA
L_TICK_MSC,FINAL_BID,FINAL_ASK,FINAL_EXECUTABLE,EXEC_MINUS_TRIGGER,DI
RECTIONAL_DELTA,MARKET_TIMESTAMP_DELTA_MS,POSITION_OPEN_BOOL,REQU
EST_ACCEPTED,RETCODE,REQUEST_PRICE,SL,TP,ORDER,DEAL,DROPPED,IO_FAIL
URES,PREVIOUS_FLUSH_US,MAX_PREVIOUS_FLUSH_US\r\n";
 bool ok=FileSeek(gDiag2435Handle,0,SEEK_END);
 if(ok && FileSize(gDiag2435Handle)==0)
```

```
  ok=(FileWriteString(gDiag2435Handle,header)==(uint)StringLen(header));
 if(!ok)
 {
  gDiag2435IoFailures++; FileClose(gDiag2435Handle);
  gDiag2435Handle=INVALID_HANDLE; Diag2435IoWarning();
 }
}

// Call at END of OnTimer, after original EOD and execution reconciliation.
// Bounded work: at most 16 queued events per flush, one file write/flush.
void Diag2435Flush()
{
 if(gDiag2435Count<=0 || gDiag2435Handle==INVALID_HANDLE) return;
 ulong begin=GetMicrosecondCount();
 int n=gDiag2435Count<16?gDiag2435Count:16;
 string batch="";
 for(int i=0;i<n;i++)
 {
  int index=(gDiag2435Head+i)%128;
  batch+=Diag2435Serialize(gDiag2435Queue[index]);
 }
 bool ok=FileSeek(gDiag2435Handle,0,SEEK_END);
 if(ok) ok=(FileWriteString(gDiag2435Handle,batch)==(uint)StringLen(batch));
 if(ok)
 {
  ResetLastError(); FileFlush(gDiag2435Handle);
  ok=(GetLastError()==0);
 }
 if(ok)
 {
  gDiag2435Head=(gDiag2435Head+n)%128; gDiag2435Count-=n;
  gDiag2435RowsWritten+=(ulong)n;
 }
 else
 {
  gDiag2435IoFailures++;
  gDiag2435Drops+=(ulong)gDiag2435Count;
  gDiag2435Count=0; gDiag2435Head=0;
  FileClose(gDiag2435Handle); gDiag2435Handle=INVALID_HANDLE;
  Diag2435IoWarning();
 }
 gDiag2435LastFlushUs=GetMicrosecondCount()-begin;
 if(gDiag2435LastFlushUs>gDiag2435MaxFlushUs)
  gDiag2435MaxFlushUs=gDiag2435LastFlushUs;
}

void Diag2435Deinit()
{
```

```
 // Bounded final attempt; remaining queue may be lost on shutdown.
 Diag2435Flush();
 if(gDiag2435Handle!=INVALID_HANDLE)
 { FileClose(gDiag2435Handle); gDiag2435Handle=INVALID_HANDLE; }
}

string S2431TicketText(ulong ticket)
{
  return StringFormat("%I64u", ticket);
}

// Parse decimal sem conversao por double e sem overflow silencioso.
bool S2431ParseTicket(string value, ulong &ticket)
{
   ticket=0;
   int n=StringLen(value);
   if(n==0) return false;
   for(int i=0;i<n;i++)
   {
      ushort c=StringGetCharacter(value,i);
      if(c<48 || c>57) return false;
      ulong digit=(ulong)(c-48);
      ulong previous=ticket;
      ticket=ticket*10+digit;
      if(ticket<previous || (ticket-digit)/10!=previous) return false;
   }
   return true;
}
bool S2431HasTicket(const ulong &items[], ulong ticket)
{
  if(ticket==0) return false;
  for(int i=0;i<ArraySize(items);i++) if(items[i]==ticket) return true;
  return false;
}

void S2431RememberTicket(ulong &items[], ulong ticket)
{
  if(ticket==0 || S2431HasTicket(items,ticket)) return;
  int n=ArraySize(items);
  if(ArrayResize(items,n+1)==n+1) items[n]=ticket;
}

void S2431ForgetTicket(ulong &items[], ulong ticket)
{
  int n=ArraySize(items);
  for(int i=0;i<n;i++)
  {
    if(items[i]!=ticket) continue;
```

```
   for(int j=i+1;j<n;j++) items[j-1]=items[j];
   ArrayResize(items,n-1);
   return;
  }
}

void S2431ExecutionIoWarning()
{
  if(g2431ExecIoWarned) return;
  g2431ExecIoWarned=true;
  Print("[REMORA2.43.1][EXEC_AUX_IO] Diario auxiliar indisponivel/incompleto. "
     "Telemetria segue em memoria; deduplicacao entre reinicios nao e garantida. Entradas
inalteradas.");
}

bool S2431WriteExecutionEvent(string event, ulong order, ulong deal,
                  ulong position, datetime stamp, double price,
                  double volume, uint ret, string source)
{
  if(g2431ExecHandle==INVALID_HANDLE)
  {
    S2431ExecutionIoWarning();
    return false;
  }
  // Delimita tambem o inicio: uma gravacao interrompida nao contamina a proxima.
  string line="\r\n"+event+","+S2431TicketText(order)+","+S2431TicketText(deal)+","+
         S2431TicketText(position)+","+IntegerToString((long)stamp)+","+
         DoubleToString(price,_Digits)+","+DoubleToString(volume,8)+","+
          IntegerToString((long)ret)+","+source+"\r\n";
   if(!FileSeek(g2431ExecHandle,0,SEEK_END) ||
      FileWriteString(g2431ExecHandle,line)!=(uint)StringLen(line))
   {
     S2431ExecutionIoWarning();
     return false;
   }
   ResetLastError();
   FileFlush(g2431ExecHandle);
   if(GetLastError()!=0)
   {
     S2431ExecutionIoWarning();
     return false;
   }
   return true;
}

void S2431RecordAccepted(ulong order, ulong deal, uint ret)
{
  S2431RememberTicket(g2431AcceptedOrders,order);
```

```
  if(!S2431HasTicket(g2431ReconciledOrders,order))
     S2431RememberTicket(g2431PendingOrders,order);
  if(!S2431HasTicket(g2431ConfirmedDeals,deal))
     S2431RememberTicket(g2431AcceptedDeals,deal);

S2431WriteExecutionEvent("ORDER_ACCEPTED",order,deal,0,TimeTradeServer(),0,0,ret,
                "SEND_RESULT");
  // DONE tambem precisa de prova DEAL_ENTRY_IN. PLACED nunca e prova de negocio.
  if(deal>0) S2431ConfirmEntryDeal(deal);
}

bool S2431BrokerEntryOrder(ulong order)
{
  if(order==0 || !HistoryOrderSelect(order)) return false;
  string symbol,comment; long magic=0,reason=0;
  if(!HistoryOrderGetString(order,ORDER_SYMBOL,symbol) ||
     !HistoryOrderGetString(order,ORDER_COMMENT,comment) ||
     !HistoryOrderGetInteger(order,ORDER_MAGIC,magic) ||
     !HistoryOrderGetInteger(order,ORDER_REASON,reason)) return false;
  return symbol==Symbol() && magic==(long)MagicCore &&
     reason==ORDER_REASON_EXPERT && StringFind(comment,"REMORA2_")==0;
}

bool S2431ConfirmEntryDeal(ulong deal)
{
  if(deal==0 || S2431HasTicket(g2431ConfirmedDeals,deal)) return false;
  if(!HistoryDealSelect(deal)) return false;
    // ENTRY_IN/BUY valem zero: falha de getter nunca e prova de execucao.
    string symbol; long magic=0,entry=0,kind=0,orderRaw=0,positionRaw=0,timeRaw=0;
    double price=0.0,volume=0.0;
    if(!HistoryDealGetString(deal,DEAL_SYMBOL,symbol) ||
       !HistoryDealGetInteger(deal,DEAL_MAGIC,magic) ||
       !HistoryDealGetInteger(deal,DEAL_ENTRY,entry) ||
       !HistoryDealGetInteger(deal,DEAL_TYPE,kind) ||
       !HistoryDealGetInteger(deal,DEAL_ORDER,orderRaw) ||
       !HistoryDealGetInteger(deal,DEAL_POSITION_ID,positionRaw) ||
       !HistoryDealGetInteger(deal,DEAL_TIME,timeRaw) ||
       !HistoryDealGetDouble(deal,DEAL_PRICE,price) ||
       !HistoryDealGetDouble(deal,DEAL_VOLUME,volume)) return false;
    if(symbol!=Symbol() || magic!=(long)MagicCore || entry!=DEAL_ENTRY_IN ||
       (kind!=DEAL_TYPE_BUY && kind!=DEAL_TYPE_SELL) || orderRaw==0 ||
       positionRaw==0 || timeRaw<=0 || !MathIsValidNumber(price) ||
       !MathIsValidNumber(volume) || price<=0.0 || volume<=0.0) return false;
    ulong order=(ulong)orderRaw,position=(ulong)positionRaw;
    datetime stamp=(datetime)timeRaw;
    bool known=S2431HasTicket(g2431AcceptedOrders,order) ||
          S2431HasTicket(g2431AcceptedDeals,deal);
    string source="SEND_RESULT_LINK";
```

```
   if(!known)
   {
     // Recupera queda depois do aceite e antes da gravacao local: a ordem
     // historica EXATA comprova o vinculo com a solicitacao REMORA2_.
     // Sem comentario preservado pela corretora e sem diario nao inventa vinculo.
     if(!S2431BrokerEntryOrder(order)) return false;
     source="BROKER_ORDER_RECOVERY";
   }
   S2431RememberTicket(g2431ConfirmedDeals,deal);
   S2431ForgetTicket(g2431AcceptedDeals,deal);
   S2431WriteExecutionEvent("EXECUTION_CONFIRMED",order,deal,position,stamp,price,
                volume,0,source);
   PrintFormat("[REMORA2.43.1][EXECUTION_CONFIRMED] Ordem=%I64u Deal=%I64u "
          "PosicaoID=%I64u Preco=%.5f Volume=%.2f Fonte=%s",
          order,deal,position,price,volume,source);
   return true;
}

void S2431ReconcileAcceptedOrder(ulong order)
{
  // Consulta EXATA de uma solicitacao conhecida, inclusive de dias anteriores.
  // O bootstrap global continua limitado ao dia atual.
  if(!HistoryOrderSelect(order)) return;
  string symbol;
  long magic=0,state=0,positionValue=0;
  if(!HistoryOrderGetString(order,ORDER_SYMBOL,symbol) ||
     !HistoryOrderGetInteger(order,ORDER_MAGIC,magic) ||
     !HistoryOrderGetInteger(order,ORDER_STATE,state) ||
     !HistoryOrderGetInteger(order,ORDER_POSITION_ID,positionValue)) return;
  if(symbol!=Symbol() || magic!=(long)MagicCore) return;
  if(state!=ORDER_STATE_FILLED && state!=ORDER_STATE_CANCELED &&
     state!=ORDER_STATE_REJECTED && state!=ORDER_STATE_EXPIRED) return;
  ulong position=(ulong)positionValue;
  double initial=0,current=0;
  if(!HistoryOrderGetDouble(order,ORDER_VOLUME_INITIAL,initial) ||
     !HistoryOrderGetDouble(order,ORDER_VOLUME_CURRENT,current)) return;
  double expected=(state==ORDER_STATE_FILLED)?initial:initial-current;
  if(!MathIsValidNumber(initial) || !MathIsValidNumber(current) ||
     initial<0 || current<0 || current>initial || expected<0) return;
  double observed=0;
  if(position>0)
  {
     if(!HistorySelectByPosition(position)) return;
     int n=HistoryDealsTotal();
     ulong tickets[];
     if(ArrayResize(tickets,n)!=n) return;
     for(int i=0;i<n;i++) tickets[i]=HistoryDealGetTicket(i);
     for(int i=0;i<n;i++)
```

```
    {
      ulong deal=tickets[i];
      if(!HistoryDealSelect(deal)) return;
      long dealOrder=0,dealMagic=0,entry=0,kind=0;
      string dealSymbol;
      if(!HistoryDealGetInteger(deal,DEAL_ORDER,dealOrder)) return;
      if((ulong)dealOrder!=order) continue;
      if(!HistoryDealGetString(deal,DEAL_SYMBOL,dealSymbol) ||
        !HistoryDealGetInteger(deal,DEAL_MAGIC,dealMagic) ||
        !HistoryDealGetInteger(deal,DEAL_ENTRY,entry) ||
        !HistoryDealGetInteger(deal,DEAL_TYPE,kind)) return;
      if(dealSymbol!=Symbol() || dealMagic!=(long)MagicCore || entry!=DEAL_ENTRY_IN)
continue;
      if(kind!=DEAL_TYPE_BUY && kind!=DEAL_TYPE_SELL) continue;
      double volume=0;
      if(!HistoryDealGetDouble(deal,DEAL_VOLUME,volume)) return;
      if(!MathIsValidNumber(volume) || volume<=0) return;
      S2431ConfirmEntryDeal(deal);
      if(!S2431HasTicket(g2431ConfirmedDeals,deal)) return;
      observed+=volume;
    }
  }
  // Nao encerrar na primeira confirmacao: todos os fragmentos devem existir.
  if(MathAbs(observed-expected)>1e-8 ||
    (state==ORDER_STATE_FILLED && initial<=0)) return;
  S2431WriteExecutionEvent("ORDER_RECONCILED",order,0,position,TimeTradeServer(),
                0,observed,0,"BROKER_TERMINAL");
   S2431RememberTicket(g2431ReconciledOrders,order);
   S2431ForgetTicket(g2431PendingOrders,order);
}

void S2431ReconcileExecutions()
{
  datetime now=TimeTradeServer();
  if(now<g2431ExecNextReconcile) return;
  g2431ExecNextReconcile=now+10;
  MqlDateTime dt; TimeToStruct(now,dt);
  datetime dayStart=now-(datetime)(dt.hour*3600+dt.min*60+dt.sec);
  if(HistorySelect(dayStart,now))
  {
    // HistoryDealSelect redefine a selecao. Copiar tickets ANTES de confirmar.
    int n=HistoryDealsTotal();
    ulong tickets[];
    if(ArrayResize(tickets,n)!=n) return;
    for(int i=0;i<n;i++) tickets[i]=HistoryDealGetTicket(i);
    for(int i=0;i<n;i++) S2431ConfirmEntryDeal(tickets[i]);
  }
  for(int i=ArraySize(g2431AcceptedDeals)-1;i>=0;i--)
```

```
    S2431ConfirmEntryDeal(g2431AcceptedDeals[i]);
  int np=ArraySize(g2431PendingOrders);
  ulong pending[];
  if(ArrayResize(pending,np)!=np) return;
  for(int i=0;i<np;i++) pending[i]=g2431PendingOrders[i];
  for(int i=0;i<np;i++) S2431ReconcileAcceptedOrder(pending[i]);
}

void S2431ExecutionInit()
{
  ArrayResize(g2431AcceptedOrders,0);
  ArrayResize(g2431AcceptedDeals,0);
  ArrayResize(g2431ConfirmedDeals,0);
  ArrayResize(g2431PendingOrders,0);
  ArrayResize(g2431ReconciledOrders,0);
  g2431ExecNextReconcile=0;
  g2431ExecIoWarned=false;
  string server=AccountInfoString(ACCOUNT_SERVER);
  string unsafe="\\/:*?\"<>|";
  for(int i=0;i<StringLen(unsafe);i++)
    StringReplace(server,StringSubstr(unsafe,i,1),"_");
  string
name=CSVFileName+"_"+IntegerToString(AccountInfoInteger(ACCOUNT_LOGIN))+"_"+
        server+"_"+Symbol()+"_"+IntegerToString(MagicCore)+"_Execution_v2431.csv";
  g2431ExecHandle=FileOpen(name,FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI);
  if(g2431ExecHandle==INVALID_HANDLE)
    S2431ExecutionIoWarning();
   else
   {
    while(!FileIsEnding(g2431ExecHandle))
    {
      string line=FileReadString(g2431ExecHandle);
      string fields[];
      if(StringSplit(line,44,fields)!=9) continue; // ',', nove campos fixos
      ulong order=0,deal=0;
      if(!S2431ParseTicket(fields[1],order) || !S2431ParseTicket(fields[2],deal)) continue;
      if(fields[0]=="ORDER_ACCEPTED" && fields[8]=="SEND_RESULT")
      {
        S2431RememberTicket(g2431AcceptedOrders,order);
        if(!S2431HasTicket(g2431ReconciledOrders,order))
          S2431RememberTicket(g2431PendingOrders,order);
        if(!S2431HasTicket(g2431ConfirmedDeals,deal))
          S2431RememberTicket(g2431AcceptedDeals,deal);
      }
      else if(fields[0]=="EXECUTION_CONFIRMED" &&
           (fields[8]=="SEND_RESULT_LINK" ||
fields[8]=="BROKER_ORDER_RECOVERY"))
      {
```

```
         S2431RememberTicket(g2431ConfirmedDeals,deal);
         S2431ForgetTicket(g2431AcceptedDeals,deal);
       }
       else if(fields[0]=="ORDER_RECONCILED" && fields[8]=="BROKER_TERMINAL")
       {
         S2431RememberTicket(g2431AcceptedOrders,order);
         S2431RememberTicket(g2431ReconciledOrders,order);
         S2431ForgetTicket(g2431PendingOrders,order);
       }
     }
     if(FileSize(g2431ExecHandle)==0)
     {
       if(FileWriteString(g2431ExecHandle,

"EVENT,ORDER,DEAL,POSITION,TIME,PRICE,VOLUME,RETCODE,SOURCE\r\n")==0)
         S2431ExecutionIoWarning();
      FileFlush(g2431ExecHandle);
    }
  }
  S2431ReconcileExecutions();
}

void S2431ExecutionDeinit()
{
  if(g2431ExecHandle!=INVALID_HANDLE) FileClose(g2431ExecHandle);
  g2431ExecHandle=INVALID_HANDLE;
}
void OnTradeTransaction(const MqlTradeTransaction &trans,
                const MqlTradeRequest &request,
                const MqlTradeResult &result)
{
  EodObserveTransaction(trans,request,result);
  // request/result so sao definidos para TRADE_TRANSACTION_REQUEST;
  // para DEAL_ADD a evidencia autoritativa e o deal historico.
  if(trans.type==TRADE_TRANSACTION_DEAL_ADD && trans.deal>0)
     S2431ConfirmEntryDeal(trans.deal);
}

//+------------------------------------------------------------------+
//| OnInit                                                           |
//+------------------------------------------------------------------+
int OnInit()
{
   //=== CIRURGIA 11 — CLASSIFICAÇÃO DO SÍMBOLO E PRESETS ============
   string sym = Symbol();
   gIsDollarFamily = (StringFind(sym,"WDO")>=0 || StringFind(sym,"DOL")>=0);
   gIsIndexFamily = (!gIsDollarFamily) && (StringFind(sym,"WIN")>=0 ||
StringFind(sym,"IND")>=0);
```

```
 if(!gIsDollarFamily && !gIsIndexFamily)
 {
   PrintFormat("[REMORA2.0][FATAL] Símbolo '%s' não pertence às famílias WIN/IND ou
WDO/DOL.", sym);
   return INIT_PARAMETERS_INCORRECT;
 }

 //=== VALIDAÇÃO DAS DISTÂNCIAS FIXAS C2/C3 (genéricas, painel/.set) ==========
 // Regra obrigatória: 0 < C3 mínimo <= C3 máximo < C2 longa
 if(Inp_Pts_C3_Min <= 0.0 ||
   Inp_Pts_C3_Max <= 0.0 ||
   Inp_Pts_C2_Longa <= 0.0 ||
   Inp_Pts_C3_Min > Inp_Pts_C3_Max ||
   Inp_Pts_C2_Longa <= Inp_Pts_C3_Max)
 {
   PrintFormat("[REMORA2.0][FATAL] Parâmetros C2/C3 inválidos | C3=[%.2f a %.2f] |
C2>=%.2f | Regra: 0 < C3min <= C3max < C2longa",
          Inp_Pts_C3_Min, Inp_Pts_C3_Max, Inp_Pts_C2_Longa);
   return INIT_PARAMETERS_INCORRECT;
 }

 //=== [FIX-5] VALIDAÇÃO C1 ABS — somente com a rota ABS habilitada ===
 // v2.41: Inp_Min_Displ_Sigma classifica o estado da POC (STALL x
 // DIVERGENT/FOLLOWING); Inp_Max_Body_Sigma sustenta o contexto STALL
 // e a telemetria do corpo. Ambos permanecem obrigatórios.
  if(Inp_EnableAbsorption &&
     (Inp_Min_Displ_Sigma <= 0.0 ||
      Inp_Max_Body_Sigma <= 0.0))
  {
     PrintFormat("[REMORA2.0][FATAL] Parametros C1 ABS invalidos | MinDisplSigma=%.4f
| MaxBodySigma=%.4f | Ambos devem ser > 0.",
            Inp_Min_Displ_Sigma, Inp_Max_Body_Sigma);
     return INIT_PARAMETERS_INCORRECT;
  }

  //=== [v2.40] VALIDAÇÃO DAS ROTAS ===================================
  if(!Inp_EnableAbsorption && !Inp_EnableContinuation)
  {
     Print("[REMORA2.0][FATAL] Nenhuma rota habilitada (Inp_EnableAbsorption e
Inp_EnableContinuation = false).");
     return INIT_PARAMETERS_INCORRECT;
  }

 // [FIX-5] Validação CONT — somente com a rota CONT habilitada
 if(Inp_EnableContinuation &&
    (Inp_Cont_Vol_Effort   <= 0.0 ||
     Inp_Cont_Min_Displ_Sigma <= 0.0 ||
```

```
    Inp_Cont_Breakout_Effort <= 0.0))
 {
   PrintFormat("[REMORA2.0][FATAL] Parametros CONT invalidos | Cont_Vol_Effort=%.4f |
Cont_Min_Displ_Sigma=%.4f | Cont_Breakout_Effort=%.4f | Todos devem ser > 0.",
         Inp_Cont_Vol_Effort, Inp_Cont_Min_Displ_Sigma, Inp_Cont_Breakout_Effort);
   return INIT_PARAMETERS_INCORRECT;
 }

  PrintFormat("[REMORA2.0][C2C3_FIXOS] C3=[%.2f a %.2f] | C2>=%.2f | Valores
carregados diretamente do painel/.set | Independentes do Sigma",
        Inp_Pts_C3_Min, Inp_Pts_C3_Max, Inp_Pts_C2_Longa);

 gTickSize = SymbolInfoDouble(Symbol(), SYMBOL_TRADE_TICK_SIZE);
 if(gTickSize <= 0.0) gTickSize = SymbolInfoDouble(Symbol(), SYMBOL_POINT);
 if(gTickSize <= 0.0)
 {
   PrintFormat("[REMORA2.0][FATAL] Tick size inválido (<=0) para %s.", sym);
   return INIT_PARAMETERS_INCORRECT;
 }

 //=== [BLINDAGEM E2] VALIDAÇÃO DO TETO DE OVERSHOOT DO TRIGGER ====
 if(Inp_Max_Trigger_Overshoot <= 0.0)
 {
   Print("[REMORA2.0][FATAL] Inp_Max_Trigger_Overshoot deve ser maior que zero.");
   return INIT_PARAMETERS_INCORRECT;
 }
 {
   double overshootRatio = Inp_Max_Trigger_Overshoot / gTickSize;
   if(MathAbs(overshootRatio - MathRound(overshootRatio)) > 1e-6)
   {
     PrintFormat("[REMORA2.0][FATAL] Inp_Max_Trigger_Overshoot=%.4f não é múltiplo
do tick size=%.4f.",
           Inp_Max_Trigger_Overshoot, gTickSize);
     return INIT_PARAMETERS_INCORRECT;
   }
 }

  //--- Validações FATAIS (sem arredondamento silencioso) -----------
  if(Inp_Max_SL_Pts <= 0.0)
  { Print("[REMORA2.0][FATAL] Inp_Max_SL_Pts <= 0."); return
INIT_PARAMETERS_INCORRECT; }
  if(gIsIndexFamily && Inp_Max_SL_Pts < 100.0)
  { PrintFormat("[REMORA2.0][FATAL] WIN/IND exige Inp_Max_SL_Pts >= 100
(atual=%.1f).", Inp_Max_SL_Pts); return INIT_PARAMETERS_INCORRECT; }
  if(gIsDollarFamily && Inp_Max_SL_Pts > 100.0)
  { PrintFormat("[REMORA2.0][FATAL] WDO/DOL exige Inp_Max_SL_Pts <= 100
(atual=%.1f).", Inp_Max_SL_Pts); return INIT_PARAMETERS_INCORRECT; }
  if(gIsIndexFamily && Inp_Breakout_Margin < 5.0)
```

```
  { PrintFormat("[REMORA2.0][FATAL] WIN/IND exige Inp_Breakout_Margin >= 5.0
(atual=%.2f).", Inp_Breakout_Margin); return INIT_PARAMETERS_INCORRECT; }
  if(gIsDollarFamily && Inp_Breakout_Margin < 2.0)
  { PrintFormat("[REMORA2.0][FATAL] WDO/DOL exige Inp_Breakout_Margin >= 2.0
(atual=%.2f).", Inp_Breakout_Margin); return INIT_PARAMETERS_INCORRECT; }
  {
    double ratio = Inp_Breakout_Margin / gTickSize;
    if(MathAbs(ratio - MathRound(ratio)) > 1e-6)
    {
      PrintFormat("[REMORA2.0][FATAL] Inp_Breakout_Margin=%.4f não é múltiplo do tick
size=%.4f.",
            Inp_Breakout_Margin, gTickSize);
      return INIT_PARAMETERS_INCORRECT;
    }
  }

  //--- Avisos NÃO fatais de preset (magic/spread/deviation) ---------
  if(gIsIndexFamily)
  {
    if(MagicCore != 50001)
      PrintFormat("[WARNING_PRESET] WIN/IND: MagicCore esperado=50001, atual=%d
(ajuste deliberado permitido).", MagicCore);
    if(SpreadMaxPts != 5.0)
      PrintFormat("[WARNING_PRESET] WIN/IND: SpreadMaxPts esperado=5.0,
atual=%.2f (ajuste deliberado permitido).", SpreadMaxPts);
    if(Inp_MaxDeviationPts != 10)
     PrintFormat("[WARNING_PRESET] WIN/IND: Inp_MaxDeviationPts esperado=10,
atual=%d (ajuste deliberado permitido).", Inp_MaxDeviationPts);
  }
  else
  {
   if(MagicCore != 20001)
     PrintFormat("[WARNING_PRESET] WDO/DOL: MagicCore esperado=20001, atual=%d
(ajuste deliberado permitido).", MagicCore);
   if(SpreadMaxPts != 1.5)
     PrintFormat("[WARNING_PRESET] WDO/DOL: SpreadMaxPts esperado=1.5,
atual=%.2f (ajuste deliberado permitido).", SpreadMaxPts);
   if(Inp_MaxDeviationPts != 2)
     PrintFormat("[WARNING_PRESET] WDO/DOL: Inp_MaxDeviationPts esperado=2,
atual=%d (ajuste deliberado permitido).", Inp_MaxDeviationPts);
  }

   //=== CIRURGIA 12 — MODO DA CONTA ================================
   ENUM_ACCOUNT_MARGIN_MODE mm =
(ENUM_ACCOUNT_MARGIN_MODE)AccountInfoInteger(ACCOUNT_MARGIN_MODE);
   gNetting = (mm != ACCOUNT_MARGIN_MODE_RETAIL_HEDGING);
   if(gNetting)
```

```
     Print("[NETTING_POLICY] Uma Rêmora por símbolo por conta. Magic não isola posição
líquida.");

 trade.SetExpertMagicNumber(MagicCore);
 trade.SetDeviationInPoints(Inp_MaxDeviationPts);

 if(!IsFokSupported())
 {
   PrintFormat("[REMORA2.0][FATAL] FOK não suportado para %s. EA bloqueado por
política de execução.",
           Symbol());
   return INIT_PARAMETERS_INCORRECT;
 }
 trade.SetTypeFilling(ORDER_FILLING_FOK); // FOK obrigatório (herdado V1)

  //=== CIRURGIA 13.1 — TIMER VALIDADO ==============================
  int timerSeconds = (MQLInfoInteger(MQL_TESTER) != 0) ? 60 : 1;
  if(!EventSetTimer(timerSeconds))
  {
    Print("[REMORA2.0][FATAL] EventSetTimer falhou. EA não pode operar sem OnTimer
(reset diário/D-1/EOD).");
    return INIT_FAILED;
  }

 //=== Estado inicial ==============================================
 MqlDateTime dtNow; TimeToStruct(TimeTradeServer(), dtNow);
 gDayKey = dtNow.year*1000 + dtNow.day_of_year;
 beCount = 0;
 beSyncPending = false;
 foreignAlertActive = false;
 eodDone = false; eodStuckLogged = false; eodNextTry = 0;
 gLastMatrixEvalBar = 0;       // [v2.42] guarda de avaliação única por M5

  ResetC1AuditSnapshot(gC1Audit); // Cirurgia 5 — call site 1
  ResetArmedSignal();      // [E6] limpa armed + armed.audit — sinal armado NUNCA é
restaurado de reinício
  ResetVwapCache();
  wwValid = false;

 d1Ready=false; nextD1Retry=0;
 RestoreDailyPersistence(); // [2.435/F1] restaura somente a cota do DIA ATUAL
 TryCalcD1();         // [2.435/F1] tentativa inicial, sem fallback de dados
 ReloadOpenPositionsToBE();

 // Cirurgia 9 — semeia lastBarTime; NÃO avalia matriz até a próxima virada real
 lastBarTime = iTime(Symbol(), Inp_Timeframe, 0);

 //=== CIRURGIA 6 — CSV VERSIONADO EM APPEND ======================
```

```
 // Estrutura de 44 colunas INALTERADA; a rota viaja no campo MODE.
 // [v2.43] Nome do arquivo: _Audit_v243.csv (mesmas 44 colunas).
 string fname =
    CSVFileName + "_" +
    Symbol() + "_" +
    IntegerToString(MagicCore) +
    "_Audit_v243.csv";

 csvHandle = FileOpen(fname, FILE_READ|FILE_WRITE|FILE_CSV|FILE_ANSI, ',');
 if(csvHandle != INVALID_HANDLE)
 {
    if(FileSize(csvHandle) == 0)
    {
       FileWrite(csvHandle,
         "TIMESTAMP","STATUS","DIR","MODE",
         "PRICE","POSREL","DISTD1","EFFORT_WW","TRIGGER","POC_STATE",
         "C1","C2","C3","C4","C5",
         "LOTS","RET",
         "VWAP_VIG","VWAP_D1","SIGMA","H1","L1",

"C23","SCORE_EFF","C1_EFFORT_OK","C1_BODY_OK","C1_DISPLACEMENT_OK","C1
_POC_OK","C1_MOVEDIR_OK","C1_WAVEDIR_OK",
       "CANDLE_BODY","MAX_DISPL_THRESHOLD",
       "PRICE_MOVE","POC_MOVE","MOVE_DIR","WAVE_DIR","C1_POC_STATE",
       "VOL_WINDOW_NOW","VOL_WINDOW_PREV","POC_NOW","POC_PREV",
       "LAST_PRICE_WINDOW_NOW","LAST_PRICE_WINDOW_PREV","ANCHORED");
    }
    FileSeek(csvHandle, 0, SEEK_END);
    FileFlush(csvHandle);
 }
 else
    PrintFormat("[REMORA2.0][CSV_FAIL] Auditoria indisponivel | Erro=%d | EA segue
operando SEM telemetria.", GetLastError());

 //=== [v2.43] CSV AUXILIAR SHADOW (telemetria versionada) ==========
 // Eventos: ARMED_SHADOW / ABS_STALL_RAW_ONLY /
ROUTE_CONFLICT_SHADOW /
 //      ABS_FOLLOWING_EXTENSION_BLOCK / POSITION_SHADOW.
 // Gravado como TEXTO (82 colunas > limite de 63 parâmetros do FileWrite).
 string sname =
    CSVFileName + "_" +
    Symbol() + "_" +
    IntegerToString(MagicCore) +
    "_Shadow_v243.csv";

  shadowHandle = FileOpen(sname, FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI);
  if(shadowHandle != INVALID_HANDLE)
  {
```

```
    if(FileSize(shadowHandle) == 0)
      WriteShadowHeader();
    FileSeek(shadowHandle, 0, SEEK_END);
    FileFlush(shadowHandle);
  }
  else
    PrintFormat("[REMORA2.0][SHADOW_CSV_FAIL] Telemetria shadow indisponivel |
Erro=%d | EA segue operando (shadow é somente telemetria).", GetLastError());

   //=== Linha única de identidade (Cirurgia 11 + Endurecimento) =====
   PrintFormat("[REMORA2.0 v2.436 CANDIDATA] %s | Familia=%s | Conta=%s | Magic=%d
| TickSize=%.4f | SpreadMax=%.2f | SLTeto=%.1f | Dev=%d | BreakMargin=%.2f |
MaxOvershoot=%.2f | TF=%s | C1=OBRIGATORIO |
C1_ABS=EFFORT+(DIVERGENCE|STALL_STRICT|FOLLOWING_REACTION) |
FOLLOWING_EXTENSION=NAO_VALIDA | BODY=SO_STALL |
STALL_RAW=TELEMETRIA | FLOW_REGIME=TELEMETRIA |
waveDirFechado_E1=TELEMETRIA | WaveViva_E2=OBRIGATORIA |
GLASS=POR_ROTA(E1)+DIRECIONAL_ABS(E2) |
ARMED_END=APS(TIMEOUT=>REEVAL_1x) | SHADOW=READ_ONLY |
POSITION_SHADOW=READ_ONLY | D1=LITERAL |
PERSIST=SHOT|D1A_LEGACY_IGNORED | ROTAS: ABS=%s CONT=%s",
          sym,
          gIsIndexFamily ? "WIN/IND" : "WDO/DOL",
          gNetting ? "NETTING/EXCHANGE" : "HEDGING",
          MagicCore, gTickSize, SpreadMaxPts, Inp_Max_SL_Pts,
          Inp_MaxDeviationPts, Inp_Breakout_Margin, Inp_Max_Trigger_Overshoot,
          EnumToString(Inp_Timeframe),
            Inp_EnableAbsorption ? "ON" : "OFF",
            Inp_EnableContinuation ? "ON" : "OFF");

   S2431ExecutionInit(); // [2.43.1/S2] diario auxiliar, sem veto operacional
   Diag2435Init(); // [2.435/D] auxiliar; falha nao afeta INIT_SUCCEEDED

   return INIT_SUCCEEDED;
}

void OnDeinit(const int reason)
{
  EventKillTimer();
  S2431ExecutionDeinit();
  Diag2435Deinit();
  if(csvHandle != INVALID_HANDLE){ FileClose(csvHandle);
csvHandle=INVALID_HANDLE; }
  if(shadowHandle != INVALID_HANDLE){ FileClose(shadowHandle);
shadowHandle=INVALID_HANDLE; } // [v2.42]
}

//+------------------------------------------------------------------+
```

```
//| [VPS RECOVERY] Reload Break Even — SOMENTE posições próprias |
//+------------------------------------------------------------------+
void ReloadOpenPositionsToBE()
{
   beCount=0;
   for(int i=PositionsTotal()-1;i>=0;i--)
   {
     if(!posInfo.SelectByIndex(i)) continue;
     if(posInfo.Symbol()!=Symbol()) continue;
     if(posInfo.Magic() !=(long)MagicCore) continue; // estrangeiras NUNCA entram no BE
     if(beCount>=64) break;

        double entry = posInfo.PriceOpen();
        double curSL = posInfo.StopLoss();
        beList[beCount].ticket    = posInfo.Ticket();
        beList[beCount].applied    = (curSL>0.0 && TickIndex(curSL)==TickIndex(entry));
        beList[beCount].backoffUntil = 0;
        beCount++;
    }
}

//+------------------------------------------------------------------+
//| POSIÇÕES — GATES POR MODO DE CONTA (Cirurgia 12)                 |
//+------------------------------------------------------------------+
bool HasOpenPosition()
{
   for(int i=PositionsTotal()-1;i>=0;i--)
 {
    if(!posInfo.SelectByIndex(i)) continue;
    if(posInfo.Symbol()!=Symbol()) continue;
    if(gNetting) return true;               // NETTING: qualquer posição no símbolo bloqueia
    if(posInfo.Magic()==(long)MagicCore) return true; // HEDGING: símbolo+magic
  }
  return false;
}

bool HasOwnPositionStrict()
{
  for(int i=PositionsTotal()-1;i>=0;i--)
  {
    if(!posInfo.SelectByIndex(i)) continue;
    if(posInfo.Symbol()!=Symbol()) continue;
    if(posInfo.Magic()==(long)MagicCore) return true;
  }
  return false;
}

bool HasOwnPendingOrder()
```

```
{
  for(int i=OrdersTotal()-1;i>=0;i--)
  {
    ulong tk = OrderGetTicket(i);
    if(tk==0) continue;
    if(OrderGetString(ORDER_SYMBOL)!=Symbol()) continue;
    if(OrderGetInteger(ORDER_MAGIC)==(long)MagicCore) return true;
  }
  return false;
}

// [E5/E7] Ordem própria ativa CRIADA HOJE (símbolo+magic+data) — segunda
// proteção da cota: uma ordem em trânsito bloqueia novo disparo.
// Ordens estrangeiras, de outro símbolo ou de outro dia NÃO contam.
bool HasOwnPendingOrderToday()
{
   MqlDateTime dt; TimeToStruct(TimeTradeServer(), dt);
   dt.hour=0; dt.min=0; dt.sec=0;
   datetime dayStart = StructToTime(dt);

 for(int i=OrdersTotal()-1;i>=0;i--)
 {
   ulong tk = OrderGetTicket(i);
   if(tk==0) continue;
   if(OrderGetString(ORDER_SYMBOL)!=Symbol()) continue;
   if(OrderGetInteger(ORDER_MAGIC)!=(long)MagicCore) continue;
      if((datetime)OrderGetInteger(ORDER_TIME_SETUP) >= dayStart) return true;
    }
    return false;
}

// Posição estrangeira em Netting: log crítico 1x por episódio; flag limpa ao flat
void UpdateForeignPositionState()
{
   if(!gNetting) return;
   bool any=false, foreign=false;
   for(int i=PositionsTotal()-1;i>=0;i--)
   {
      if(!posInfo.SelectByIndex(i)) continue;
      if(posInfo.Symbol()!=Symbol()) continue;
      any=true;
      if(posInfo.Magic()!=(long)MagicCore) foreign=true;
   }
   if(foreign && !foreignAlertActive)
   {
      foreignAlertActive=true;
      PrintFormat("[REMORA2.0][NETTING_FOREIGN] CRÍTICO: posição estrangeira em %s
(magic != %d). Novas entradas bloqueadas; BE/EOD NÃO atuarão sobre ela.",
```

```
            Symbol(), MagicCore);
  }
  if(!any && foreignAlertActive)
  {
    foreignAlertActive=false;
    Print("[REMORA2.0][NETTING_FOREIGN] Símbolo flat. Alerta limpo; fluxo normal
restabelecido.");
  }
}

//+------------------------------------------------------------------+
//| OnTick — MOTOR PRINCIPAL (ordem da Cirurgia 9 / Seção XVI)       |
//| SEM EOD e SEM reset diário aqui (ambos no OnTimer).              |
//| v2.42: com Armed ativo e nova M5, roda SHADOW (read-only) antes  |
//| do Estágio 2. Se o Armed termina com APS_ENDED_REEVAL_ALLOWED    |
//| nesta nova M5, o fluxo segue pelos gates normais e avalia a      |
//| matriz UMA ÚNICA vez (guarda gLastMatrixEvalBar).                |
//| v2.43: com posição aberta (gate preservado), a nova M5 da sessão |
//| gera POSITION_SHADOW read-only ANTES do return já existente.     |
//+------------------------------------------------------------------+
void OnTick()
{
  // 1) Gate BacktestStart
  if(TimeCurrent() < BacktestStart) return;

   // 2) Break-even primeiro
 ManageBreakEven();

 // 3-4) Detecção de barra
 datetime bt = iTime(Symbol(), Inp_Timeframe, 0);
 if(bt <= 0) return;

 // 5) newBar
 bool newBar = (bt != lastBarTime);

 // 6) Nova M5: atualiza caches (MESMO com sinal armado)
 if(newBar)
 {
   lastBarTime = bt; // imediatamente
   gVwapOk = UpdateVwapIncremental(bt);
   gVwapValue = gVwapOk ? (vwapSumPV / vwapSumVol) : 0.0;
   if(gVwapOk && gVwapValue <= 0.0) gVwapOk = false;
   R2436DiagPublishedCache(bt); // observacao apos publicacao original
   RebuildWeisCache();
 }

 MqlDateTime dt; TimeToStruct(TimeTradeServer(), dt);
 int hhmm = dt.hour*100 + dt.min;
```

```
  // 7) Sinal armado processa tick a tick
  if(armed.active)
  {
    // [2.436/A] Uma unica SHADOW por nova barra do timeframe configurado.
    // So esta verificacao reutiliza seu snapshot; a eventual reavaliacao
    // operacional posterior continua pelo EvaluateMatrix original.
    bool hardCancelled=false;
    if(newBar)
    {
      MatrixSnapshot shadowSnapshot;
      RunArmedShadow(shadowSnapshot);
      if(HasHardDirectionalContradiction(shadowSnapshot,armed.dir))
      {
        PrintFormat("[REMORA2.436][HARD_DIRECTIONAL_CONTRADICTION]
Symbol=%s Magic=%d Bar=%s TF=%s | ArmedDir=%s Route=%s Mode=%s Trigger=%.8f |
ABS Dir=%s Valid=%d OppositeDirValid=%d C1=%d C2=%d C3=%d C23=%d C4=%d
C5=%d Score=%d/3 Class=%s | CONT Dir=%s Valid=%d OppositeDirValid=%d C1=%d
C2=%d C3=%d C23=%d C4=%d C5=%d Score=%d/3 Class=%s | CancelOld=1
Stage2OldSkipped=1 NormalGatesNext=1",

Symbol(),MagicCore,TimeToString(bt,TIME_DATE|TIME_SECONDS),EnumToString(Inp_Ti
meframe),
          DirStr(armed.dir),armed.route,armed.mode,
          ComputeTrigger(armed.dir,armed.regionHigh,armed.regionLow),
          DirStr(shadowSnapshot.dir),(int)shadowSnapshot.absValid,
          (int)(shadowSnapshot.absValid && shadowSnapshot.dir == -armed.dir),
           (int)shadowSnapshot.c1,(int)shadowSnapshot.c2,(int)shadowSnapshot.c3,

(int)shadowSnapshot.c23,(int)shadowSnapshot.c4,(int)shadowSnapshot.c5AbsOk,
          shadowSnapshot.scoreEff,AbsClassStr(shadowSnapshot),
          DirStr(shadowSnapshot.dirCont),(int)shadowSnapshot.contValid,
          (int)(shadowSnapshot.contValid && shadowSnapshot.dirCont == -armed.dir),

(int)shadowSnapshot.c1Cont,(int)shadowSnapshot.c2Cont,(int)shadowSnapshot.c3Cont,

(int)shadowSnapshot.c23Cont,(int)shadowSnapshot.c4Cont,(int)shadowSnapshot.c5ContOk
,
              shadowSnapshot.scoreContEff,ContClassStr(shadowSnapshot));
      // Evento novo no auxiliar existente, com suas mesmas colunas.
      // Snapshot e sinal antigo registrados ANTES do reset.

WriteShadowRow("HARD_DIRECTIONAL_CONTRADICTION","OPERATIONAL",shadowSn
apshot,true);
      ResetArmedSignal();
      hardCancelled=true;
    }
  }
```

```
   // Cancelamento forte nunca chama ProcessArmedSignal neste evento.
   if(!hardCancelled)
   {
   // 7b) Estágio 2 com resultado operacional explícito
   ENUM_ARMED_PROCESS_RESULT aps = ProcessArmedSignal(hhmm);

   if(aps == APS_STILL_ARMED || aps == APS_TRADE_OR_POSITION)
      return; // enquanto armado / após ordem: não reavalia a matriz

    if(aps == APS_ENDED_FAIL_CLOSED)
    {
       if(newBar)
          Print("[REMORA2.0][ARMED_ENDED_FAIL_CLOSED] Sinal encerrado por condição
fail-closed nesta nova M5. Matriz NÃO reavaliada neste evento (registro da M5 disponível
no ARMED_SHADOW).");
       return;
    }

   // aps == APS_ENDED_REEVAL_ALLOWED
   if(!newBar)
   {
      Print("[REMORA2.0][ARMED_ENDED_MIDBAR] Sinal encerrado (REEVAL_ALLOWED)
fora da virada de M5. Sem reavaliação (exige newBar).");
      return;
   }
   if(armed.active) return; // defensivo: REEVAL exige Armed já resetado
   Print("[REMORA2.0][ARMED_ENDED_REEVAL] Sinal anterior encerrado SEM operação
na virada da M5. A M5 fechada segue pelos gates normais (avaliação ÚNICA).");
   // segue para os gates abaixo (newBar == true)
   } // !hardCancelled
 }

    // 8) Sem sinal armado e sem barra nova: nada a fazer
    if(!newBar) return;

  // 9) Gates: sessão / posição / trade diário / cooldown
  if(hhmm < Inp_StartHHMM || hhmm >= Inp_StopEntryHHMM) {
Diag2435M5(DIAG2435_OUTSIDE_SESSION,bt,TimeTradeServer(),hhmm); return; }
  if(HasOpenPosition())
  {
     // [v2.43] POSITION_SHADOW — SOMENTE TELEMETRIA. O bloqueio operacional
     // permanece IDÊNTICO (return logo abaixo). Não arma, não envia, não
     // altera posição/BE/SL/TP/cota/gLastMatrixEvalBar/persistência.
     RunPositionShadow();
     Diag2435M5(DIAG2435_POSITION_GATE,bt,TimeTradeServer(),hhmm);
     return;
  }
```

```
  if(ordersEnteredToday >= Inp_MaxTradesDay) {
Diag2435M5(DIAG2435_DAILY_QUOTA,bt,TimeTradeServer(),hhmm); return; } // Tiro único
diário
  if(lastEntryTime>0 &&
     (bt - lastEntryTime) < (datetime)(Inp_CooldownBars * PeriodSeconds(Inp_Timeframe)))
     { Diag2435M5(DIAG2435_COOLDOWN,bt,TimeTradeServer(),hhmm); return; }

    // 10) Bloqueios de dados: D-1 e caches
    if(!d1Ready) { Diag2435M5(DIAG2435_D1_INVALID,bt,TimeTradeServer(),hhmm); return;
}
 if(!gVwapOk || gVwapValue <= 0.0)
 {
    WriteAudit("VWAP_CACHE_INVALID","NONE","NONE",0,0,0,0,0,"NA",0,0,0,0,0,0,0,0);
    Diag2435M5(DIAG2435_VWAP_INVALID,bt,TimeTradeServer(),hhmm);
    return;
 }
 if(!wwValid)
 {
    Print("[REMORA2.0][WW_CACHE_INVALID] Cache Weis Wave fechado indisponível
nesta M5. Avaliação bloqueada.");
    Diag2435M5(DIAG2435_WEIS_INVALID,bt,TimeTradeServer(),hhmm);
    return;
 }

    // 11) [v2.42] Guarda: a mesma M5 NUNCA passa duas vezes pela matriz operacional
    if(gLastMatrixEvalBar == bt)
    {
   Print("[REMORA2.0][MATRIX_DUP_GUARD] M5 já avaliada operacionalmente. Segunda
avaliação BLOQUEADA.");

Diag2435M5(DIAG2435_MATRIX_ALREADY_EVALUATED,bt,TimeTradeServer(),hhmm);
    return;
  }
  gLastMatrixEvalBar = bt;

    // 12) Avaliação da matriz
    EvaluateMatrix();
}

//+------------------------------------------------------------------+
//| ESTÁGIO 2 — GATILHO DE REAÇÃO (Rompimento de Região)             |
//| Margem em pontos (trigger normalizado ceil/floor) +              |
//| FORÇA da ONDA VIVA Weis Wave >= armed.breakoutEffortMin (O(1)). |
//| A ONDA VIVA (waveDir == armed.dir) é OBRIGATÓRIA nas 2 rotas. |
//| v2.41 — GLASS DINÂMICO DIRECIONAL:                               |
//| CONT: qualquer Glass => LATE_GLASS_BLOCK (inalterado).           |
//| ABS : bloqueia somente Glass do lado OPOSTO ao extremo de        |
//|      origem; Glass do MESMO lado => continua validação           |
```

```
//|     (ABS_LATE_GLASS_SAMESIDE_ALLOW, 1x por sinal).                              |
//| v2.42 — RETORNO EXPLÍCITO (lógica idêntica à v2.41):                            |
//| ABORT_TIME_REF_INVALID => FAIL_CLOSED                                           |
//| TIMEOUT_BREAKOUT                    => REEVAL_ALLOWED                           |
//| ABORT_SESSION_END                    => FAIL_CLOSED                             |
//| OVERSHOOT_BLOCK                     => FAIL_CLOSED (bloqueio protetivo)        |
//| LATE_VWAP_INVALID (x2) => FAIL_CLOSED (integridade)                             |
//| LATE_GLASS_BLOCK                   => FAIL_CLOSED (bloqueio protetivo)         |
//| posição/cota no disparo => TRADE_OR_POSITION                                    |
//| SendOrder aceito             => TRADE_OR_POSITION                               |
//| SendOrder recusado             => FAIL_CLOSED                                   |
//| demais saídas (aguarda) => STILL_ARMED                                          |
//| v2.43: INALTERADO.                                                              |
//+-----------------------------------------------------------------+
ENUM_ARMED_PROCESS_RESULT ProcessArmedSignal(int hhmm)
{
   double trigger = ComputeTrigger(armed.dir, armed.regionHigh, armed.regionLow);

   //--- 1) TIMEOUT TEMPORAL (Janela de rompimento por velas) -------
   // armed.regionEndTime = ABERTURA da última barra fechada da região
   int barsPassed = iBarShift(Symbol(), Inp_Timeframe, armed.regionEndTime, true);

   // [E4] Referência temporal inválida: iBarShift()==-1 desativaria o
   // timeout e deixaria o sinal armado esperando indefinidamente.
   // Política conservadora: cancela o sinal, audita e NÃO dispara.
   if(barsPassed < 0)
 {
   PrintFormat("[REMORA2.0][ABORT_TIME_REF_INVALID] iBarShift=-1 para
regionEndTime=%s. Referência temporal inválida; sinal %s (%s) CANCELADO por
segurança.",
          TimeToString(armed.regionEndTime, TIME_DATE|TIME_SECONDS),
          DirStr(armed.dir), armed.route);
   WriteAudit("ABORT_TIME_REF_INVALID", DirStr(armed.dir), armed.mode, 0,
         armed.posRel, armed.distD1, armed.effort, trigger, "NA",
         armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
   ResetArmedSignal();
   return APS_ENDED_FAIL_CLOSED;
 }

 if(barsPassed > Inp_Breakout_Max_Bars)
 {
   PrintFormat("[REMORA2.0][TIMEOUT_BREAKOUT] %s (%s) não rompeu a região em
%d velas. Sinal falhou (lateralização). ABORTA.",
          DirStr(armed.dir), armed.route, Inp_Breakout_Max_Bars);
   WriteAudit("TIMEOUT_BREAKOUT", DirStr(armed.dir), armed.mode, trigger,
         armed.posRel, armed.distD1, armed.effort, trigger, "NA",
         armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
   ResetArmedSignal();
```

```
   return APS_ENDED_REEVAL_ALLOWED;
 }

 //--- 2) Segurança extra: Fim da sessão -------------------------
 if(hhmm >= Inp_StopEntryHHMM)
 {
   WriteAudit("ABORT_SESSION_END", DirStr(armed.dir), armed.mode, trigger,
         armed.posRel, armed.distD1, armed.effort, trigger, "NA",
         armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
   ResetArmedSignal();
   return APS_ENDED_FAIL_CLOSED;
 }

  MqlTick tk;
  if(!SymbolInfoTick(Symbol(), tk)) {
Diag2435Armed(DIAG2435_ARMED_TICK_UNAVAILABLE,lastBarTime,TimeTradeServer(),
hhmm,trigger,0.0,0.0,0); return APS_STILL_ARMED; }

 // [BLINDAGEM E2] EVENTO DE NEGÓCIO REAL --------------------------
 // SymbolInfoTick mantém Bid/Ask/Last/volume preenchidos mesmo quando
 // somente Bid/Ask mudaram; por isso last>0, isoladamente, não prova
 // que ESTE evento foi um negócio. Exige-se flag de negócio real e
 // usa-se tk.last diretamente (PROIBIDO (tk.bid+tk.ask)/2.0).
 bool realTradeEvent =
    ((tk.flags & TICK_FLAG_LAST) != 0) ||
    ((tk.flags & TICK_FLAG_VOLUME) != 0) ||
     ((tk.flags & TICK_FLAG_BUY) != 0) ||
     ((tk.flags & TICK_FLAG_SELL) != 0);
  if(!realTradeEvent) {
Diag2435Armed(DIAG2435_ARMED_NO_REAL_TRADE,lastBarTime,TimeTradeServer(),h
hmm,trigger,tk.last,0.0,0); return APS_STILL_ARMED; }

  double last = tk.last;
  if(last <= 0.0) {
Diag2435Armed(DIAG2435_ARMED_LAST_INVALID,lastBarTime,TimeTradeServer(),hhm
m,trigger,last,0.0,0); return APS_STILL_ARMED; }

  //--- 3) VALIDAÇÃO DO ROMPIMENTO (trigger normalizado) ----------
  // BUY : last >= trigger | SELL: last <= trigger
  bool breakout = false;
  if(armed.dir == -1 && last <= trigger) breakout = true;
  if(armed.dir == 1 && last >= trigger) breakout = true;
  if(!breakout) {
Diag2435Armed(DIAG2435_ARMED_BREAKOUT_NOT_CONFIRMED,lastBarTime,TimeTr
adeServer(),hhmm,trigger,last,0.0,0); return APS_STILL_ARMED; } // Aguarda
pacientemente dentro da janela de velas permitida

  //--- 4) [BLINDAGEM E2] TETO DE ULTRAPASSAGEM DO TRIGGER ---------
```

```
  // Compra sai no Ask e venda no Bid: mede-se a ultrapassagem do último
  // negócio E do preço executável, bloqueando pela pior delas (MathMax).
  double executablePrice = (armed.dir == 1) ? tk.ask : tk.bid;
  if(executablePrice <= 0.0) {
Diag2435Armed(DIAG2435_ARMED_EXECUTABLE_INVALID,lastBarTime,TimeTradeServ
er(),hhmm,trigger,last,0.0,0); return APS_STILL_ARMED; }

 double overshootLast =
  (armed.dir == 1) ? (last - trigger) : (trigger - last);
 double overshootExecutable =
  (armed.dir == 1) ? (executablePrice - trigger) : (trigger - executablePrice);
 double overshoot = MathMax(overshootLast, overshootExecutable);

  if(overshoot > Inp_Max_Trigger_Overshoot)
  {
     PrintFormat("[REMORA2.0][OVERSHOOT_BLOCK] Rompimento ultrapassou o teto |
Last=%.2f | ExecPrice=%.2f | Trigger=%.2f | OverLast=%.2f | OverExec=%.2f | Max=%.2f |
Sinal CANCELADO.",
            last, executablePrice, trigger, overshootLast, overshootExecutable,
            Inp_Max_Trigger_Overshoot);
     WriteAudit("OVERSHOOT_BLOCK", DirStr(armed.dir), armed.mode, last,
           armed.posRel, armed.distD1, armed.effort, trigger, "NA",
           armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
     ResetArmedSignal();
     return APS_ENDED_FAIL_CLOSED;
  }
  //--- 5) FORÇA DA ONDA VIVA (WW1P via cache O(1) — Cirurgia 8) --
  if(!wwValid) {
Diag2435Armed(DIAG2435_ARMED_WEIS_INVALID,lastBarTime,TimeTradeServer(),hhm
m,trigger,last,0.0,0); return APS_STILL_ARMED; } // cache não pronto neste tick: não
dispara

 double curEff, curVol, avgVol; int waveDir;
 ComputeLiveWaveO1(curEff, curVol, avgVol, waveDir);

 if(waveDir != armed.dir)
 {
    // Onda viva ainda não está a favor da direção do sinal.

Diag2435Armed(DIAG2435_ARMED_WAVE_DIRECTION,lastBarTime,TimeTradeServer(),h
hmm,trigger,last,curEff,waveDir);
     return APS_STILL_ARMED;
  }
  if(curEff < armed.breakoutEffortMin)
  {
     // Esforço da ONDA VIVA insuficiente (mínimo da ROTA armada) => ignora o tick.
```

```
Diag2435Armed(DIAG2435_ARMED_EFFORT_INSUFFICIENT,lastBarTime,TimeTradeServ
er(),hhmm,trigger,last,curEff,waveDir);
   return APS_STILL_ARMED;
 }

 //--- 6) [BLINDAGEM E2] REVALIDAÇÃO DINÂMICA DO C5 (Teto de Vidro)
 // VWAP viva reconstruída no instante do negócio (snapshot local).
 // NÃO usa UpperBand1/LowerBand1/armed.vwapVig (bandas congeladas no
 // Estágio 1); apenas variáveis locais. armed.vwapVig segue como
 // registro histórico e NÃO é sobrescrita.
 double liveVwap = 0.0;
 if(!GetLiveVwapSnapshot((ulong)tk.time_msc, liveVwap))
 {
   Print("[REMORA2.0][LATE_VWAP_INVALID] Não foi possível reconstruir a VWAP viva no
momento do rompimento. Sinal CANCELADO.");
   WriteAudit("LATE_VWAP_INVALID", DirStr(armed.dir), armed.mode, last,
         armed.posRel, armed.distD1, curEff, trigger, "NA",
         armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
   ResetArmedSignal();
   return APS_ENDED_FAIL_CLOSED;
 }

 double lateUpper = liveVwap + SigmaD1 * BandMultiplier;
 double lateLower = liveVwap - SigmaD1 * BandMultiplier;
 double lateRange = lateUpper - lateLower;
 if(lateRange <= 0.0)
 {
    WriteAudit("LATE_VWAP_INVALID", DirStr(armed.dir), armed.mode, last,
         armed.posRel, armed.distD1, curEff, trigger, "NA",
         liveVwap, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
    ResetArmedSignal();
    return APS_ENDED_FAIL_CLOSED;
 }

 double currentPosRel = (last - lateLower) / lateRange;
 bool dynamicGlass =
  (currentPosRel > Inp_VWAP_Outer_Pct) ||
  (currentPosRel < (1.0 - Inp_VWAP_Outer_Pct));

 // [v2.41] GLASS DIRECIONAL POR ROTA
 // ABS SELL nasceu de extensão SUPERIOR: bloqueia apenas se o preço já
 // atravessou até o Glass INFERIOR (venda tardia/kamikaze).
 // ABS BUY nasceu de extensão INFERIOR: bloqueia apenas se o preço já
 // atravessou até o Glass SUPERIOR.
 // CONT (ou qualquer rota não-ABS, fail-closed): qualquer Glass bloqueia.
 bool isAbsRoute = (armed.route == "ABS");
 bool oppositeGlassABS =
   (armed.dir == -1 && currentPosRel < (1.0 - Inp_VWAP_Outer_Pct)) ||
```

```
   (armed.dir == 1 && currentPosRel > Inp_VWAP_Outer_Pct);
 bool lateGlassBlock = isAbsRoute ? oppositeGlassABS : dynamicGlass;

 if(lateGlassBlock)
 {
   PrintFormat("[REMORA2.0][LATE_GLASS_BLOCK] Rota=%s | Teto de Vidro rompido no
Estágio 2 | Last=%.2f | LiveVWAP=%.2f | LateUpper=%.2f | LateLower=%.2f |
PosRelAtual=%.4f | Limites=%.2f/%.2f | Sinal CANCELADO.",
          armed.route, last, liveVwap, lateUpper, lateLower, currentPosRel,
          Inp_VWAP_Outer_Pct, 1.0 - Inp_VWAP_Outer_Pct);
   WriteAudit("LATE_GLASS_BLOCK", DirStr(armed.dir), armed.mode, last,
         currentPosRel, armed.distD1, curEff, trigger, "NA",
         liveVwap, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
   ResetArmedSignal();
   return APS_ENDED_FAIL_CLOSED;
 }

  // ABS + Glass do MESMO lado do extremo original: NÃO bloqueia; registra 1x por sinal.
  if(isAbsRoute && dynamicGlass && !armed.sameSideGlassLogged)
  {
    armed.sameSideGlassLogged = true;
    PrintFormat("[REMORA2.0][ABS_LATE_GLASS_SAMESIDE_ALLOW] %s ABS: Glass
do MESMO lado do extremo original | Last=%.2f | LiveVWAP=%.2f | PosRelAtual=%.4f |
Limites=%.2f/%.2f | Validação CONTINUA (onda viva e esforço vivo já confirmados).",
          DirStr(armed.dir), last, liveVwap, currentPosRel,
          Inp_VWAP_Outer_Pct, 1.0 - Inp_VWAP_Outer_Pct);
   WriteAudit("ABS_LATE_GLASS_SAMESIDE_ALLOW", DirStr(armed.dir), armed.mode,
last,
         currentPosRel, armed.distD1, curEff, trigger, "NA",
         liveVwap, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
  }

  //--- 7) Revalidação de segurança no momento do disparo ---------
  if(HasOpenPosition()) {
Diag2435Armed(DIAG2435_ARMED_POSITION_CANCEL,lastBarTime,TimeTradeServer(),
hhmm,trigger,last,curEff,waveDir,true); ResetArmedSignal(); return
APS_TRADE_OR_POSITION; }
  if(ordersEnteredToday >= Inp_MaxTradesDay) {
Diag2435Armed(DIAG2435_ARMED_QUOTA_CANCEL,lastBarTime,TimeTradeServer(),hh
mm,trigger,last,curEff,waveDir,true); ResetArmedSignal(); return
APS_TRADE_OR_POSITION; }
  if(!SpreadOk())
  {
    WriteAudit("HOLD_SPREAD", DirStr(armed.dir), armed.mode, last,
          armed.posRel, armed.distD1, armed.effort, trigger, "NA",
          armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
```

```
Diag2435Armed(DIAG2435_ARMED_HOLD_SPREAD,lastBarTime,TimeTradeServer(),hhm
m,trigger,last,curEff,waveDir);
   return APS_STILL_ARMED;
  }

 //--- 8) DISPARA A ORDEM (usa armed.activeSL) -------------------
 PrintFormat("[REMORA2.0][DISPARO_%s] %s rompeu a REGIÃO nível=%.2f (last=%.2f) |
ONDA VIVA EFF=%.2f (>=%.2f) Fluxo Confirmado. Executando %s | SL=%.1f pts.",
         armed.route, DirStr(armed.dir), trigger, last, curEff, armed.breakoutEffortMin,
         armed.mode, armed.activeSL);

   bool orderAccepted =
    SendOrder(armed.dir, armed.lots, armed.mode, armed.activeSL, armed.activeTP,
          armed.posRel, armed.distD1, curEff, trigger, "NA",
          armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5);

   // [2.435/D] Somente apos SendOrder: nenhum custo novo antes do envio valido.
   Diag2435SendBegin(armed.dir,armed.lots,trigger,last,tk.time_msc);
   Diag2435SendFinish(lastBarTime,TimeTradeServer(),hhmm);
   ResetArmedSignal();
   return orderAccepted ? APS_TRADE_OR_POSITION : APS_ENDED_FAIL_CLOSED;
}

//+------------------------------------------------------------------+
//| CIRURGIA 13.2 — RESET DIÁRIO (exclusivo do OnTimer)              |
//| [E1/E2] Zera explicitamente estrutura D-1, data esperada e Sigma.|
//| [E3/E5] As chaves persistentes são POR DATA: na virada usa-se a |
//| chave nova (normalmente ausente => dia limpo); as antigas são |
//| ignoradas. NUNCA apagar a chave do dia atual durante a sessão. |
//+------------------------------------------------------------------+
void DailyReset()
{
  ordersEnteredToday = 0;
  gQuotaState = COTA_DESCONHECIDA;
  gQuotaHistoryState = HISTORICO_INDISPONIVEL;
  gQuotaNextRetry = 0;
  gQuotaPersistPending = false;
  gQuotaUnknownLogged = false;

   UpperBand1=0.0; LowerBand1=0.0;
   beCount=0;
   beSyncPending=false;
   lastEntryTime=0;
   ResetArmedSignal();       // inclui armed.audit ([E6]: nunca persistido)
   ResetC1AuditSnapshot(gC1Audit); // snapshot global
   ResetVwapCache();          // cache VWAP
   wwValid=false;         // cache Weis
```

```
   // [E1/E2/E14] Zera explicitamente estrutura D-1, data esperada e Sigma
   VWAP_D1=0.0; H1=0.0; L1=0.0; VWAP_D1_Official=false; gD1ExpectedDate=0;
   SigmaD1=0.0;

   d1Ready=false; nextD1Retry=0;  // flags D-1
   eodDone=false; eodStuckLogged=false; eodNextTry=0; // flags/backoff EOD
   foreignAlertActive=false;            // alerta estrangeira

   RestoreDailyPersistence(); // [2.435/F1] cota do NOVO dia; D1A legado ignorado
   TryCalcD1();
   ReloadOpenPositionsToBE();
   lastBarTime = iTime(Symbol(), Inp_Timeframe, 0); // sincroniza com a barra atual
}

//+------------------------------------------------------------------+
//| CIRURGIA 13.5 — EOD NO OnTimer                                   |
//| Uma tentativa por ciclo | backoff 10s | DONE/PLACED/falha        |
//| eodDone somente sem posição PRÓPRIA | estrangeira intocada       |
//| Após 18:30: EOD_STUCK uma única vez se posição própria persistir |
//+------------------------------------------------------------------+
// [2.43.3/EOD] Identity and evidence are local to EOD. No entry gate is added.
// Keep this journal AND its lock marker across restarts. request_id is volatile.
bool gEodLoaded=false, gEodJournalOK=true, gEodPending=false;
bool gEodUnsent=false, gEodPendingWrite=false, gEodSafetyLogged=false;
bool gEodLegacy=false, gEodOrderAttributable=false;
bool gEodRejectionProven=false, gEodRejectionDurable=false;
ulong gEodTicket=0, gEodPositionId=0, gEodOrder=0, gEodDeal=0;
ulong gEodAttemptSerial=0, gEodBoundAttemptSerial=0;
ulong gEodDeviation=0;
uint gEodRetcode=0, gEodRequestId=0;
uint gEodSeenRequestIds[];
datetime gEodStarted=0;
string gEodNamespace="", gEodIdentity="", gEodProof="NONE", gEodTerminal="NONE";
string gEodLoadedScope="";
double gEodVolume=0.0, gEodPrice=0.0;
long gEodCloseType=0, gEodExecMode=0;
int gEodJournalLock=INVALID_HANDLE;

// Administrative retry is local to this live instance, never journal evidence.
bool gEodIoRetryable=false, gEodIoExpectedPending=false;
int gEodIoError=0, gEodIoRetries=0;
datetime gEodIoNextRetry=0;
string gEodIoExpected="", gEodIoBaseHash="", gEodReadHash="";
ulong gEodIoBaseBytes=0, gEodIoBaseRows=0, gEodReadBytes=0, gEodReadRows=0;

// The existing reader parses into globals. Keep the live attempt intact on IO
// failure, and while comparing an older durable prefix with that live attempt.
struct EodIoMemory
```

```
{
   string ns,id,proof,terminal;
   ulong serial,ticket,pid,order,deal,deviation;
   datetime started;
   uint retcode;
   double volume,price;
   long type,exec;
   bool pending,legacy,attributable,rejected;
};

void EodSaveIoMemory(EodIoMemory &s)
{
  s.ns=gEodNamespace; s.id=gEodIdentity; s.proof=gEodProof; s.terminal=gEodTerminal;
  s.serial=gEodAttemptSerial; s.ticket=gEodTicket; s.pid=gEodPositionId;
  s.order=gEodOrder; s.deal=gEodDeal; s.deviation=gEodDeviation;
  s.started=gEodStarted; s.retcode=gEodRetcode; s.volume=gEodVolume;
s.price=gEodPrice;
  s.type=gEodCloseType; s.exec=gEodExecMode;
  s.pending=gEodPending; s.legacy=gEodLegacy;
  s.attributable=gEodOrderAttributable; s.rejected=gEodRejectionDurable;
}

void EodRestoreIoMemory(const EodIoMemory &s)
{
  gEodNamespace=s.ns; gEodIdentity=s.id; gEodProof=s.proof; gEodTerminal=s.terminal;
  gEodAttemptSerial=s.serial; gEodTicket=s.ticket; gEodPositionId=s.pid;
  gEodOrder=s.order; gEodDeal=s.deal; gEodDeviation=s.deviation;
  gEodStarted=s.started; gEodRetcode=s.retcode; gEodVolume=s.volume;
gEodPrice=s.price;
  gEodCloseType=s.type; gEodExecMode=s.exec;
  gEodPending=s.pending; gEodLegacy=s.legacy;
  gEodOrderAttributable=s.attributable; gEodRejectionDurable=s.rejected;
}

string EodJournalName()
{
  string server=AccountInfoString(ACCOUNT_SERVER);
  StringReplace(server,"\\","_"); StringReplace(server,"/","_");
  StringReplace(server,":","_"); StringReplace(server,"*","_");
  StringReplace(server,"?","_"); StringReplace(server,"\"","_");
  StringReplace(server,"<","_"); StringReplace(server,">","_");
  StringReplace(server,"|","_");
  return StringFormat("R2431_EOD_%s_%I64d_%s_%d.csv",server,
     (long)AccountInfoInteger(ACCOUNT_LOGIN),Symbol(),MagicCore);
}

bool EodParseUlong(string token,ulong &value)
{
```

```
  int n=StringLen(token); if(n==0) return false;
  value=0;
  for(int i=0;i<n;i++)
  {
    ushort c=StringGetCharacter(token,i);
    if(c<48 || c>57) return false;
    ulong d=(ulong)(c-48);
    if(value>((ulong)-1-d)/10) return false;
    value=value*10+d;
  }
  return true;
}

string EodHexText(string text)
{
  string result="";
  for(int i=0;i<StringLen(text);i++)
     result+=StringFormat("%04X",(uint)StringGetCharacter(text,i));
  return result;
}

string EodScope()
{
  // Hex keeps delimiters, Unicode, and filename sanitization unambiguous.
  return StringFormat("%I64d|%s|%s|%d",(long)AccountInfoInteger(ACCOUNT_LOGIN),

EodHexText(AccountInfoString(ACCOUNT_SERVER)),EodHexText(Symbol()),MagicCore);
}

bool EodDigest(string text,uchar &digest[])
{
  uchar data[],key[];
  int n=StringToCharArray(text,data,0,WHOLE_ARRAY,CP_UTF8);
  if(n<=0 || ArrayResize(data,n-1)!=n-1) return false;
  return CryptEncode(CRYPT_HASH_SHA256,data,key,digest)==32;
}

string EodHash(string text)
{
  uchar digest[];
  if(!EodDigest(text,digest)) return "";
  string result="";
  for(int i=0;i<32;i++) result+=StringFormat("%02X",(uint)digest[i]);
  return result;
}

string EodAttemptIdentity(ulong serial)
{
```

```
  // 128 digest bits, base32, plus prefix = 31 ASCII characters. The full
  // scope, namespace, monotonic cycle and position remain in the journal.
  uchar digest[];
  string data=EodScope()+"|"+gEodNamespace+"|"+StringFormat("%I64u",serial)+"|"+
    StringFormat("%I64u|%I64u",gEodTicket,gEodPositionId);
  if(!EodDigest(data,digest)) return "";
  string alphabet="ABCDEFGHIJKLMNOPQRSTUVWXYZ234567",result="R2433";
  uint buffer=0; int bits=0;
  for(int i=0;i<16;i++)
  {
    buffer=(buffer<<8)|(uint)digest[i]; bits+=8;
    while(bits>=5)
    {
      bits-=5;
      result+=StringSubstr(alphabet,(int)((buffer>>bits)&31),1);
    }
  }
  if(bits>0) result+=StringSubstr(alphabet,(int)((buffer<<(5-bits))&31),1);
  return result;
}

void EodSafetyAlert(string reason)
{
  eodDone=false;
  MqlDateTime dt; TimeToStruct(TimeTradeServer(),dt);
  if(dt.hour*100+dt.min>=1830 && !eodStuckLogged)
  {
    eodStuckLogged=true;
    PrintFormat("[REMORA2.0][EOD_STUCK] CRITICO: zeragem PROPRIA em %s nao
pode ser comprovada apos 18:30. Intervencao manual necessaria. %s",Symbol(),reason);
    WriteAudit("EOD_STUCK","NONE","NONE",0,0,0,0,0,"NA",0,0,0,0,0,0,0,0);
  }
  if(gEodSafetyLogged) return;
  gEodSafetyLogged=true;
  PrintFormat("[REMORA2.43.3][EOD_UNCONFIRMED] %s. Nenhum fechamento incerto
sera repetido; intervencao manual se a evidencia nao puder ser recuperada.",reason);
}

bool EodKnownRejection(uint ret)
{
   return ret==TRADE_RETCODE_REQUOTE || ret==TRADE_RETCODE_REJECT ||
    ret==TRADE_RETCODE_CANCEL || ret==TRADE_RETCODE_INVALID ||
    ret==TRADE_RETCODE_INVALID_VOLUME ||
ret==TRADE_RETCODE_INVALID_PRICE ||
    ret==TRADE_RETCODE_INVALID_STOPS ||
ret==TRADE_RETCODE_TRADE_DISABLED ||
    ret==TRADE_RETCODE_MARKET_CLOSED || ret==TRADE_RETCODE_NO_MONEY
||
```

```
    ret==TRADE_RETCODE_PRICE_CHANGED || ret==TRADE_RETCODE_PRICE_OFF
||
    ret==TRADE_RETCODE_TOO_MANY_REQUESTS ||
ret==TRADE_RETCODE_SERVER_DISABLES_AT ||
    ret==TRADE_RETCODE_CLIENT_DISABLES_AT ||
ret==TRADE_RETCODE_INVALID_EXPIRATION ||
    ret==TRADE_RETCODE_FROZEN || ret==TRADE_RETCODE_ONLY_REAL ||
    ret==TRADE_RETCODE_LIMIT_ORDERS || ret==TRADE_RETCODE_LIMIT_VOLUME
||
    ret==TRADE_RETCODE_LIMIT_POSITIONS || ret==TRADE_RETCODE_LONG_ONLY
||
    ret==TRADE_RETCODE_SHORT_ONLY || ret==TRADE_RETCODE_CLOSE_ONLY ||
    ret==TRADE_RETCODE_FIFO_CLOSE ||
ret==TRADE_RETCODE_HEDGE_PROHIBITED ||
    ret==TRADE_RETCODE_INVALID_FILL || ret==TRADE_RETCODE_INVALID_ORDER
||
    ret==TRADE_RETCODE_POSITION_CLOSED ||
ret==TRADE_RETCODE_INVALID_CLOSE_VOLUME;
}

string EodRecord(string state)
{
  string payload="R2433|"+state+"|"+EodScope()+"|"+gEodNamespace+"|"+
    StringFormat("%I64u",gEodAttemptSerial)+"|"+gEodIdentity+"|"+
    StringFormat("%I64u|%I64u|%I64d",gEodTicket,gEodPositionId,(long)gEodStarted)+"|"+
    IntegerToString(gEodCloseType)+"|"+DoubleToString(gEodVolume,16)+"|"+
    DoubleToString(gEodPrice,16)+"|"+IntegerToString(gEodExecMode)+"|"+
      StringFormat("%I64u",gEodDeviation)+"|"+
      StringFormat("%I64u|%I64u|%u",gEodOrder,gEodDeal,gEodRetcode)+"|"+
      gEodProof+"|"+(gEodRejectionDurable?"1":"0")+"|"+gEodTerminal;
   string hash=EodHash(payload);
   if(hash=="") return "";
   return payload+"|"+hash+"|END2433";
}

bool EodParseRecord(string line,bool &haveRecord,string &last)
{
  string f[];
  if(StringSubstr(line,0,6)!="R2433|")
  {
    // Legacy proof is not promoted to a new per-attempt identity.
    if(StringSplit(line,'|',f)!=8 ||
       (f[0]!="PENDING" && f[0]!="RESOLVED") ||
       (f[7]!="END2431" && f[7]!="END2431R") || gEodNamespace!="") return false;
    ulong v[6];
    for(int i=0;i<6;i++) if(!EodParseUlong(f[i+1],v[i])) return false;
    if(v[1]==0 || v[5]>4294967295) return false;
    gEodTicket=v[0]; gEodPositionId=v[1]; gEodOrder=v[2]; gEodDeal=v[3];
```

```
    gEodStarted=(datetime)v[4]; gEodRetcode=(uint)v[5];
    gEodPending=(f[0]=="PENDING"); gEodLegacy=gEodPending;
    gEodOrderAttributable=false;
    haveRecord=true; last=line;
    return true;
  }
  if(StringSplit(line,'|',f)!=25 || f[24]!="END2433") return false;
  string payload=f[0];
  for(int i=1;i<23;i++) payload+="|"+f[i];
  if(StringLen(f[23])!=64 || EodHash(payload)!=f[23]) return false;
  if(f[2]+"|"+f[3]+"|"+f[4]+"|"+f[5]!=EodScope()) return false;
  if(StringLen(f[6])!=64 || (gEodNamespace!="" && f[6]!=gEodNamespace)) return false;
  ulong serial,ticket,pid,started,type,exec,deviation,order,deal,ret;
  if(!EodParseUlong(f[7],serial) || !EodParseUlong(f[9],ticket) ||
     !EodParseUlong(f[10],pid) || !EodParseUlong(f[11],started) ||
     !EodParseUlong(f[12],type) || !EodParseUlong(f[15],exec) ||
     !EodParseUlong(f[16],deviation) || !EodParseUlong(f[17],order) ||
     !EodParseUlong(f[18],deal) || !EodParseUlong(f[19],ret) ||
     ret>4294967295) return false;
  double volume=StringToDouble(f[13]),price=StringToDouble(f[14]);
  if(!MathIsValidNumber(volume) || !MathIsValidNumber(price) ||
     DoubleToString(volume,16)!=f[13] || DoubleToString(price,16)!=f[14]) return false;
  if(gEodLegacy) return false;
  if(f[1]=="INIT")
  {
     if(gEodNamespace!="" || serial!=0 || ticket!=0 || pid!=0 ||
        f[8]!="" || order!=0 || deal!=0 || f[20]!="NONE" ||
     f[21]!="0" || f[22]!="INIT") return false;
  }
  else
  {
     if(f[1]!="PENDING" && f[1]!="RESOLVED") return false;
     if(serial==0 || ticket==0 || pid==0 || started==0 || volume<=0 || price<=0 ||
        (type!=ORDER_TYPE_BUY && type!=ORDER_TYPE_SELL) ||
        (exec!=SYMBOL_TRADE_EXECUTION_REQUEST &&
exec!=SYMBOL_TRADE_EXECUTION_INSTANT &&
         exec!=SYMBOL_TRADE_EXECUTION_MARKET &&
exec!=SYMBOL_TRADE_EXECUTION_EXCHANGE)) return false;
     if(serial==gEodAttemptSerial)
     {
        if(!gEodPending || ticket!=gEodTicket || pid!=gEodPositionId ||
           started!=(ulong)gEodStarted || type!=(ulong)gEodCloseType ||
           volume!=gEodVolume || price!=gEodPrice || exec!=(ulong)gEodExecMode ||
deviation!=gEodDeviation ||
           f[8]!=gEodIdentity || (gEodOrder>0 && order!=gEodOrder)) return false;
     }
     else if(gEodPending || gEodAttemptSerial==(ulong)-1 ||
             serial!=gEodAttemptSerial+1 || f[1]!="PENDING" || order!=0 || deal!=0 ||
```

```
          ret!=0 || f[20]!="NONE" || f[21]!="0" || f[22]!="NONE") return false;
    if(f[20]!="NONE" && f[20]!="DIRECT" && f[20]!="ORDER_COMMENT" &&
       f[20]!="DEAL_COMMENT") return false;
    if((f[20]!="NONE")!=(order>0) || (f[21]!="0" && f[21]!="1")) return false;
    if(f[21]=="1" && (order!=0 || deal!=0 || !EodKnownRejection((uint)ret))) return false;
    if(f[1]=="PENDING" && f[22]!="NONE") return false;
    if(f[1]=="RESOLVED" && f[22]!="UNSENT" && f[22]!="CANCELED" &&
       f[22]!="REJECTED" && f[22]!="EXPIRED" && f[22]!="FILLED") return false;
  }
  gEodNamespace=f[6]; gEodAttemptSerial=serial; gEodIdentity=f[8];
  gEodTicket=ticket; gEodPositionId=pid; gEodStarted=(datetime)started;
  gEodCloseType=(long)type; gEodVolume=volume; gEodPrice=price;
gEodExecMode=(long)exec;
  gEodDeviation=deviation;
  if(serial>0 && EodAttemptIdentity(serial)!=gEodIdentity) return false;
  gEodOrder=order; gEodDeal=deal; gEodRetcode=(uint)ret;
  gEodProof=f[20]; gEodRejectionDurable=(f[21]=="1"); gEodTerminal=f[22];
  gEodPending=(f[1]=="PENDING");
  gEodOrderAttributable=(order>0 && gEodProof!="NONE");
  haveRecord=true; last=line;
  return true;
}

bool EodReadJournal(string &last)
{
  gEodIoRetryable=false; gEodIoError=0;
  ResetLastError();
    int h=FileOpen(EodJournalName(),FILE_READ|FILE_TXT|FILE_ANSI);
    if(h==INVALID_HANDLE)
    { gEodIoRetryable=true; gEodIoError=GetLastError(); return false; }
    EodIoMemory live; EodSaveIoMemory(live);
    gEodNamespace=""; gEodIdentity=""; gEodAttemptSerial=0; gEodPending=false;
    gEodLegacy=false; gEodOrderAttributable=false;
    bool ok=true,haveRecord=false;
    string used[]; ulong previous=0;
    string chain=""; ulong rows=0;
    ResetLastError();
    while(!FileIsEnding(h))
    {
      string line=FileReadString(h);
      if(GetLastError()!=0)
      { gEodIoRetryable=true; gEodIoError=GetLastError(); ok=false; break; }
      if(line=="" || !EodParseRecord(line,haveRecord,last)){ ok=false; break; }
      chain=EodHash(chain+"|"+line);
      if(chain==""){ ok=false; break; }
      rows++;
      if(gEodAttemptSerial>previous)
      {
```

```
         int n=ArraySize(used);
         for(int i=0;i<n;i++) if(used[i]==gEodIdentity) ok=false;
         if(!ok || ArrayResize(used,n+1)!=n+1){ ok=false; break; }
         used[n]=gEodIdentity; previous=gEodAttemptSerial;
       }
     }
     if(ok && GetLastError()!=0)
     { gEodIoRetryable=true; gEodIoError=GetLastError(); ok=false; }
     ulong bytes=FileTell(h);
     if(GetLastError()!=0 && ok)
     { gEodIoRetryable=true; gEodIoError=GetLastError(); ok=false; }
     FileClose(h);
     if(!ok || !haveRecord){ EodRestoreIoMemory(live); return false; }
     gEodReadHash=chain; gEodReadRows=rows; gEodReadBytes=bytes;
     return true;
}

bool EodWriteLine(string line)
{
  gEodIoRetryable=false; gEodIoError=0;
  if(line=="" || !gEodJournalOK || gEodJournalLock==INVALID_HANDLE ||
     gEodLoadedScope!=EodScope()) return false;
  ResetLastError();
  int h=FileOpen(EodJournalName(),FILE_READ|FILE_WRITE|FILE_TXT|FILE_ANSI);
  if(h==INVALID_HANDLE)
  { gEodIoRetryable=true; gEodIoError=GetLastError(); return false; }
  bool seek=FileSeek(h,0,SEEK_END);
    uint bytes=0;
    if(seek) bytes=FileWriteString(h,line+"\r\n");
    FileFlush(h);
    int err=GetLastError();
    FileClose(h);
    bool ok=seek && bytes==(uint)(StringLen(line)+2) && err==0;
    // A known partial row is invalid, never a retryable administrative failure.
    if(!ok && (bytes==0 || bytes==(uint)(StringLen(line)+2)))
    { gEodIoRetryable=true; gEodIoError=err; }
    return ok;
}

bool EodInitializeNamespace()
{

gEodNamespace=EodHash(EodScope()+"|"+TerminalInfoString(TERMINAL_DATA_PATH)+"
|"+
     StringFormat("%I64d|%I64u|%I64u|%I64d",(long)TimeLocal(),GetTickCount64(),
       GetMicrosecondCount(),ChartID()));
  if(gEodNamespace=="") return false;
  gEodAttemptSerial=0; gEodIdentity=""; gEodTicket=0; gEodPositionId=0;
```

```
  gEodOrder=0; gEodDeal=0; gEodStarted=0; gEodRetcode=0;
  gEodCloseType=0; gEodVolume=0; gEodPrice=0; gEodExecMode=0; gEodDeviation=0;
  gEodProof="NONE"; gEodRejectionDurable=false; gEodTerminal="INIT";
  return EodWriteLine(EodRecord("INIT"));
}

bool EodLoadIntent()
{
  if(gEodLoaded)
  {
    if(gEodLoadedScope!=EodScope())
    { gEodJournalOK=false; gEodIoRetryable=false; }
    if(!gEodJournalOK && gEodUnsent && gEodIoRetryable) return EodRecoverUnsentIo();
    return gEodJournalOK;
  }
  if(gEodJournalLock==INVALID_HANDLE)
  {
    // No FILE_SHARE flags: one EOD writer per terminal/context, including send.
    // MT5 closes this handle on unload. Never delete the retained marker.

gEodJournalLock=FileOpen(EodJournalName()+".lock",FILE_READ|FILE_WRITE|FILE_TX
T|FILE_ANSI);
    if(gEodJournalLock==INVALID_HANDLE) return false;
    gEodLoadedScope=EodScope();
  }
  if(gEodLoadedScope!=EodScope()){ gEodJournalOK=false; return false; }
  bool retained=(FileSize(gEodJournalLock)>0);
   if(retained)
   {
     if(!FileSeek(gEodJournalLock,0,SEEK_SET) ||
        FileReadString(gEodJournalLock)!="R2433_LOCK|"+EodScope())
     { gEodJournalOK=false; return false; }
   }
   bool exists=FileIsExist(EodJournalName());
   if(!exists && retained){ gEodJournalOK=false; return false; }
   if(!exists)
   {
     if(!EodInitializeNamespace()){ gEodJournalOK=false; return false; }
   }
   string last="";
   if(!EodReadJournal(last)){ gEodJournalOK=false; return false; }
   // Only a fully parsed legacy RESOLVED journal can start a new namespace.
   // An unresolved legacy row, including END2431R, stays a manual case.
   if(gEodNamespace=="" && !gEodPending)
   {
     if(!EodInitializeNamespace() || !EodReadJournal(last))
     { gEodJournalOK=false; return false; }
   }
```

```
   if(!retained)
   {
      string marker="R2433_LOCK|"+EodScope()+"\r\n";
      ResetLastError();
      if(!FileSeek(gEodJournalLock,0,SEEK_SET) ||
         FileWriteString(gEodJournalLock,marker)!=(uint)StringLen(marker))
      { gEodJournalOK=false; return false; }
      FileFlush(gEodJournalLock);
      if(GetLastError()!=0){ gEodJournalOK=false; return false; }
   }
   gEodLoaded=true; gEodLoadedScope=EodScope();
   return gEodJournalOK;
}

bool EodAppendIntent(bool pending)
{
 if(!gEodJournalOK) return false;
 string expected=EodRecord(pending?"PENDING":"RESOLVED"),last="";
 EodIoMemory live; EodSaveIoMemory(live);
 string prefix=gEodReadHash;
 ulong rows=gEodReadRows,bytes=gEodReadBytes;
 bool ok=EodWriteLine(expected);
 if(ok) ok=EodReadJournal(last);
 if(ok && last!=expected){ ok=false; gEodIoRetryable=false; }
 if(!ok)
 {
    EodRestoreIoMemory(live);
   gEodIoExpected=expected; gEodIoExpectedPending=pending;
   gEodIoBaseHash=prefix; gEodIoBaseRows=rows; gEodIoBaseBytes=bytes;
   gEodJournalOK=false; gEodIoNextRetry=TimeTradeServer()+10;
   PrintFormat("[REMORA2.434][EOD_STORE_FAIL] class=%s error=%d attempt=%s
rows=%I64u bytes=%I64u. Sem liberacao de envio.",
     gEodIoRetryable?"IO":"INVALID",gEodIoError,gEodIdentity,rows,bytes);
   return false;
 }
 return true; // Only a flushed, re-read, checksum-validated record authorizes progress.
}

bool EodRecoverUnsentIo()
{
 // No send, no broker inference: this proof is volatile and is never restored.
 if(!gEodLoaded || !gEodUnsent || !gEodIoRetryable || gEodIoExpected=="" ||
    !gEodPending || gEodLegacy || gEodLoadedScope!=EodScope() ||
    gEodJournalLock==INVALID_HANDLE) return false;
 datetime now=TimeTradeServer();
 if(now<gEodIoNextRetry || gEodIoRetries>=3) return false;
 gEodIoRetries++; gEodIoNextRetry=now+10;
```

```
  PrintFormat("[REMORA2.434][EOD_IO_RECHECK] retry=%d/3 attempt=%s
unsent=1",gEodIoRetries,gEodIdentity);
  ResetLastError();
  bool seek=FileSeek(gEodJournalLock,0,SEEK_SET);
  string marker=seek?FileReadString(gEodJournalLock):"";
  if(!seek || GetLastError()!=0)
  {
    PrintFormat("[REMORA2.434][EOD_IO_BLOCKED] cause=LOCK_IO error=%d
retry=%d/3 attempt=%s",GetLastError(),gEodIoRetries,gEodIdentity);
    if(gEodIoRetries==3) Print("[REMORA2.434][EOD_IO_LIMIT] Retentativas esgotadas;
intervencao manual necessaria.");
    return false;
  }
  if(marker!="R2433_LOCK|"+EodScope())
  {
    gEodIoRetryable=false;
    PrintFormat("[REMORA2.434][EOD_IO_INVALID] cause=MARKER_OR_CONTEXT
attempt=%s. Intervencao manual necessaria.",gEodIdentity);
    return false;
  }
  EodIoMemory live; EodSaveIoMemory(live);
  string last="";
  bool read=EodReadJournal(last);
  EodRestoreIoMemory(live);
  if(!read)
  {
    PrintFormat("[REMORA2.434][EOD_IO_BLOCKED] class=%s error=%d retry=%d/3
attempt=%s",
       gEodIoRetryable?"IO":"INVALID",gEodIoError,gEodIoRetries,gEodIdentity);
    if(gEodIoRetries==3) Print("[REMORA2.434][EOD_IO_LIMIT] Retentativas esgotadas;
intervencao manual necessaria.");
    return false;
  }
  bool before=(gEodReadRows==gEodIoBaseRows &&
gEodReadBytes==gEodIoBaseBytes &&
    gEodReadHash==gEodIoBaseHash);
  bool after=(last==gEodIoExpected && gEodReadRows==gEodIoBaseRows+1 &&
    gEodReadBytes==gEodIoBaseBytes+(ulong)StringLen(gEodIoExpected)+2 &&
    gEodReadHash==EodHash(gEodIoBaseHash+"|"+gEodIoExpected));
  if((!before && !after) ||
EodRecord(gEodIoExpectedPending?"PENDING":"RESOLVED")!=gEodIoExpected)
  {
    gEodIoRetryable=false;
    PrintFormat("[REMORA2.434][EOD_IO_INVALID] cause=PREFIX_OR_MEMORY
attempt=%s serial=%I64u rows=%I64u bytes=%I64u. Intervencao manual necessaria.",
       gEodIdentity,gEodAttemptSerial,gEodReadRows,gEodReadBytes);
    return false;
  }
```

```
  // Validated prefix or exact complete write; never append over unknown bytes.
  gEodJournalOK=true;
  if(before && !EodAppendIntent(gEodIoExpectedPending))
  {
     if(gEodIoRetries==3) Print("[REMORA2.434][EOD_IO_LIMIT] Retentativas esgotadas;
intervencao manual necessaria.");
     return false;
  }
  if(!gEodIoExpectedPending)
  {
     // The failed append was already the proven UNSENT resolution.
     gEodPending=false; gEodUnsent=false; gEodPendingWrite=false;
     gEodBoundAttemptSerial=0; gEodRequestId=0; gEodRejectionProven=false;
  }
  gEodIoRetryable=false; gEodIoRetries=0; gEodIoNextRetry=0; gEodSafetyLogged=false;
  PrintFormat("[REMORA2.434][EOD_IO_RECOVERED] attempt=%s serial=%I64u
journal_valid=1 unsent=%d",
     gEodIdentity,gEodAttemptSerial,(int)gEodUnsent);
  return true;
}

bool EodOwnSnapshot(bool &hasPosition,bool &hasOrder)
{
  hasPosition=false; hasOrder=false;
  if(MQLInfoInteger(MQL_TESTER)==0 &&
     !TerminalInfoInteger(TERMINAL_CONNECTED)) return false;
  int np=PositionsTotal(),no=OrdersTotal();
  for(int i=np-1;i>=0;i--)
    {
       if(!posInfo.SelectByIndex(i)) return false;
       string symbol; long magic;
       if(!PositionGetString(POSITION_SYMBOL,symbol) ||
          !PositionGetInteger(POSITION_MAGIC,magic)) return false;
       if(symbol==Symbol() && magic==(long)MagicCore) hasPosition=true;
    }
    for(int i=no-1;i>=0;i--)
    {
      if(OrderGetTicket(i)==0) return false;
      string symbol; long magic;
      if(!OrderGetString(ORDER_SYMBOL,symbol) ||
         !OrderGetInteger(ORDER_MAGIC,magic)) return false;
      if(symbol==Symbol() && magic==(long)MagicCore) hasOrder=true;
    }
    return np==PositionsTotal() && no==OrdersTotal();
}

bool EodOrderInteger(ulong ticket,bool history,ENUM_ORDER_PROPERTY_INTEGER
prop,long &value)
```

```
{
  if(history) return HistoryOrderGetInteger(ticket,prop,value);
  return OrderGetInteger(prop,value);
}

bool EodOrderString(ulong ticket,bool history,ENUM_ORDER_PROPERTY_STRING
prop,string &value)
{
  if(history) return HistoryOrderGetString(ticket,prop,value);
  return OrderGetString(prop,value);
}

bool EodOrderShape(ulong ticket,bool history,bool alreadyBound)
{
  if(!history && !OrderSelect(ticket)) return false;
  string symbol; long magic,pid,type,filling,timeType;
  double volume;
  if(!EodOrderString(ticket,history,ORDER_SYMBOL,symbol) ||
     !EodOrderInteger(ticket,history,ORDER_MAGIC,magic) ||
     !EodOrderInteger(ticket,history,ORDER_POSITION_ID,pid) ||
     !EodOrderInteger(ticket,history,ORDER_TYPE,type) ||
     !EodOrderInteger(ticket,history,ORDER_TYPE_FILLING,filling) ||
     !EodOrderInteger(ticket,history,ORDER_TYPE_TIME,timeType)) return false;
  if(history)
  { if(!HistoryOrderGetDouble(ticket,ORDER_VOLUME_INITIAL,volume)) return false; }
  else if(!OrderGetDouble(ORDER_VOLUME_INITIAL,volume)) return false;
  return symbol==Symbol() && magic==(long)MagicCore && type==gEodCloseType &&
     filling==ORDER_FILLING_FOK && timeType==ORDER_TIME_GTC &&
    MathIsValidNumber(volume) && volume==gEodVolume &&
    ((ulong)pid==gEodPositionId || (pid==0 && alreadyBound));
}

bool EodDealShape(ulong deal,ulong &order,double &volume)
{
  string symbol; long magic,entry,pid,type,orderValue;
  if(!HistoryDealGetString(deal,DEAL_SYMBOL,symbol) ||
     !HistoryDealGetInteger(deal,DEAL_MAGIC,magic) ||
     !HistoryDealGetInteger(deal,DEAL_ENTRY,entry) ||
     !HistoryDealGetInteger(deal,DEAL_POSITION_ID,pid) ||
     !HistoryDealGetInteger(deal,DEAL_TYPE,type) ||
     !HistoryDealGetInteger(deal,DEAL_ORDER,orderValue) ||
     !HistoryDealGetDouble(deal,DEAL_VOLUME,volume)) return false;
  order=(ulong)orderValue;
  return symbol==Symbol() && magic==(long)MagicCore && (ulong)pid==gEodPositionId &&
     entry==DEAL_ENTRY_OUT &&
     ((gEodCloseType==ORDER_TYPE_BUY && type==DEAL_TYPE_BUY) ||
      (gEodCloseType==ORDER_TYPE_SELL && type==DEAL_TYPE_SELL)) &&
     order>0 && MathIsValidNumber(volume) && volume>0 && volume<=gEodVolume;
```

```
}

bool EodMergeCandidate(ulong value,ulong &candidate)
{
  if(value==0 || (candidate>0 && candidate!=value)) return false;
  candidate=value;
  return true;
}

bool EodRecoverBrokerLink()
{
  if(gEodLegacy || gEodIdentity=="" || !gEodJournalOK || gEodLoadedScope!=EodScope())
return false;
  if(!HistorySelect(0,TimeTradeServer())) return false;
  ulong candidate=gEodOrderAttributable?gEodOrder:0;
  string proof=gEodProof;
  int active=OrdersTotal(),history=HistoryOrdersTotal(),deals=HistoryDealsTotal();
  for(int pass=0;pass<2;pass++)
  {
    bool historical=(pass==1);
    int n=historical?history:active;
    for(int i=0;i<n;i++)
    {
      ulong tk=historical?HistoryOrderGetTicket(i):OrderGetTicket(i);
      if(tk==0) return false;
      string symbol,comment; long magic;
      if(!EodOrderString(tk,historical,ORDER_SYMBOL,symbol) ||
         !EodOrderInteger(tk,historical,ORDER_MAGIC,magic) ||
           !EodOrderString(tk,historical,ORDER_COMMENT,comment)) return false;
        if(symbol!=Symbol() || magic!=(long)MagicCore) continue;
        bool bound=(gEodOrderAttributable && tk==gEodOrder);
        if(comment!=gEodIdentity && !bound) continue;
        if(!EodOrderShape(tk,historical,bound) || !EodMergeCandidate(tk,candidate)) return
false;
        if(!bound) proof="ORDER_COMMENT";
    }
  }
  for(int i=0;i<deals;i++)
  {
    ulong d=HistoryDealGetTicket(i); if(d==0) return false;
    string symbol,comment; long magic;
    if(!HistoryDealGetString(d,DEAL_SYMBOL,symbol) ||
       !HistoryDealGetInteger(d,DEAL_MAGIC,magic) ||
       !HistoryDealGetString(d,DEAL_COMMENT,comment)) return false;
    if(symbol!=Symbol() || magic!=(long)MagicCore || comment!=gEodIdentity) continue;
    ulong order; double volume;
    if(!EodDealShape(d,order,volume)) return false;
    bool bound=(gEodOrderAttributable && order==gEodOrder);
```

```
    bool inHistory=EodOrderShape(order,true,bound);
    if(!inHistory && !EodOrderShape(order,false,bound)) return false;
    if(!EodMergeCandidate(order,candidate)) return false;
    if(!bound) proof="DEAL_COMMENT";
  }
  if(active!=OrdersTotal() || history!=HistoryOrdersTotal() || deals!=HistoryDealsTotal()) return
false;
  if(candidate>0 && !gEodOrderAttributable)
  {
    gEodOrder=candidate; gEodProof=proof; gEodOrderAttributable=true;
    if(!EodAppendIntent(true)) return false;
    PrintFormat("[REMORA2.43.3][EOD_LINK] attempt=%s order=%I64u
proof=%s",gEodIdentity,gEodOrder,gEodProof);
  }
  return true;
}

bool EodHasCurrentRequest()
{
  return gEodAttemptSerial>0 && gEodBoundAttemptSerial==gEodAttemptSerial &&
gEodRequestId>0;
}

void EodBindRequestResult(const MqlTradeResult &result)
{
  // Called only with the immediate result of our exact synchronous OrderSend.
  gEodBoundAttemptSerial=0; gEodRequestId=0;
  if(result.request_id>0)
 {
    int n=ArraySize(gEodSeenRequestIds); bool seen=false;
    for(int i=0;i<n;i++) if(gEodSeenRequestIds[i]==result.request_id) seen=true;
    if(!seen && ArrayResize(gEodSeenRequestIds,n+1)==n+1)
    {
      gEodSeenRequestIds[n]=result.request_id;
      gEodRequestId=result.request_id; gEodBoundAttemptSerial=gEodAttemptSerial;
    }
  }
  gEodRetcode=result.retcode; gEodDeal=result.deal;
  if(result.order>0)
  {
    gEodOrder=result.order; gEodOrderAttributable=true; gEodProof="DIRECT";
  }
  else if(result.deal>0 && HistoryDealSelect(result.deal))
  {
    ulong order; double volume;
    if(EodDealShape(result.deal,order,volume))
    { gEodOrder=order; gEodOrderAttributable=true; gEodProof="DIRECT"; }
  }
```

```
  gEodRejectionProven=(result.order==0 && result.deal==0 &&
EodKnownRejection(result.retcode));
  gEodRejectionDurable=gEodRejectionProven;
}

bool EodReconcileIntent()
{
  if(!gEodPending) return true;
  if(!gEodJournalOK || gEodLegacy || gEodLoadedScope!=EodScope()) return false;
  if(gEodPendingWrite)
  {
     if(!EodAppendIntent(true)) return false;
     gEodPendingWrite=false;
  }
  if(!EodRecoverBrokerLink()) return false;
  bool hasPosition=false,hasOrder=false;
  if(!EodOwnSnapshot(hasPosition,hasOrder) || hasOrder) return false;
  bool resolved=false;
  if(gEodOrder>0)
  {
     if(!gEodOrderAttributable || !HistoryOrderSelect(gEodOrder) ||
        !EodOrderShape(gEodOrder,true,true)) return false;
     long state;
     if(!HistoryOrderGetInteger(gEodOrder,ORDER_STATE,state)) return false;
     if(state==ORDER_STATE_CANCELED){ resolved=true; gEodTerminal="CANCELED"; }
     else if(state==ORDER_STATE_REJECTED){ resolved=true;
gEodTerminal="REJECTED"; }
     else if(state==ORDER_STATE_EXPIRED){ resolved=true; gEodTerminal="EXPIRED"; }
    else if(state==ORDER_STATE_FILLED)
    {
     if(!HistorySelectByPosition(gEodPositionId)) return false;
     double total=0.0;
     int n=HistoryDealsTotal();
     for(int i=0;i<n;i++)
     {
        ulong d=HistoryDealGetTicket(i); long orderValue;
        if(d==0 || !HistoryDealGetInteger(d,DEAL_ORDER,orderValue)) return false;
        if((ulong)orderValue!=gEodOrder) continue;
        ulong order; double volume;
        if(!EodDealShape(d,order,volume)) return false;
        total+=volume; gEodDeal=d;
     }
     resolved=(n==HistoryDealsTotal() && MathAbs(total-gEodVolume)<=1e-8);
     if(resolved) gEodTerminal="FILLED";
    }
  }
  else if(gEodRejectionDurable && EodKnownRejection(gEodRetcode))
  { resolved=true; gEodTerminal="REJECTED"; }
```

```
  // Flat alone, DONE, PLACED, timeout and an old request_id are never proof.
  if(!resolved) return false;
  if(!EodAppendIntent(false)) return false;
  PrintFormat("[REMORA2.43.3][EOD_TERMINAL] attempt=%s order=%I64u
state=%s",gEodIdentity,gEodOrder,gEodTerminal);
  gEodPending=false; gEodPendingWrite=false; gEodSafetyLogged=false;
  gEodBoundAttemptSerial=0; gEodRequestId=0; gEodRejectionProven=false;
  return true;
}

bool EodRequestMatches(const MqlTradeRequest &request)
{
  return request.action==TRADE_ACTION_DEAL && request.position==gEodTicket &&
    request.position_by==0 && request.symbol==Symbol() &&
request.magic==(ulong)MagicCore &&
    request.comment==gEodIdentity && request.type==gEodCloseType &&
    request.volume==gEodVolume && request.price==gEodPrice && request.sl==0 &&
request.tp==0 &&
    request.type_filling==ORDER_FILLING_FOK &&
request.type_time==ORDER_TIME_GTC &&
    request.deviation==gEodDeviation;
}

void EodObserveTransaction(const MqlTradeTransaction &trans,
               const MqlTradeRequest &request,const MqlTradeResult &result)
{
  if(!gEodLoaded || !gEodPending || gEodUnsent || !gEodJournalOK || gEodLegacy ||
     gEodLoadedScope!=EodScope()) return;
  if(trans.type==TRADE_TRANSACTION_REQUEST && EodHasCurrentRequest() &&
     result.request_id>0 && result.request_id==gEodRequestId &&
EodRequestMatches(request))
  {
     if((gEodOrder>0 && result.order>0 && gEodOrder!=result.order) ||
        (gEodDeal>0 && result.deal>0 && gEodDeal!=result.deal)) return;
     if(result.order>0)
     { gEodOrder=result.order; gEodOrderAttributable=true; gEodProof="DIRECT"; }
     if(result.deal>0) gEodDeal=result.deal;
     gEodRetcode=result.retcode;
     gEodRejectionProven=(gEodOrder==0 && gEodDeal==0 &&
EodKnownRejection(result.retcode));
     gEodRejectionDurable=gEodRejectionProven;
     gEodPendingWrite=true;
  }
  else if(trans.type==TRADE_TRANSACTION_DEAL_ADD && gEodOrderAttributable &&
          HistoryDealSelect(trans.deal))
  {
     ulong order; double volume;
     if(EodDealShape(trans.deal,order,volume) && order==gEodOrder)
```

```
    { gEodDeal=trans.deal; gEodPendingWrite=true; }
  }
}

bool EodPrepareRequest(ulong ticket,ulong pid,MqlTradeRequest &request,long
&execMode)
{
  if(!PositionSelectByTicket(ticket)) return false;
  string symbol; long magic,position,type;
  double volume;
  if(!PositionGetString(POSITION_SYMBOL,symbol) ||
!PositionGetInteger(POSITION_MAGIC,magic) ||
    !PositionGetInteger(POSITION_IDENTIFIER,position) ||
!PositionGetInteger(POSITION_TYPE,type) ||
    !PositionGetDouble(POSITION_VOLUME,volume) ||
!SymbolInfoInteger(Symbol(),SYMBOL_TRADE_EXEMODE,execMode) ||
    symbol!=Symbol() || magic!=(long)MagicCore || (ulong)position!=pid ||
    (type!=POSITION_TYPE_BUY && type!=POSITION_TYPE_SELL) ||
    !MathIsValidNumber(volume) || volume<=0) return false;
  // Preserve PositionClose(ticket): request the position's full volume once.
  // Broker rejection is reconciled; no splitting, new cap or sizing rule.
  MqlTick tick={};
  if(!SymbolInfoTick(Symbol(),tick) || tick.bid<=0 || tick.ask<=0 ||
    !MathIsValidNumber(tick.bid) || !MathIsValidNumber(tick.ask)) return false;
  request.action=TRADE_ACTION_DEAL; request.position=ticket;
  request.symbol=Symbol(); request.magic=(ulong)MagicCore;
  request.volume=volume;
request.type=(type==POSITION_TYPE_BUY?ORDER_TYPE_SELL:ORDER_TYPE_BUY);
   request.price=(request.type==ORDER_TYPE_SELL?tick.bid:tick.ask);
   request.deviation=(ulong)Inp_MaxDeviationPts;
   request.type_filling=ORDER_FILLING_FOK; request.type_time=ORDER_TIME_GTC;
   return true;
}

bool EodBeginAttempt()
{
  if(gEodPending || !gEodJournalOK || gEodAttemptSerial==(ulong)-1) return false;
  gEodAttemptSerial++;
  gEodBoundAttemptSerial=0; gEodRequestId=0; gEodRejectionProven=false;
  gEodRejectionDurable=false; gEodOrderAttributable=false; gEodLegacy=false;
  gEodOrder=0; gEodDeal=0; gEodRetcode=0; gEodProof="NONE";
gEodTerminal="NONE";
  gEodIdentity=EodAttemptIdentity(gEodAttemptSerial);
  return StringLen(gEodIdentity)==31;
}

void ProcessEOD(int hhmm)
{
```

```
  if(hhmm < Inp_EOD_CloseHHMM) return;
  if(armed.active)
  {
    WriteAudit("ABORT_EOD", DirStr(armed.dir), armed.mode, 0,
      armed.posRel, armed.distD1, armed.effort, 0, "NA",
      armed.vwapVig, armed.c1,armed.c2,armed.c3,armed.c4,armed.c5, 0, 0);
    ResetArmedSignal();
  }
  if(!EodLoadIntent())
  { EodSafetyAlert("Diario de fechamento indisponivel, incompleto ou contexto divergente");
return; }
  if(gEodUnsent)
  {
    // A failed/partial journal is never repaired by appending past the damage.
    if(!gEodJournalOK)
    { EodSafetyAlert("Falha de journal antes do envio; nenhum envio autorizado"); return; }
    gEodTerminal="UNSENT";
    if(!EodAppendIntent(false))
    { EodSafetyAlert("Nao foi possivel persistir a ausencia de envio"); return; }
    gEodUnsent=false; gEodPending=false;
  }
  if(gEodPending && !EodReconcileIntent())
  { EodSafetyAlert("Tentativa EOD sem vinculo unico ou sem evidencia terminal da
corretora"); return; }
  bool hasPosition=false,hasOrder=false;
  if(!EodOwnSnapshot(hasPosition,hasOrder))
  { EodSafetyAlert("Corretora desconectada ou snapshot proprio inconclusivo"); return; }
  if(!hasPosition && !hasOrder && !gEodPending)
 {
    if(!eodDone) Print("[REMORA2.0][EOD] Confirmado: sem posicao propria e sem
ordem/fechamento proprio pendente. Zeragem concluida.");
    eodDone=true; gEodSafetyLogged=false; return;
  }
  eodDone=false;
  if(hhmm>=1830)
  {
    if(!eodStuckLogged)
    {
      eodStuckLogged=true;
      PrintFormat("[REMORA2.0][EOD_STUCK] CRITICO: posicao ou ordem PROPRIA em
%s ainda pendente apos 18:30. Intervencao manual necessaria.",Symbol());
      WriteAudit("EOD_STUCK","NONE","NONE",0,0,0,0,0,"NA",0,0,0,0,0,0,0,0,0);
    }
    return;
  }
  if(!hasPosition) return;
  if(hasOrder)
```

```
  { EodSafetyAlert("Ordem propria ativa sem termino comprovado; nao sera adotada por
semelhanca"); return; }
  datetime now=TimeTradeServer();
  if(eodNextTry>0 && now<eodNextTry) return;
  eodNextTry=now+10;
  for(int i=PositionsTotal()-1;i>=0;i--)
  {
    if(!posInfo.SelectByIndex(i))
    { EodSafetyAlert("Posicao nao selecionavel; zeragem nao comprovada"); return; }
    string symbol; long magic,ticketValue,positionValue;
    if(!PositionGetString(POSITION_SYMBOL,symbol) ||
!PositionGetInteger(POSITION_MAGIC,magic) ||
       !PositionGetInteger(POSITION_TICKET,ticketValue) ||
!PositionGetInteger(POSITION_IDENTIFIER,positionValue))
    { EodSafetyAlert("Identidade da posicao nao pode ser consultada"); return; }
    if(symbol!=Symbol() || magic!=(long)MagicCore) continue;
    ulong ticket=(ulong)ticketValue,pid=(ulong)positionValue;
    if(ticket==0 || pid==0)
    { EodSafetyAlert("Ticket/identificador de posicao propria invalido"); return; }
    MqlTradeRequest request={};
    long execMode;
    if(!EodPrepareRequest(ticket,pid,request,execMode))
    { EodSafetyAlert("Fechamento exato/FOK integral nao pode ser preparado"); return; }
    gEodTicket=ticket; gEodPositionId=pid; gEodVolume=request.volume;
    gEodCloseType=request.type; gEodPrice=request.price; gEodExecMode=execMode;
    gEodDeviation=request.deviation;
    if(!EodBeginAttempt())
    { EodSafetyAlert("Identidade de tentativa EOD nao pode ser criada"); return; }
    gEodStarted=now; gEodPending=true; request.comment=gEodIdentity;
   if(!EodAppendIntent(true))
   {
     gEodUnsent=true;
     EodSafetyAlert("Identidade EOD nao persistida e verificada; solicitacao NAO enviada");
return;
   }
   // Recheck after disk IO; if position/volume/side changed, this request is unsent.
   MqlTradeRequest check={};
   long checkMode;
   bool flatPosition=false,conflict=false;
   if(!EodOwnSnapshot(flatPosition,conflict) || conflict ||
     !EodPrepareRequest(ticket,pid,check,checkMode) || check.volume!=request.volume ||
     check.type!=request.type || checkMode!=gEodExecMode)
   {
     gEodUnsent=true; gEodTerminal="UNSENT";
     EodSafetyAlert("Posicao ou ordens mudaram antes do envio; nenhuma solicitacao
enviada"); return;
   }
   // Preserve the persisted price, deviation and exact request; requotes stay refusals.
```

```
   MqlTradeResult closeResult={};
   bool sent=OrderSend(request,closeResult);
   EodBindRequestResult(closeResult);
   gEodPendingWrite=!EodAppendIntent(true);
   if(sent && closeResult.retcode==TRADE_RETCODE_DONE)
      PrintFormat("[REMORA2.0][EOD] Fechamento DONE ticket=%I64u. Aguardando
confirmacao terminal e flat.",ticket);
   else if(closeResult.retcode==TRADE_RETCODE_PLACED)
      PrintFormat("[REMORA2.0][EOD] Fechamento PLACED ticket=%I64u. Aguardando
execucao, sem repetir solicitacao pendente.",ticket);
   else
      PrintFormat("[REMORA2.0][EOD] Resultado ticket=%I64u | Ret=%u | %s |
Reconciliacao antes de nova tentativa.",ticket,closeResult.retcode,closeResult.comment);
   break;
  }
}


//+------------------------------------------------------------------+
//| OnTimer — reset diário, D-1 resiliente, BE-sync, EOD             |
//+------------------------------------------------------------------+
void OnTimer()
{
   if(TimeCurrent() < BacktestStart) return;

  MqlDateTime dt; TimeToStruct(TimeTradeServer(), dt);
  int key = dt.year * 1000 + dt.day_of_year;
  int hhmm = dt.hour * 100 + dt.min;
   if(key != gDayKey)
   {
      gDayKey = key;
      DailyReset();
      // SEM retorno geral após reset — fluxo continua
   }

   if((gQuotaState == COTA_DESCONHECIDA ||
       gQuotaHistoryState == HISTORICO_INDISPONIVEL || gQuotaPersistPending) &&
      TimeTradeServer() >= gQuotaNextRetry)
      ReconcileDailyQuota();

   UpdateForeignPositionState();

   // [2.435/F1] Retry original de 60s; recuperacao libera apenas futuras avaliacoes.
   // OnTick preserva newBar, janela operacional e todos os demais gates.
   if(!d1Ready && TimeTradeServer() >= nextD1Retry)
      TryCalcD1();

   // Sincronização pendente do BE (mapeamento PLACED/indisponível)
```

```
   if(beSyncPending)
   {
     ReloadOpenPositionsToBE();
     if(beCount > 0) beSyncPending = false;
     else if(!HasOwnPositionStrict() && !HasOwnPendingOrder()) beSyncPending = false;
   }

   ProcessEOD(hhmm);
   S2431ReconcileExecutions(); // [2.43.1/S2] somente Timer, limitado a 10s
   Diag2435Flush(); // [2.435/D] depois de TODO o trabalho operacional original
}

//+------------------------------------------------------------------+
//| FILTROS                                                          |
//+------------------------------------------------------------------+
bool SpreadOk()
{
  double spread = SymbolInfoDouble(Symbol(),SYMBOL_ASK) -
           SymbolInfoDouble(Symbol(),SYMBOL_BID);
  return (spread <= SpreadMaxPts);
}

bool MarginOk(ENUM_ORDER_TYPE otype, double lots)
{
 double margin=0.0;
 double price = (otype==ORDER_TYPE_BUY) ?
SymbolInfoDouble(Symbol(),SYMBOL_ASK)
                    : SymbolInfoDouble(Symbol(),SYMBOL_BID);
   if(!OrderCalcMargin(otype, Symbol(), lots, price, margin)) return false;
   return (AccountInfoDouble(ACCOUNT_MARGIN_FREE) >= margin);
}

//+------------------------------------------------------------------+
//| SIZING (MarginOk / Lotes por Margem Livre)                       |
//+------------------------------------------------------------------+
int FullSizeLot()
{
  double bal = AccountInfoDouble(ACCOUNT_BALANCE);
  if(bal < MargemPorContrato) return 0;
  int lots = (int)MathFloor(bal/MargemPorContrato);
  return MathMax(1, MathMin(lots, Inp_MaxLotes));
}

//+------------------------------------------------------------------+
//| ENVIO DE ORDEM (FOK) — SL/TP normalizados direcionalmente        |
//| Mapeamento de posição via ResultDeal->DEAL_POSITION_ID           |
//| (PROIBIDO usar ResultOrder como ticket de posição)               |
//| [E7] Revalidação final imediatamente antes de PositionOpen.      |
```

```
//| [E5] DONE/PLACED incrementa e PERSISTE a cota imediatamente. |
//| v2.40: 'mode' carrega a rota ("BASE_CONT"/"TURBO_CONT") e vai |
//| para o comentário da ordem, para o CSV e para o histórico.    |
//+------------------------------------------------------------------+
bool SendOrder(int direction, int lots, string mode, double slPts, double tpPts,
               double posRel, double distD1, double effort, double poc, string pocState,
               double vwapVig, int c1,int c2,int c3,int c4,int c5)
{
  ENUM_ORDER_TYPE otype = (direction==1) ? ORDER_TYPE_BUY :
ORDER_TYPE_SELL;
  string dStr = DirStr(direction);

   // Preço de referência SOMENTE para auditoria dos gates tardios abaixo.
   // NÃO alimenta SL/TP nem o envio: toda a geometria e a ordem derivam
   // exclusivamente do finalExecutablePrice capturado adiante.
   double refPrice = (direction==1) ? SymbolInfoDouble(Symbol(),SYMBOL_ASK)
                     : SymbolInfoDouble(Symbol(),SYMBOL_BID);

 // [E7] GATES TARDIOS — posição, cota e ordem própria do dia.
 // Cobrem a janela entre a última verificação do Estágio 2 e o envio.
 // Bloqueio tardio: não envia, não incrementa, não persiste; audita e sai.
 if(HasOpenPosition())
 {
   Print("[REMORA2.0][LATE_POSITION_BLOCK] Posição detectada imediatamente antes
do envio. Ordem NÃO enviada.");
   WriteAudit("LATE_POSITION_BLOCK", dStr, mode, refPrice, posRel, distD1, effort, poc,
pocState,
           vwapVig, c1,c2,c3,c4,c5, lots, 0);
  Diag2435SendReason(DIAG2435_LATE_POSITION_BLOCK);
  return false;
 }
 if(gQuotaState == COTA_DESCONHECIDA)
 {
    Print("[REMORA2.0][QUOTA_UNKNOWN] Ordem NÃO enviada: cota ainda não
reconciliada.");
    WriteAudit("QUOTA_UNKNOWN", dStr, mode, refPrice, posRel, distD1, effort, poc,
          pocState, vwapVig, c1,c2,c3,c4,c5, lots, 0);
    Diag2435SendReason(DIAG2435_QUOTA_UNKNOWN);
    return false;
 }
 if(ordersEnteredToday >= Inp_MaxTradesDay)
 {
    Print("[REMORA2.0][LATE_QUOTA_BLOCK] Cota diária já consumida imediatamente
antes do envio. Ordem NÃO enviada.");
    WriteAudit("LATE_QUOTA_BLOCK", dStr, mode, refPrice, posRel, distD1, effort, poc,
pocState,
           vwapVig, c1,c2,c3,c4,c5, lots, 0);
    Diag2435SendReason(DIAG2435_LATE_QUOTA_BLOCK);
```

```
    return false;
  }
  if(HasOwnPendingOrderToday())
  {
    Print("[REMORA2.0][LATE_PENDING_BLOCK] Ordem própria do dia ainda ativa
(símbolo+magic). Ordem NÃO enviada.");
    WriteAudit("LATE_PENDING_BLOCK", dStr, mode, refPrice, posRel, distD1, effort, poc,
pocState,
           vwapVig, c1,c2,c3,c4,c5, lots, 0);
    Diag2435SendReason(DIAG2435_LATE_PENDING_BLOCK);
    return false;
  }

  // [BLINDAGEM E2/E2+] CAPTURA DO TICK FINAL — FONTE DEFINITIVA DO PREÇO.
  // Neste call site, o parâmetro 'poc' contém o TRIGGER congelado
  // (ProcessArmedSignal é o ÚNICO chamador do SendOrder e passa 'trigger'
  // na posição de 'poc').
  MqlTick finalTick;
  if(!SymbolInfoTick(Symbol(), finalTick)) {
Diag2435SendReason(DIAG2435_FINAL_TICK_UNAVAILABLE);
    return false; }

  double finalExecutablePrice = (direction == 1) ? finalTick.ask : finalTick.bid;
  if(finalExecutablePrice <= 0.0) {
Diag2435SendReason(DIAG2435_FINAL_EXECUTABLE_INVALID);
    Diag2435SendTick(finalTick.bid,finalTick.ask,finalTick.time_msc,finalExecutablePrice);
    return false; }
 // O rompimento ainda precisa EXISTIR no instante do envio. O teto de
 // overshoot só limita o lado distante do trigger; um recuo do preço de
 // volta para dentro da região gera overshoot NEGATIVO, que passaria.
 bool finalBreakout =
   (direction == 1) ? (finalExecutablePrice >= poc) : (finalExecutablePrice <= poc);
 if(!finalBreakout)
 {
   PrintFormat("[REMORA2.0][LATE_BREAKOUT_LOST] Preço executável voltou para
dentro da região antes do envio | ExecPrice=%.2f | Trigger=%.2f | Ordem NÃO enviada.",
          finalExecutablePrice, poc);
   WriteAudit("LATE_BREAKOUT_LOST", dStr, mode, finalExecutablePrice,
         posRel, distD1, effort, poc, pocState,
         vwapVig, c1,c2,c3,c4,c5, lots, 0);
   Diag2435SendReason(DIAG2435_LATE_BREAKOUT_LOST);
   Diag2435SendTick(finalTick.bid,finalTick.ask,finalTick.time_msc,finalExecutablePrice);
   return false;
 }

 double lateOvershoot =
   (direction == 1) ? (finalExecutablePrice - poc) : (poc - finalExecutablePrice);
 if(lateOvershoot > Inp_Max_Trigger_Overshoot)
```

```
{
   PrintFormat("[REMORA2.0][LATE_OVERSHOOT_BLOCK] Preço executável afastou-se
do trigger antes do envio | ExecPrice=%.2f | Trigger=%.2f | Overshoot=%.2f | Max=%.2f |
Ordem NÃO enviada.",
           finalExecutablePrice, poc, lateOvershoot, Inp_Max_Trigger_Overshoot);
   WriteAudit("LATE_OVERSHOOT_BLOCK", dStr, mode, finalExecutablePrice,
         posRel, distD1, effort, poc, pocState,
         vwapVig, c1,c2,c3,c4,c5, lots, 0);
   Diag2435SendReason(DIAG2435_LATE_OVERSHOOT_BLOCK);
   Diag2435SendTick(finalTick.bid,finalTick.ask,finalTick.time_msc,finalExecutablePrice);
   return false;
}

// [BLINDAGEM E2+] finalExecutablePrice é a FONTE DEFINITIVA de price,
// SL, TP, STOPS_LEVEL e margem enviados. SL/TP recalculados a partir dele,
// preservando as distâncias estratégicas slPts/tpPts.
double price = finalExecutablePrice;
double slRaw = (direction==1) ? price - slPts : price + slPts;
double tpRaw = (direction==1) ? price + tpPts : price - tpPts;

// Normalização direcional: preserva AO MENOS a distância estratégica
double sl = NormEntrySL(direction, slRaw);
double tp = NormEntryTP(direction, tpRaw);

// CIRURGIA 13.6 — Validação de STOPS_LEVEL (sem reposicionamento silencioso)
// Revalidada com o preço final e o SL/TP recalculados.
long stopsLevel = SymbolInfoInteger(Symbol(), SYMBOL_TRADE_STOPS_LEVEL);
 double point      = SymbolInfoDouble(Symbol(), SYMBOL_POINT);
 double minDist = (double)stopsLevel * point;
 bool stopsOk = true;
 if(direction==1){ if((price - sl) < minDist || (tp - price) < minDist) stopsOk = false; }
 else         { if((sl - price) < minDist || (price - tp) < minDist) stopsOk = false; }
 if(!stopsOk)
 {
   PrintFormat("[REMORA2.0][STOPS_LEVEL_BLOCK] SL/TP violam STOPS_LEVEL=%d
pts. Entrada BLOQUEADA (sem reposicionamento silencioso).",
          (int)stopsLevel);
   WriteAudit("STOPS_LEVEL_BLOCK", dStr, mode, price, posRel, distD1, effort, poc,
pocState,
          vwapVig, c1,c2,c3,c4,c5, lots, 0);
   Diag2435SendReason(DIAG2435_STOPS_LEVEL_BLOCK);
   Diag2435SendTick(finalTick.bid,finalTick.ask,finalTick.time_msc,finalExecutablePrice);
   return false;
 }

// Margem revalidada por último (MarginOk usa o preço de mercado corrente)
if(!MarginOk(otype,(double)lots))
{
```

```
   WriteAudit("NO_MARGIN", dStr, mode, price, posRel, distD1, effort, poc, pocState,
         vwapVig, c1,c2,c3,c4,c5, lots, 0);
   Diag2435SendReason(DIAG2435_NO_MARGIN);
   Diag2435SendTick(finalTick.bid,finalTick.ask,finalTick.time_msc,finalExecutablePrice);
   return false;
}

trade.SetExpertMagicNumber(MagicCore);
trade.SetTypeFilling(ORDER_FILLING_FOK);

bool sent = trade.PositionOpen(
   Symbol(),
   otype,
   (double)lots,
   price,
   sl,
   tp,
   "REMORA2_"+mode
);

uint ret = trade.ResultRetcode();
bool ok =
  sent &&
  (
     ret == TRADE_RETCODE_DONE ||
     ret == TRADE_RETCODE_PLACED
  );
if(ok)
{
   // [E5] TIRO CONSUMIDO: somente DONE/PLACED consome a cota.
   // Incrementa e persiste O MAIS CEDO POSSÍVEL após o retcode aceito,
   // ANTES do mapeamento/BE/logs, para sobreviver a queda imediata.
   ordersEnteredToday++;
   PersistShotCount();
   S2431RecordAccepted(trade.ResultOrder(),trade.ResultDeal(),ret); // somente telemetria

  // CIRURGIA 13.7 — Mapeamento correto da POSIÇÃO
  ulong posTicket = 0;
  if(ret==TRADE_RETCODE_DONE)
  {
     ulong dealTicket = trade.ResultDeal();
     if(dealTicket>0 && HistoryDealSelect(dealTicket))
     {
        long posId = HistoryDealGetInteger(dealTicket, DEAL_POSITION_ID);
        for(int i=PositionsTotal()-1;i>=0;i--)
        {
          if(!posInfo.SelectByIndex(i)) continue;
          if(posInfo.Identifier()!=posId) continue;
```

```
         if(posInfo.Symbol()!=Symbol()) continue;
         if(posInfo.Magic()!=(long)MagicCore) continue;
         posTicket = posInfo.Ticket(); // POSITION_TICKET real
         break;
        }
      }
    }

    if(posTicket>0 && beCount<64)
    {
      beList[beCount].ticket=posTicket;
      beList[beCount].applied=false;
      beList[beCount].backoffUntil=0;
      beCount++;
    }
    else
    {
      // PLACED ou mapeamento indisponível: sincroniza e resolve no OnTimer
      ReloadOpenPositionsToBE();
      if(beCount==0) beSyncPending = true;
    }

    lastEntryTime = iTime(Symbol(), Inp_Timeframe, 0);
    PrintFormat("[REMORA2.0][%s] SOLICITACAO ACEITA %s | %d lotes @ %.2f | SL=%.2f
TP=%.2f | PosRel=%.3f DistD1=%.2f WW=%.2f TRIG=%.2f(%s)",
          mode, dStr, lots, price, sl, tp, posRel, distD1, effort, poc, pocState);
     WriteAudit("ORDER_ACCEPTED", dStr, mode, price, posRel, distD1, effort, poc,
pocState,
          vwapVig, c1,c2,c3,c4,c5, lots, (int)ret);
   }
   else
   {
     PrintFormat("[REMORA2.0][%s] FALHA | Ret=%u | %s", mode, ret,
trade.ResultComment());
     WriteAudit("ORDER_FAILED", dStr, mode, price, posRel, distD1, effort, poc, pocState,
          vwapVig, c1,c2,c3,c4,c5, lots, (int)ret);
   }
   // [2.435/D] Copias APOS PositionOpen, persistencia da cota e gestao originais.
   Diag2435SendReason(ok ? DIAG2435_ORDER_ACCEPTED :
DIAG2435_ORDER_FAILED);
   Diag2435SendTick(finalTick.bid,finalTick.ask,finalTick.time_msc,finalExecutablePrice);
   Diag2435SendOutcome(sent,ok,ret,price,sl,tp,trade.ResultOrder(),trade.ResultDeal());
   return ok;
}

//+------------------------------------------------------------------+
//| SIGMA DE RISCO — nivelado pelo teto financeiro (SL/BE/Locks) |
//| [E-doc] C1, C5 (bandas), deslocamento/corpo e ancoragem de POC |
```

```
//| usam SigmaD1 real; C2 e C3 usam DISTÂNCIAS FIXAS genéricas.        |
//| SigmaRisk atua SOMENTE no módulo de risco (SL, BE, Lock1/2). |
//+------------------------------------------------------------------+
double SigmaRisk()
{
   if(Inp_Mult_SL <= 0.0) return SigmaD1;
   return MathMin(SigmaD1, Inp_Max_SL_Pts / Inp_Mult_SL);
}

//+------------------------------------------------------------------+
//| GERENCIADOR DE PROTEÇÃO DINÂMICA (Break-Even + Profit Lock)               |
//| Cirurgia 13.4: guardas de topo, backoff, releitura de SL/TP, |
//| comparação por índice de tick, STOPS/FREEZE, retcodes.             |
//| [E13] A proteção exige apenas SigmaD1>0 + posição própria; ela |
//| NÃO depende da estrutura D-1 e NÃO libera novas entradas.           |
//+------------------------------------------------------------------+
void ManageBreakEven()
{
   if(beCount==0) return;
   if(SigmaD1<=0.0) return; // sem Sigma válido, BE não atua

 datetime now     = TimeTradeServer();
 double point   = SymbolInfoDouble(Symbol(), SYMBOL_POINT);
 long stopsLevel = SymbolInfoInteger(Symbol(), SYMBOL_TRADE_STOPS_LEVEL);
 long freezeLevel = SymbolInfoInteger(Symbol(), SYMBOL_TRADE_FREEZE_LEVEL);
 double minStopDist = (double)stopsLevel * point;
double minFreezeDist = (double)freezeLevel * point;

for(int i = 0; i < beCount; i++)
{
  if(beList[i].ticket == 0) continue;
  if(beList[i].backoffUntil > 0 && now < beList[i].backoffUntil) continue;
  if(!posInfo.SelectByTicket(beList[i].ticket)) continue;
  if(posInfo.Symbol()!=Symbol()) continue;            // só posições próprias
  if(posInfo.Magic() !=(long)MagicCore) continue;

 // Releitura de SL/TP em TODA tentativa
 double entry = posInfo.PriceOpen();
 double current = posInfo.PriceCurrent();
 double curSL = posInfo.StopLoss();
 double curTP = posInfo.TakeProfit();
 int dir = (posInfo.PositionType() == POSITION_TYPE_BUY) ? 1 : -1;
 double profPts = (double)dir * (current - entry);

 // LÊ OS INPUTS DINÂMICOS DO PAINEL DO MT5 BASEADOS NO SIGMA VIGENTE
 double triggerBE = SigmaRisk() * Inp_Mult_BE;
 double triggerLock1 = SigmaRisk() * Inp_Mult_Lock1_Trig;
 double profitLock1 = SigmaRisk() * Inp_Mult_Lock1_Prof;
```

```
double triggerLock2 = SigmaRisk() * Inp_Mult_Lock2_Trig;
double profitLock2 = SigmaRisk() * Inp_Mult_Lock2_Prof;

// LÓGICA DE ESCALONAMENTO (A Escada Perfeita) — maior degrau primeiro
double targetSL = 0.0;
bool want = false;
if(profPts >= triggerLock2)    { targetSL = entry + (double)dir * profitLock2; want=true; }
else if(profPts >= triggerLock1) { targetSL = entry + (double)dir * profitLock1; want=true; }
else if(profPts >= triggerBE) { targetSL = entry;                    want=true; }

if(!want) continue;

// Normalização do SL de PROTEÇÃO: BUY floor | SELL ceil
targetSL = NormProtSL(dir, targetSL);

// Comparação por ÍNDICE INTEIRO DE TICK + melhoria direcional
long idxNew = TickIndex(targetSL);
long idxCur = (curSL>0.0) ? TickIndex(curSL) : 0;
bool improves;
if(dir==1) improves = (curSL<=0.0) ? true : (idxNew > idxCur);
else     improves = (curSL<=0.0) ? true : (idxNew < idxCur);
if(!improves) continue;
if(curSL>0.0 && idxNew==idxCur) continue; // anti-spam: mesmo tick

// STOPS_LEVEL / FREEZE_LEVEL
double dist = MathAbs(current - targetSL);
      if(minStopDist > 0.0 && dist < minStopDist) continue;
      if(minFreezeDist > 0.0 && dist <= minFreezeDist) continue;

      trade.SetExpertMagicNumber(MagicCore);
      bool sent = trade.PositionModify(beList[i].ticket, targetSL, curTP);
      uint ret = trade.ResultRetcode();

      if(sent && ret==TRADE_RETCODE_DONE)
      {
        beList[i].backoffUntil = 0;
        // 'applied' = BE (ou melhor) confirmado
        if((dir==1 && idxNew >= TickIndex(entry)) || (dir==-1 && idxNew <= TickIndex(entry)))
          beList[i].applied = true;
        PrintFormat("[%s] PROTEÇÃO ATIVA | Ticket=%I64u | SL movido para %.2f",
              Symbol(), beList[i].ticket, targetSL);
      }
      else if(ret==TRADE_RETCODE_PLACED)
      {
        // Não marca aplicado; aguarda confirmação com backoff
        beList[i].backoffUntil = now + 5;
      }
      else
```

```
      {
        beList[i].backoffUntil = now + 5; // falha: backoff 5s (sem spam)
        PrintFormat("[%s][BE] Falha modify ticket=%I64u | Ret=%u | %s | retry em 5s.",
              Symbol(), beList[i].ticket, ret, trade.ResultComment());
      }
    }
}

//+------------------------------------------------------------------+
//| CONTAGEM ANTI-FRAGMENTAÇÃO B3 — símbolo + magic + ENTRY_IN
|
//| [E5] Autoridade para deals EXECUTADOS; a cota final do dia é |
//| max(CountTodayTrades, valor persistido) — ver Restore/OnInit. |
//+------------------------------------------------------------------+
ENUM_QUOTA_HISTORY CountTodayTrades(int &count)
{
  count = 0;
  datetime now = TimeTradeServer();
  MqlDateTime dt; TimeToStruct(now, dt);
  dt.hour=0; dt.min=0; dt.sec=0;
  datetime dayStart = StructToTime(dt);
  if(!HistorySelect(dayStart, now)) return HISTORICO_INDISPONIVEL;

   // Mesma anti-fragmentação da 2.43: POSITION_ID único, símbolo+Magic+ENTRY_IN.
   ulong uniq[64]; int uc=0;
   int total = HistoryDealsTotal();
   for(int i=0;i<total;i++)
   {
      ulong d = HistoryDealGetTicket(i);
      string symbol; long magic=0, entry=0, pid=0;
      if(d==0 || !HistoryDealGetString(d, DEAL_SYMBOL, symbol))
        return HISTORICO_INDISPONIVEL;
      if(symbol != Symbol()) continue;
      if(!HistoryDealGetInteger(d, DEAL_MAGIC, magic)) return
HISTORICO_INDISPONIVEL;
      if(magic != MagicCore) continue;
      if(!HistoryDealGetInteger(d, DEAL_ENTRY, entry)) return
HISTORICO_INDISPONIVEL;
      if(entry != DEAL_ENTRY_IN) continue;
      if(!HistoryDealGetInteger(d, DEAL_POSITION_ID, pid) || pid <= 0)
        return HISTORICO_INDISPONIVEL;
      bool found=false;
      for(int j=0;j<uc;j++) if(uniq[j]==(ulong)pid){found=true;break;}
      if(!found && uc<64) uniq[uc++]=(ulong)pid;
   }
   count = uc;
   return HISTORICO_OK;
}
```

```
//+==================================================================+
//| [v2.42] NÚCLEO PURO DA MATRIZ — ComputeMatrixSnapshot()          |
//| EXTRAÇÃO LITERAL das equações da EvaluateMatrix v2.41 (ABS v2.41 |
//| + CONT v2.40). NÃO escreve CSV, NÃO imprime, NÃO arma, NÃO altera|
//| UpperBand1/LowerBand1, gC1Audit, armed, cota ou persistência. |
//| Única saída: a struct 's'. Mesma ordem de parada antecipada da |
//| v2.41 (POC só é calculado após os portões de C5, como antes). |
//| Acréscimo SOMENTE de telemetria: absStallRaw e c1RawHyp (NÃO |
//| entram em c1, absValid ou contValid).                  |
//| [v2.43] ÚNICA alteração operacional: absPocContextOk passa a |
//| aceitar absFollowingReaction. Telemetria de fluxo adicionada. |
//| Bloco CONT: NENHUMA linha alterada.                    |
//+==================================================================+
ENUM_MATRIX_SNAP_STATUS ComputeMatrixSnapshot(MatrixSnapshot &s)
{
   ResetMatrixSnapshot(s);

  //--- Validações estruturais (mesma ordem da v2.41) -----------------
  if(VWAP_D1<=0.0 || !VWAP_D1_Official){ s.status = MSS_VWAP_D1_INVALID; return
s.status; }
  if(SigmaD1<=0.0){ s.status = MSS_SIGMA_INVALID; return s.status; }
  if(H1<=0.0 || L1<=0.0 || H1<=L1){ s.status = MSS_STRUCT_INVALID; return s.status; }

    // Cirurgia 7 — VWAP vigente vem do cache incremental (validado no OnTick)
    s.vwapVig = gVwapValue;
    if(s.vwapVig<=0.0){ s.status = MSS_STRUCT_INVALID; return s.status; }

    s.upper = s.vwapVig + SigmaD1 * BandMultiplier;
 s.lower = s.vwapVig - SigmaD1 * BandMultiplier;
 s.bandsComputed = true;
 s.bandRange = s.upper - s.lower;
 if(s.bandRange<=0.0){ s.status = MSS_STRUCT_INVALID; return s.status; }

 //--- Preço estrutural (última barra fechada do TF) ----------------
 s.price     = iClose(Symbol(), Inp_Timeframe, 1);
 s.open1     = iOpen (Symbol(), Inp_Timeframe, 1);
 s.candleBody = MathAbs(s.price - s.open1); // CORPO da vela (NUNCA High-Low)
 if(s.price<=0.0){ s.status = MSS_STRUCT_INVALID; return s.status; }

 //--- C5 — régua VWAP + teto de vidro (portão por rota) ----------
 s.posRel = (s.price - s.lower) / s.bandRange;
 s.glass = (s.posRel > Inp_VWAP_Outer_Pct) || (s.posRel < (1.0 - Inp_VWAP_Outer_Pct));

 s.dir = 0;                        // ROTA A — ABS
 if(s.posRel >= Inp_VWAP_Inner_Pct)        s.dir = -1; // extensão p/ cima => SELL
 else if(s.posRel <= (1.0 - Inp_VWAP_Inner_Pct)) s.dir = 1; // extensão p/ baixo => BUY
```

```
  s.dirCont = 0;                      // ROTA B — CONT
  if(s.posRel >= Inp_VWAP_Inner_Pct)        s.dirCont = 1; // extensão superior =>
continuação BUY
  else if(s.posRel <= (1.0 - Inp_VWAP_Inner_Pct)) s.dirCont = -1; // extensão inferior =>
continuação SELL

  // Paradas antecipadas IDÊNTICAS à v2.41 (antes do cálculo de POC)
  if(s.glass && !Inp_EnableAbsorption){ s.status = MSS_STOP_KAMIKAZE_CONT_ONLY;
return s.status; }
  if(s.dir==0 && s.dirCont==0){ s.status = MSS_STOP_NO_C5_TRIGGER; return s.status; }

 // [v2.41] C5 operacional POR ROTA (portão; NÃO soma score)
 s.c5AbsOk = (s.dir != 0);          // Glass NÃO invalida ABS
 s.c5ContOk = (s.dirCont != 0) && !s.glass; // Glass bloqueia CONT

 //--- C2 (Vazio) / C3 (Retenção) — DISTÂNCIAS FIXAS EM PONTOS ----
 double distSigned = s.price - VWAP_D1;
 s.distAbs       = MathAbs(distSigned);
 bool priceAbove = (s.price > VWAP_D1);

 // C1 ABS — limites independentes, ambos adaptativos ao Sigma. (INALTERADOS)
 s.minDispl = SigmaD1 * Inp_Min_Displ_Sigma;
 s.maxBody = SigmaD1 * Inp_Max_Body_Sigma;
 // [FIX-1/FIX-2] C1 CONT — piso de deslocamento PRÓPRIO da continuação
 s.contMinDispl = SigmaD1 * Inp_Cont_Min_Displ_Sigma;

 //--- ROTA A (ABS) — inalterado
 bool sideC2Ok =
    ((s.dir == -1 && priceAbove) ||
   (s.dir == 1 && !priceAbove)
  );
s.c2 = (sideC2Ok && s.distAbs >= Inp_Pts_C2_Longa);

bool sideC3Ok =
  (
     (s.dir == -1 && !priceAbove) ||
     (s.dir == 1 && priceAbove)
  );
s.c3 = (sideC3Ok &&
      s.distAbs >= Inp_Pts_C3_Min &&
      s.distAbs <= Inp_Pts_C3_Max);

// Cirurgia 1 — bloco geométrico unificado
s.c23 = (s.c2 || s.c3);

//--- ROTA B (CONT) — mesmos thresholds, alinhados à continuação
bool sideContOk =
   (
```

```
     (s.dirCont == 1 && s.price > VWAP_D1) ||
     (s.dirCont == -1 && s.price < VWAP_D1)
   );
s.c2Cont = (sideContOk && s.distAbs >= Inp_Pts_C2_Longa);
s.c3Cont = (sideContOk &&
         s.distAbs >= Inp_Pts_C3_Min &&
         s.distAbs <= Inp_Pts_C3_Max);
s.c23Cont = (s.c2Cont || s.c3Cont);

//--- C4 — EXTREMO (posição no Range D-1 / 40% exaustão) ---------
s.posRange = (s.price - L1) / (H1 - L1);
// ABS — inalterado
s.c4 = (s.dir==-1) ? (s.posRange >= 1.0 - Inp_Rule_40_60)
           : (s.posRange <= Inp_Rule_40_60);
// CONT — extremo A FAVOR da tendência
s.c4Cont = (s.dirCont==1) ? (s.posRange >= 1.0 - Inp_Rule_40_60)
                : (s.posRange <= Inp_Rule_40_60);

//--- CIRURGIAS 2/3 — POC E PREÇO NAS MESMAS JANELAS + 4 ESTADOS -
// Janelas ancoradas na VIRADA REAL da M5 (t0), sem sobreposição.
datetime t0    = iTime(Symbol(), Inp_Timeframe, 0);
ulong t0_msc     = (ulong)t0 * 1000;
ulong inertia_msc = (ulong)Inp_Inertia_Timeout * 1000;

if(inertia_msc > 0 &&
   t0_msc > 2 * inertia_msc)
{
   ulong fromNow = t0_msc - inertia_msc;
   ulong toNow = t0_msc - 1;
     ulong fromPrev = t0_msc - 2 * inertia_msc;
     ulong toPrev = t0_msc - inertia_msc - 1;

    s.winNowOk = ComputePOCWindow(fromNow, toNow, s.pocNow, s.vA,
s.lastPriceNow);
    s.winPrevOk = ComputePOCWindow(fromPrev, toPrev, s.pocPrev, s.vB,
s.lastPricePrev);
  }
  s.windowsValid = (s.winNowOk && s.winPrevOk);

 s.pocState = "POC_INVALID";
 if(s.windowsValid)
 {
    // Preço e POC representam EXATAMENTE as mesmas duas janelas
    s.priceMove = s.lastPriceNow - s.lastPricePrev;
    s.pocMove = s.pocNow - s.pocPrev;

     s.displacementOk = (MathAbs(s.priceMove) >= s.minDispl);
     if(s.displacementOk)
```

```
      s.moveDir = (s.priceMove > 0.0) ? 1 : -1;

    if(!s.displacementOk)
      s.pocState = "POC_NO_DISPLACEMENT";
    else
    {
      bool follows = (s.pocMove * s.priceMove > 0.0) &&
             (MathAbs(s.pocMove) >= MathAbs(s.priceMove) * Inp_POC_FollowRatio);
      s.pocState = follows ? "POC_FOLLOWING" : "POC_DIVERGENT";
    }
}

//--- [FIX-2] GATE DE DESLOCAMENTO + ESTADO DA POC PRÓPRIOS DA CONT
s.contDisplacementOk =
   s.windowsValid &&
   MathAbs(s.priceMove) >= s.contMinDispl;

s.contMoveDir = 0;
s.contPocState = "POC_INVALID";
if(s.windowsValid)
{
   if(s.contDisplacementOk)
      s.contMoveDir = (s.priceMove > 0.0) ? 1 : -1;

    if(!s.contDisplacementOk)
      s.contPocState = "POC_NO_DISPLACEMENT";
    else
    {
      bool contFollows =
        (s.pocMove * s.priceMove > 0.0) &&
        (MathAbs(s.pocMove) >=
         MathAbs(s.priceMove) * Inp_POC_FollowRatio);
       s.contPocState =
        contFollows ? "POC_FOLLOWING"
              : "POC_DIVERGENT";
    }
}

// 'anchored' — TELEMETRIA APENAS (não define estado, não entra em C1)
double pocDist = (s.winNowOk && s.pocNow > 0.0) ? MathAbs(s.price - s.pocNow) :
DBL_MAX;
s.anchored =
   s.winNowOk &&
   s.pocNow > 0.0 &&
   pocDist <= SigmaD1 * BandMultiplier * Inp_POC_AnchorMult;

//--- CIRURGIA 4 — C1: PILAR MESTRE (fluxo fechado do CACHE) -----
s.wwok = wwValid;
```

```
s.effort = wwEffortClosed;
s.waveDir = wwWaveDirClosed;

//--- ROTA A (ABS) — C1 POR CONTEXTO (v2.41 + v2.43)
s.c1_effort_ok   = s.wwok && s.effort >= Inp_Vol_Effort;
s.c1_body_ok       = s.candleBody <= s.maxBody;
s.c1_displacement_ok = s.displacementOk;
s.c1_movedir_ok     = s.moveDir == -s.dir;
s.c1_wavedir_ok     = s.waveDir == s.moveDir;

// A) DIVERGÊNCIA
s.absDivergence =
   s.windowsValid &&
   (s.pocState == "POC_DIVERGENT");

 // B) PARALISAÇÃO (STALL_STRICT — OPERACIONAL): sem deslocamento E corpo travado
 s.absStallStrict =
    s.windowsValid &&
    (s.pocState == "POC_NO_DISPLACEMENT") &&
    s.c1_body_ok;

// [v2.42] STALL_RAW — SOMENTE TELEMETRIA (sem body_ok). NÃO entra em c1.
s.absStallRaw =
   s.windowsValid &&
   (s.pocState == "POC_NO_DISPLACEMENT");

// [v2.43] C) FOLLOWING DIRECIONAL — preço+POC já caminham NA DIREÇÃO DA
// REVERSÃO desejada (moveDir == dir). SEM waveDir fechado como veto,
// SEM candleDir, displacementOk INALTERADO.
s.absFollowingReaction =
   s.windowsValid &&
   (s.pocState == "POC_FOLLOWING") &&
   s.displacementOk &&
   (s.moveDir == s.dir);

// [v2.43] FOLLOWING aprofundando o extremo original (moveDir == -dir).
// NÃO valida ABS. Usado somente para diagnóstico.
s.absFollowingExtension =
   s.windowsValid &&
   (s.pocState == "POC_FOLLOWING") &&
   s.displacementOk &&
   (s.moveDir == -s.dir);

// [v2.43] FOLLOWING_EXTENSION e POC_INVALID NUNCA validam a ABS.
s.absPocContextOk =
   s.absDivergence ||
   s.absStallStrict ||
```

```
   s.absFollowingReaction;
 s.c1_poc_ok = s.absPocContextOk;

 s.c1 =
   s.c1_effort_ok &&
   s.absPocContextOk;

  // [v2.42/v2.43] Hipotético SOMENTE para medição (NÃO operacional):
  // C1 com STALL_RAW no lugar de STALL_STRICT.
  s.c1RawHyp = s.c1_effort_ok && (s.absDivergence || s.absStallRaw ||
s.absFollowingReaction);

 s.absCtxTag = s.absDivergence        ? "ABS_CONTEXT_DIVERGENCE"
       : (s.absStallStrict    ? "ABS_CONTEXT_STALL"
       : (s.absFollowingReaction ? "ABS_CONTEXT_REACTION_FOLLOWING"
                   : "ABS_CONTEXT_NONE"));

 //--- [v2.43] TELEMETRIA DE FLUXO — SOMENTE DIAGNÓSTICO (NÃO vota) ---
 if(!s.c1_effort_ok)        s.flowRegime = "FLOW_WEAK";
 else if(!s.displacementOk)      s.flowRegime = "FLOW_STALL";
 else if(s.moveDir == s.dir) s.flowRegime = "FLOW_REACTION";
 else if(s.moveDir == -s.dir) s.flowRegime = "FLOW_EXTENSION";
 else                 s.flowRegime = "FLOW_NA"; // defensivo (dir==0 inatingível aqui)

 s.volRatio      = (s.vB > 0.0) ? s.vA / s.vB : 0.0;
 s.priceEfficiency = (s.minDispl > 0.0) ? MathAbs(s.priceMove) / s.minDispl : 0.0;
 s.pocResponse = (MathAbs(s.priceMove) > 0.0) ? MathAbs(s.pocMove) /
MathAbs(s.priceMove) : 0.0;
    //--- ROTA B (CONT) — C1_CONT
    s.candleDir = (s.price > s.open1) ? 1 : ((s.price < s.open1) ? -1 : 0);

    s.cont_effort_ok    = s.wwok && s.effort >= Inp_Cont_Vol_Effort;
    s.cont_displacement_ok = s.contDisplacementOk;
    s.cont_poc_ok       = (s.contPocState == "POC_FOLLOWING");
    s.cont_movedir_ok      = (s.contMoveDir == s.dirCont);
    s.cont_wavedir_ok      = (s.waveDir == s.dirCont);
    s.cont_candle_dir_ok =
      !Inp_Cont_RequireCandleDirection ||
      (s.dirCont == 1 && s.price > s.open1) ||
      (s.dirCont == -1 && s.price < s.open1);

    s.c1Cont =
      s.cont_effort_ok &&
      s.cont_displacement_ok &&
      s.cont_poc_ok &&
      s.cont_movedir_ok &&
      s.cont_wavedir_ok &&
      s.cont_candle_dir_ok;
```

```
   //--- CIRURGIA 1 — CLASSIFICAÇÃO FINAL (C1 OBRIGATÓRIO) ----------
   // ABS — fórmula inalterada
   s.turboAbs = s.c1 && s.c23 && s.c4;
   s.baseAbs = s.c1 && (s.c23 || s.c4) && !s.turboAbs;
   s.scoreEff =
     (s.c1 ? 1 : 0) +
     (s.c23 ? 1 : 0) +
     (s.c4 ? 1 : 0);

   // CONT — mesma filosofia 2/3 e 3/3
   s.turboCont = s.c1Cont && s.c23Cont && s.c4Cont;
   s.baseCont = s.c1Cont && (s.c23Cont || s.c4Cont) && !s.turboCont;
   s.scoreContEff =
     (s.c1Cont ? 1 : 0) +
     (s.c23Cont ? 1 : 0) +
     (s.c4Cont ? 1 : 0);

   //--- VALIDADE POR ROTA (mesma fórmula v2.41) ---------------------
   s.absValid = Inp_EnableAbsorption && s.c5AbsOk && (s.turboAbs || s.baseAbs);
   s.contValid = Inp_EnableContinuation && s.c5ContOk && (s.turboCont || s.baseCont);

   s.status = MSS_OK;
   return s.status;
}

//+------------------------------------------------------------------+
//| [v2.42] HELPERS DE TELEMETRIA SHADOW (somente leitura/escrita    |
//| no CSV auxiliar e no log; NUNCA alteram estado operacional)      |
//+------------------------------------------------------------------+
string MatrixStatusStr(ENUM_MATRIX_SNAP_STATUS st)
{
  switch(st)
  {
    case MSS_NOT_COMPUTED:                return "NOT_COMPUTED";
    case MSS_OK:                  return "OK";
    case MSS_VWAP_D1_INVALID:                return "VWAP_D1_INVALID";
    case MSS_SIGMA_INVALID:                return "SIGMA_INVALID";
    case MSS_STRUCT_INVALID:                return "STRUCT_INVALID";
    case MSS_STOP_KAMIKAZE_CONT_ONLY: return "STOP_KAMIKAZE_CONT_ONLY";
    case MSS_STOP_NO_C5_TRIGGER:                return
"STOP_NO_C5_TRIGGER";
    case MSS_SHADOW_DATA_INVALID: return "SHADOW_DATA_INVALID";
  }
  return "UNKNOWN";
}

string AbsContextStr(const MatrixSnapshot &s)
```

```
{
  if(s.status != MSS_OK)     return "NA";
  if(s.absDivergence)      return "DIVERGENCE";
  if(s.absStallStrict)  return "STALL_STRICT";
  if(s.absFollowingReaction) return "REACTION_FOLLOWING";   // [v2.43]
  if(s.absFollowingExtension) return "FOLLOWING_EXTENSION_BLOCK"; // [v2.43]
diagnóstico
  if(s.absStallRaw)      return "STALL_RAW";
  return "NONE";
}

string AbsClassStr(const MatrixSnapshot &s)
{
  if(s.status != MSS_OK) return "NA";
  return s.turboAbs ? "TURBO" : (s.baseAbs ? "BASE" : "NONE");
}

string ContClassStr(const MatrixSnapshot &s)
{
  if(s.status != MSS_OK) return "NA";
  return s.turboCont ? "TURBO_CONT" : (s.baseCont ? "BASE_CONT" : "NONE");
}

// Flags SOMENTE diagnósticas (relativas ao Armed atual).
// SAME_ROUTE_NEW_VALID          = rota armada válida nesta nova M5 (qualquer
direção)
// ARMED_CONTEXT_STILL_VALID = idem E mesma direção do Armed
// OPPOSITE_ROUTE_VALID         = a outra rota válida nesta nova M5
// ROUTE_CONFLICT_SHADOW = ABS e CONT válidas simultaneamente
void ComputeShadowFlags(const MatrixSnapshot &s,
              bool &oppositeValid, bool &sameRouteValid, int &sameRouteDir,
              bool &armedStillValid, bool &conflict)
{
   oppositeValid = false; sameRouteValid = false; sameRouteDir = 0;
   armedStillValid = false; conflict = false;
   if(!armed.active || s.status != MSS_OK) return;

   if(armed.route == "ABS")
   {
     sameRouteValid = s.absValid;
     sameRouteDir = s.absValid ? s.dir : 0;
     oppositeValid = s.contValid;
   }
   else if(armed.route == "CONT")
   {
     sameRouteValid = s.contValid;
     sameRouteDir = s.contValid ? s.dirCont : 0;
     oppositeValid = s.absValid;
```

```
   }
   armedStillValid = sameRouteValid && (sameRouteDir == armed.dir);
   conflict    = s.absValid && s.contValid;
}

void ShAdd(string &line, string v){ if(StringLen(line) > 0) line += ","; line += v; }
string ShD2(double v){ return DoubleToString(v, 2); }
string ShD4(double v){ return DoubleToString(v, 4); }
string ShI (int v) { return IntegerToString(v); }
string ShB (bool v) { return v ? "1" : "0"; }

void WriteShadowHeader()
{
 if(shadowHandle == INVALID_HANDLE) return;
 string h = "";
 ShAdd(h,"TIMESTAMP");      ShAdd(h,"EVENT");     ShAdd(h,"ORIGIN");
 ShAdd(h,"BAR_TIME");      ShAdd(h,"SNAP_STATUS");
 ShAdd(h,"ARMED_ACTIVE"); ShAdd(h,"ARMED_ROUTE");
ShAdd(h,"ARMED_MODE");
 ShAdd(h,"ARMED_DIR");      ShAdd(h,"ARMED_REGION_HIGH");
ShAdd(h,"ARMED_REGION_LOW");
 ShAdd(h,"ARMED_TRIGGER"); ShAdd(h,"ARMED_BARS_PASSED");
 ShAdd(h,"PRICE");      ShAdd(h,"OPEN");      ShAdd(h,"SIGMA");
 ShAdd(h,"VWAP_VIG");      ShAdd(h,"POSREL");     ShAdd(h,"GLASS");
 ShAdd(h,"DIST_D1");     ShAdd(h,"EFFORT");      ShAdd(h,"WAVE_DIR_CLOSED");
 ShAdd(h,"CANDLE_BODY"); ShAdd(h,"MAX_BODY");          ShAdd(h,"CANDLE_DIR");
 ShAdd(h,"WINDOWS_VALID"); ShAdd(h,"PRICE_MOVE");        ShAdd(h,"POC_MOVE");
 ShAdd(h,"POC_NOW");        ShAdd(h,"POC_PREV");    ShAdd(h,"VOL_NOW");
 ShAdd(h,"VOL_PREV");      ShAdd(h,"MIN_DISPL_ABS");
ShAdd(h,"MIN_DISPL_CONT");
 // ABS (20)
 ShAdd(h,"ABS_DIR");       ShAdd(h,"ABS_C5");       ShAdd(h,"ABS_EFFORT_OK");
 ShAdd(h,"ABS_POC_STATE"); ShAdd(h,"ABS_CONTEXT");
ShAdd(h,"ABS_DIVERGENCE");
 ShAdd(h,"ABS_STALL_STRICT"); ShAdd(h,"ABS_STALL_RAW");
ShAdd(h,"ABS_BODY_OK");
 ShAdd(h,"ABS_DISPLACEMENT_OK"); ShAdd(h,"ABS_MOVE_DIR");
ShAdd(h,"ABS_C1");
 ShAdd(h,"ABS_C1_RAW_HYPOTHETICAL"); ShAdd(h,"ABS_C2"); ShAdd(h,"ABS_C3");
 ShAdd(h,"ABS_C23");       ShAdd(h,"ABS_C4");       ShAdd(h,"ABS_SCORE");
 ShAdd(h,"ABS_CLASS");       ShAdd(h,"ABS_VALID");
 // CONT (16)
 ShAdd(h,"CONT_DIR");       ShAdd(h,"CONT_C5");
ShAdd(h,"CONT_EFFORT_OK");
 ShAdd(h,"CONT_DISPLACEMENT_OK"); ShAdd(h,"CONT_POC_STATE");
ShAdd(h,"CONT_MOVE_DIR");
 ShAdd(h,"CONT_WAVEDIR_OK"); ShAdd(h,"CONT_CANDLE_DIR_OK");
ShAdd(h,"CONT_C1");
```

```
  ShAdd(h,"CONT_C2");       ShAdd(h,"CONT_C3");       ShAdd(h,"CONT_C23");
  ShAdd(h,"CONT_C4");       ShAdd(h,"CONT_SCORE");       ShAdd(h,"CONT_CLASS");
  ShAdd(h,"CONT_VALID");
  // FLAGS (5)
  ShAdd(h,"OPPOSITE_ROUTE_VALID"); ShAdd(h,"SAME_ROUTE_NEW_VALID");
ShAdd(h,"SAME_ROUTE_DIR");
  ShAdd(h,"ARMED_CONTEXT_STILL_VALID");
ShAdd(h,"ROUTE_CONFLICT_SHADOW");
  // [v2.43] FLOW (7) — anexadas ao FINAL (posições v2.42 preservadas)
  ShAdd(h,"ABS_CTX_TAG"); ShAdd(h,"ABS_FOLLOWING_REACTION");
ShAdd(h,"ABS_FOLLOWING_EXTENSION");
  ShAdd(h,"FLOW_REGIME"); ShAdd(h,"VOL_RATIO");
ShAdd(h,"PRICE_EFFICIENCY");
  ShAdd(h,"POC_RESPONSE");
  FileWriteString(shadowHandle, h + "\r\n");
  FileFlush(shadowHandle);
}

void WriteShadowRow(string evt, string origin, const MatrixSnapshot &s, bool withArmed)
{
  if(shadowHandle == INVALID_HANDLE) return;

 bool armedOn = withArmed && armed.active;
 double trig = 0.0;
 int bars = -1;
 if(armedOn)
 {
    trig = ComputeTrigger(armed.dir, armed.regionHigh, armed.regionLow);
    bars = iBarShift(Symbol(), Inp_Timeframe, armed.regionEndTime, true);
 }
 bool opp=false, sameValid=false, still=false, conf=false; int sameDir=0;
 if(armedOn) ComputeShadowFlags(s, opp, sameValid, sameDir, still, conf);
 else     conf = (s.status == MSS_OK) && s.absValid && s.contValid;

  string L = "";
  ShAdd(L, TimeToString(TimeTradeServer(), TIME_DATE|TIME_SECONDS));
  ShAdd(L, evt);
  ShAdd(L, origin);
  ShAdd(L, TimeToString(iTime(Symbol(), Inp_Timeframe, 1),
TIME_DATE|TIME_MINUTES));
  ShAdd(L, MatrixStatusStr(s.status));
  ShAdd(L, ShB(armedOn));
  ShAdd(L, armedOn ? armed.route : "NONE");
  ShAdd(L, armedOn ? armed.mode : "NONE");
  ShAdd(L, armedOn ? DirStr(armed.dir) : "NONE");
  ShAdd(L, ShD2(armedOn ? armed.regionHigh : 0.0));
  ShAdd(L, ShD2(armedOn ? armed.regionLow : 0.0));
  ShAdd(L, ShD2(trig));
```

```
  ShAdd(L, ShI(bars));
  ShAdd(L, ShD2(s.price));
  ShAdd(L, ShD2(s.open1));
  ShAdd(L, ShD4(SigmaD1));
  ShAdd(L, ShD2(s.vwapVig));
  ShAdd(L, ShD4(s.posRel));
  ShAdd(L, ShB(s.glass));
  ShAdd(L, ShD2(s.distAbs));
  ShAdd(L, ShD2(s.effort));
  ShAdd(L, ShI(s.waveDir));
  ShAdd(L, ShD2(s.candleBody));
  ShAdd(L, ShD2(s.maxBody));
  ShAdd(L, ShI(s.candleDir));
  ShAdd(L, ShB(s.windowsValid));
  ShAdd(L, ShD2(s.priceMove));
  ShAdd(L, ShD2(s.pocMove));
  ShAdd(L, ShD2(s.pocNow));
  ShAdd(L, ShD2(s.pocPrev));
  ShAdd(L, DoubleToString(s.vA, 0));
  ShAdd(L, DoubleToString(s.vB, 0));
  ShAdd(L, ShD2(s.minDispl));
  ShAdd(L, ShD2(s.contMinDispl));
  // ABS (20)
  ShAdd(L, DirStr(s.dir));
  ShAdd(L, ShB(s.c5AbsOk));
  ShAdd(L, ShB(s.c1_effort_ok));
  ShAdd(L, s.pocState);
  ShAdd(L, AbsContextStr(s));
  ShAdd(L, ShB(s.absDivergence));
ShAdd(L, ShB(s.absStallStrict));
ShAdd(L, ShB(s.absStallRaw));
ShAdd(L, ShB(s.c1_body_ok));
ShAdd(L, ShB(s.displacementOk));
ShAdd(L, ShI(s.moveDir));
ShAdd(L, ShB(s.c1));
ShAdd(L, ShB(s.c1RawHyp));
ShAdd(L, ShB(s.c2));
ShAdd(L, ShB(s.c3));
ShAdd(L, ShB(s.c23));
ShAdd(L, ShB(s.c4));
ShAdd(L, ShI(s.scoreEff));
ShAdd(L, AbsClassStr(s));
ShAdd(L, ShB(s.absValid));
// CONT (16)
ShAdd(L, DirStr(s.dirCont));
ShAdd(L, ShB(s.c5ContOk));
ShAdd(L, ShB(s.cont_effort_ok));
ShAdd(L, ShB(s.cont_displacement_ok));
```

```
ShAdd(L, s.contPocState);
ShAdd(L, ShI(s.contMoveDir));
ShAdd(L, ShB(s.cont_wavedir_ok));
ShAdd(L, ShB(s.cont_candle_dir_ok));
ShAdd(L, ShB(s.c1Cont));
ShAdd(L, ShB(s.c2Cont));
ShAdd(L, ShB(s.c3Cont));
ShAdd(L, ShB(s.c23Cont));
ShAdd(L, ShB(s.c4Cont));
ShAdd(L, ShI(s.scoreContEff));
ShAdd(L, ContClassStr(s));
ShAdd(L, ShB(s.contValid));
// FLAGS (5)
ShAdd(L, ShB(opp));
ShAdd(L, ShB(sameValid));
ShAdd(L, DirStr(sameDir));
ShAdd(L, ShB(still));
ShAdd(L, ShB(conf));
// [v2.43] FLOW (7)
ShAdd(L, s.absCtxTag);
ShAdd(L, ShB(s.absFollowingReaction));
ShAdd(L, ShB(s.absFollowingExtension));
ShAdd(L, s.flowRegime);
ShAdd(L, ShD4(s.volRatio));
ShAdd(L, ShD4(s.priceEfficiency));
ShAdd(L, ShD4(s.pocResponse));

FileWriteString(shadowHandle, L + "\r\n");
FileFlush(shadowHandle);
}

// [v2.42] STALL_RAW=true && STALL_STRICT=false — SOMENTE telemetria
void LogStallRawOnly(const MatrixSnapshot &s, string origin, bool withArmed)
{
  PrintFormat("[REMORA2.0][ABS_STALL_RAW_ONLY] Origem=%s | Dir=%s |
STALL_RAW=1 STALL_STRICT=0 (body_ok=0) | Price=%.2f Sigma=%.4f | Corpo=%.2f
MaxBody=%.2f | PriceMove=%.2f MinDispl=%.2f PocMove=%.2f | Eff=%.2f (effOk=%d) |
POC=%.2f/%.2f | C23=%d C4=%d | PosRel=%.4f Glass=%s | C1_RAW_HIPOTETICO=%d |
NÃO ARMA (telemetria).",
        origin, DirStr(s.dir), s.price, SigmaD1,
        s.candleBody, s.maxBody,
        s.priceMove, s.minDispl, s.pocMove,
        s.effort, (int)s.c1_effort_ok,
        s.pocNow, s.pocPrev,
        (int)s.c23, (int)s.c4,
        s.posRel, s.glass ? "SIM" : "NAO",
        (int)s.c1RawHyp);
  WriteShadowRow("ABS_STALL_RAW_ONLY", origin, s, withArmed);
```

```
}

// [v2.42] Detalhamento do conflito ABS x CONT — SOMENTE telemetria
void LogRouteConflict(const MatrixSnapshot &s, string origin, bool withArmed)
{
  PrintFormat("[REMORA2.0][ROUTE_CONFLICT_SHADOW] Origem=%s | ABS: dir=%s
ctx=%s C1=%d C23=%d C4=%d score=%d/3 modo=%s POC=%s | CONT: dir=%s C1=%d
C23=%d C4=%d score=%d/3 modo=%s POC=%s",
        origin,
        DirStr(s.dir), AbsContextStr(s), (int)s.c1, (int)s.c23, (int)s.c4, s.scoreEff,
AbsClassStr(s), s.pocState,
        DirStr(s.dirCont), (int)s.c1Cont, (int)s.c23Cont, (int)s.c4Cont, s.scoreContEff,
ContClassStr(s), s.contPocState);
  PrintFormat("[REMORA2.0][ROUTE_CONFLICT_SHADOW] PriceMove=%.2f
MinDisplABS=%.2f MinDisplCONT=%.2f PocMove=%.2f | Eff=%.2f | Glass=%s PosRel=%.4f
| Nenhuma prioridade; execução inalterada.",
        s.priceMove, s.minDispl, s.contMinDispl, s.pocMove,
        s.effort, s.glass ? "SIM" : "NAO", s.posRel);
  WriteShadowRow("ROUTE_CONFLICT_SHADOW", origin, s, withArmed);
}

//+------------------------------------------------------------------+
//| [v2.42] AVALIAÇÃO SHADOW DURANTE ARMED — READ-ONLY               |
//| Chamada SOMENTE com armed.active && newBar, ANTES do Estágio 2. |
//| Usa a MESMA ComputeMatrixSnapshot() da rota operacional.         |
//| PROIBIDO: armar, ResetArmedSignal, enviar ordem, consumir cota,  |
//| alterar lastEntryTime/gC1Audit/UpperBand1/LowerBand1/armed,      |
//| persistir, alterar gestão, SL/TP ou prioridade de rota.          |
//+------------------------------------------------------------------+
// [2.436/A] Formula fornecida no pedido; usa validade integral da matriz 2.435.
// OPPOSITE_ROUTE_VALID da SHADOW significa OUTRA ROTA, nao direcao contraria.
// Snapshot invalido nunca e evidencia de contradicao. Nenhum criterio TURBO novo.
bool HasHardDirectionalContradiction(const MatrixSnapshot &s,int armedDir)
{
  if(s.status != MSS_OK) return false;
  return (s.absValid && s.dir == -armedDir) ||
       (s.contValid && s.dirCont == -armedDir);
}

void RunArmedShadow(MatrixSnapshot &s)
{
 if(!armed.active) return;

 bool dataOk = (d1Ready && gVwapOk && gVwapValue > 0.0 && wwValid);
 if(dataOk)
   ComputeMatrixSnapshot(s);
 else
 {
```

```
   ResetMatrixSnapshot(s);
   s.status = MSS_SHADOW_DATA_INVALID;
 }

 bool opp=false, sameValid=false, still=false, conf=false; int sameDir=0;
 ComputeShadowFlags(s, opp, sameValid, sameDir, still, conf);

 double trig = ComputeTrigger(armed.dir, armed.regionHigh, armed.regionLow);
 int bars = iBarShift(Symbol(), Inp_Timeframe, armed.regionEndTime, true);

 PrintFormat("[REMORA2.0][ARMED_SHADOW] ARMED=%s %s %s | RegHigh=%.2f
RegLow=%.2f | Trigger=%.2f | Idade=%d velas | Snapshot=%s | SOMENTE TELEMETRIA",
       armed.route, armed.mode, DirStr(armed.dir),
       armed.regionHigh, armed.regionLow, trig, bars,
       MatrixStatusStr(s.status));

  if(s.status == MSS_OK)
  {
     PrintFormat("[REMORA2.0][ARMED_SHADOW][ABS] Dir=%s Glass=%s C5=%d |
Eff=%.2f effOk=%d | POC=%s CTX=%s | body=%d displ=%d moveDir=%d waveDir=%d |
C1=%d C23=%d C4=%d SCORE=%d/3 | %s | VALID=%d | FLOW=%s react=%d ext=%d",
           DirStr(s.dir), s.glass ? "SIM" : "NAO", (int)s.c5AbsOk,
           s.effort, (int)s.c1_effort_ok,
           s.pocState, AbsContextStr(s),
           (int)s.c1_body_ok, (int)s.displacementOk, s.moveDir, s.waveDir,
           (int)s.c1, (int)s.c23, (int)s.c4, s.scoreEff,
           AbsClassStr(s), (int)s.absValid,
           s.flowRegime, (int)s.absFollowingReaction, (int)s.absFollowingExtension);
    PrintFormat("[REMORA2.0][ARMED_SHADOW][CONT] Dir=%s Glass=%s C5=%d |
Eff=%.2f effOk=%d | displ=%d POC=%s moveDir=%d waveDir=%d candleDir=%d | C1=%d
C23=%d C4=%d SCORE=%d/3 | %s | VALID=%d",
           DirStr(s.dirCont), s.glass ? "SIM" : "NAO", (int)s.c5ContOk,
           s.effort, (int)s.cont_effort_ok,
           (int)s.cont_displacement_ok, s.contPocState, s.contMoveDir, s.waveDir,
s.candleDir,
           (int)s.c1Cont, (int)s.c23Cont, (int)s.c4Cont, s.scoreContEff,
           ContClassStr(s), (int)s.contValid);
  }
  PrintFormat("[REMORA2.0][ARMED_SHADOW][FLAGS] OPPOSITE_ROUTE_VALID=%d
| SAME_ROUTE_NEW_VALID=%d (dir=%s) | ARMED_CONTEXT_STILL_VALID=%d |
ROUTE_CONFLICT_SHADOW=%d | Execução INALTERADA.",
         (int)opp, (int)sameValid, DirStr(sameDir), (int)still, (int)conf);

    WriteShadowRow("ARMED_SHADOW", "SHADOW", s, true);

    if(s.status == MSS_OK && Inp_EnableAbsorption && s.absStallRaw && !s.absStallStrict)
      LogStallRawOnly(s, "SHADOW", true);
```

```
   if(conf)
      LogRouteConflict(s, "SHADOW", true);
}

//+------------------------------------------------------------------+
//| [v2.43] AVALIAÇÃO SHADOW COM POSIÇÃO ABERTA — READ-ONLY          |
//| Chamada SOMENTE no gate HasOpenPosition() do OnTick (newBar e    |
//| dentro da sessão), imediatamente ANTES do return já existente.   |
//| Usa a MESMA ComputeMatrixSnapshot() (função pura).               |
//| PROIBIDO: armar, enviar ordem, alterar/fechar/inverter posição,  |
//| cancelar sinal, mudar contador, alterar gLastMatrixEvalBar,      |
//| modificar BE/SL/TP, alterar persistência, gC1Audit ou bandas.    |
//+------------------------------------------------------------------+
void RunPositionShadow()
{
  MatrixSnapshot s;
  bool dataOk = (d1Ready && gVwapOk && gVwapValue > 0.0 && wwValid);
  if(dataOk)
    ComputeMatrixSnapshot(s);
  else
  {
    ResetMatrixSnapshot(s);
    s.status = MSS_SHADOW_DATA_INVALID;
  }

   if(s.status == MSS_OK)
   {
  PrintFormat("[REMORA2.0][POSITION_SHADOW] ABS Dir=%s | C1=%d C23=%d
C4=%d C5=%d | WW(eff=%.2f effOk=%d wDir=%d) | POC_STATE=%s moveDir=%d |
FLOW=%s | REACTION=%d EXTENSION=%d | CTX=%s | ABS_HIPOTETICO=%s |
CONT_HIPOTETICO=%s | SOMENTE TELEMETRIA (posição aberta; nenhuma ação).",
        DirStr(s.dir),
        (int)s.c1, (int)s.c23, (int)s.c4, (int)s.c5AbsOk,
        s.effort, (int)s.c1_effort_ok, s.waveDir,
        s.pocState, s.moveDir,
        s.flowRegime,
        (int)s.absFollowingReaction, (int)s.absFollowingExtension,
        s.absCtxTag,
        AbsClassStr(s), ContClassStr(s));
 }
 else
 {
  PrintFormat("[REMORA2.0][POSITION_SHADOW] Snapshot=%s | SOMENTE
TELEMETRIA (posição aberta; nenhuma ação).",
        MatrixStatusStr(s.status));
 }

   WriteShadowRow("POSITION_SHADOW", "POSITION", s, false);
```

```
}

//+==================================================================+
//| ESTÁGIO 1 — MATRIZ HONESTA (Cirurgias 1/2/3/4/5) — DUAS ROTAS |
//| Responde: existe CONTEXTO institucional? (NÃO executa.)          |
//| C5 = PORTÃO por rota (não vota). C1 = PILAR MESTRE.              |
//| C23=(C2||C3) e C4 = confirmações. BASE=2/3 c/ C1 | TURBO=3/3. |
//| ROTA A (ABS) v2.43: C5_ABS=dir!=0 (Glass observa, não aborta); |
//| C1 = esforço && (DIVERGENCE || STALL || FOLLOWING_REACTION). |
//| ROTA B (CONT): equações v2.40; C5_CONT = dirCont!=0 && !glass. |
//| Em caso de confluência: NÃO executa, MAPEIA A REGIÃO e ARMA. |
//| v2.42: cálculo vem de ComputeMatrixSnapshot() (equações idênticas)|
//| e é exposto por aliases locais com os MESMOS nomes da v2.41. |
//+==================================================================+
void EvaluateMatrix()
{
   // Cirurgia 5 — call site 3: limpa snapshot ANTES de qualquer retorno
   ResetC1AuditSnapshot(gC1Audit);

    // [v2.42] Núcleo puro compartilhado com a SHADOW
    MatrixSnapshot s;
    ComputeMatrixSnapshot(s);

 //--- Validações estruturais (mesmas auditorias/retornos da v2.41) ---
 if(s.status == MSS_VWAP_D1_INVALID){
WriteAudit("VWAP_D1_INVALID","NONE","NONE",0,0,0,0,0,"NA",0,0,0,0,0,0,0,0); return; }
 if(s.status == MSS_SIGMA_INVALID){
WriteAudit("SIGMA_INVALID","NONE","NONE",0,0,0,0,0,"NA",0,0,0,0,0,0,0,0); return; }

 // Bandas publicadas nos globais SOMENTE pela rota operacional (como na v2.41)
 if(s.bandsComputed)
 {
    UpperBand1 = s.upper;
    LowerBand1 = s.lower;
 }
 if(s.status == MSS_STRUCT_INVALID) return;

 double vwapVig = s.vwapVig;
 double price = s.price;
 double posRel = s.posRel;
 bool glass = s.glass;

 PrintFormat("[REMORA2.0][AUDIT] Price=%.2f VIG=%.2f Up=%.2f Lo=%.2f | PosRel=%.4f
| Glass=%s",
       price, vwapVig, UpperBand1, LowerBand1, posRel, glass?"SIM":"NAO");

 // PORTÃO 1 — Teto de vidro POR ROTA (sem RETURN global)
 if(glass)
```

```
 {
   if(!Inp_EnableAbsorption)
   {
      // Somente CONT habilitada: nenhuma rota pode aproveitar o extremo.
      // Comportamento IDÊNTICO ao v2.40 nesta configuração.
      PrintFormat("[REMORA2.0][TETO_VIDRO] PosRel=%.4f rompeu Banda 2 (>%.2f /
<%.2f). Somente CONT habilitada => Kamikaze => ABORTADO.",
            posRel, Inp_VWAP_Outer_Pct, 1.0-Inp_VWAP_Outer_Pct);

WriteAudit("KAMIKAZE_GLASS","NONE","NONE",price,posRel,0,0,0,"NA",vwapVig,0,0,0,0,
0,0,0);
     return;
   }
   PrintFormat("[REMORA2.0][ABS_GLASS_OBSERVE] PosRel=%.4f além da Banda 2
(>%.2f / <%.2f). EXTREMO: ABS continua investigando absorção (Glass sozinho NÃO
arma) | CONT=%s.",
          posRel, Inp_VWAP_Outer_Pct, 1.0-Inp_VWAP_Outer_Pct,
          Inp_EnableContinuation ? "BLOQUEADA (C5_CONT=0)" : "OFF");
 }

  // PORTÃO 2 — Sem gatilho direcional (ramo separado, auditoria própria)
  if(s.status == MSS_STOP_NO_C5_TRIGGER){
WriteAudit("NO_C5_TRIGGER","NONE","NONE",price,posRel,0,0,0,"NA",vwapVig,0,0,0,0,0,
0,0); return; }
  if(s.status != MSS_OK) return; // defensivo (inatingível com o fluxo acima)

 //--- Aliases locais (MESMOS nomes da v2.41) — somente leitura do snapshot
int dir   = s.dir;
int dirCont = s.dirCont;
bool c5AbsOk = s.c5AbsOk;
bool c5ContOk = s.c5ContOk;

double distAbs     = s.distAbs;
double distC2_Fixa = Inp_Pts_C2_Longa;
double distC3_Min_Fixa = Inp_Pts_C3_Min;
double distC3_Max_Fixa = Inp_Pts_C3_Max;
double minDispl_Dyn = s.minDispl;
double maxBody_Dyn       = s.maxBody;
double contMinDispl_Dyn = s.contMinDispl;

bool c2 = s.c2, c3 = s.c3, c23 = s.c23, c4 = s.c4;
bool c2Cont = s.c2Cont, c3Cont = s.c3Cont, c23Cont = s.c23Cont, c4Cont = s.c4Cont;

double candleBody = s.candleBody;
double pocNow       = s.pocNow;
double pocPrev      = s.pocPrev;
double vA        = s.vA;
double vB        = s.vB;
```

```
double lastPriceNow = s.lastPriceNow;
double lastPricePrev = s.lastPricePrev;
double priceMove = s.priceMove;
double pocMove       = s.pocMove;
int moveDir      = s.moveDir;
string pocState   = s.pocState;
int contMoveDir = s.contMoveDir;
string contPocState = s.contPocState;
bool anchored      = s.anchored;

double effort = s.effort;
int waveDir = s.waveDir;

bool c1_effort_ok     = s.c1_effort_ok;
bool c1_body_ok        = s.c1_body_ok;
bool c1_displacement_ok = s.c1_displacement_ok;
bool c1_movedir_ok      = s.c1_movedir_ok;
bool c1_wavedir_ok      = s.c1_wavedir_ok;
bool c1_poc_ok        = s.c1_poc_ok;
bool c1           = s.c1;
string absCtxTag      = s.absCtxTag;

int candleDir       = s.candleDir;
bool cont_effort_ok    = s.cont_effort_ok;
bool cont_displacement_ok = s.cont_displacement_ok;
bool cont_poc_ok       = s.cont_poc_ok;
bool cont_movedir_ok     = s.cont_movedir_ok;
 bool cont_wavedir_ok    = s.cont_wavedir_ok;
 bool cont_candle_dir_ok = s.cont_candle_dir_ok;
 bool c1Cont         = s.c1Cont;

 bool turbo    = s.turboAbs;
 int scoreEff = s.scoreEff;
 bool turboCont = s.turboCont;
 bool baseCont = s.baseCont;
 int scoreContEff = s.scoreContEff;

 //================================================================
 // SELEÇÃO DA ROTA
 // Cada rota só arma se habilitada E com o próprio C5 aprovado.
 // v2.41: as rotas NÃO são mais mutuamente exclusivas por construção
 // (ABS STALL x CONT FOLLOWING podem coexistir). [FIX-3] o conflito
 // é BLOQUEADO, sem prioridade arbitrária.
 //================================================================
 bool absValid = s.absValid; // = Inp_EnableAbsorption && c5AbsOk && (turbo || base)
 bool contValid = s.contValid; // = Inp_EnableContinuation && c5ContOk && (turboCont ||
baseCont)
```

```
  // [v2.42] STALL_RAW sem STALL_STRICT — SOMENTE telemetria (não altera
c1/absValid)
  if(Inp_EnableAbsorption && s.absStallRaw && !s.absStallStrict)
     LogStallRawOnly(s, "OPERATIONAL", false);

  // [v2.43] FOLLOWING_EXTENSION — SOMENTE diagnóstico (NÃO é gate novo).
  // Explica por que POC_FOLLOWING não validou a ABS nesta M5.
  if(Inp_EnableAbsorption && s.absFollowingExtension)
  {
     PrintFormat("[REMORA2.0][ABS_FOLLOWING_EXTENSION_BLOCK] Dir=%s |
POC_FOLLOWING com moveDir=%d (== -dir): preço+POC seguem aprofundando o
extremo original | PriceMove=%.2f PocMove=%.2f MinDispl=%.2f | Eff=%.2f effOk=%d |
FLOW=%s | FOLLOWING NÃO valida C1 ABS (diagnóstico).",
            DirStr(s.dir), s.moveDir,
            s.priceMove, s.pocMove, s.minDispl,
            s.effort, (int)s.c1_effort_ok, s.flowRegime);
     WriteShadowRow("ABS_FOLLOWING_EXTENSION_BLOCK", "OPERATIONAL", s,
false);
  }

 // [FIX-3] ROUTE_CONFLICT_BLOCK — nenhuma prioridade arbitrária
 if(absValid && contValid)
 {
    PrintFormat("[REMORA2.0][ROUTE_CONFLICT_BLOCK] ABS(%s, POC=%s) e
CONT(POC=%s) simultaneamente válidas. Nenhuma prioridade. Entrada BLOQUEADA.",
            absCtxTag, pocState, contPocState);
    WriteAudit("ROUTE_CONFLICT_BLOCK", "NONE", "NONE", price, posRel, distAbs,
effort,
            pocNow, "NA", vwapVig,
            (int)c1, (int)c2, (int)c3, (int)c4, (int)c5AbsOk, 0, 0);
    // [v2.42] Detalhamento das duas rotas no CSV auxiliar (bloqueio INALTERADO)
    LogRouteConflict(s, "OPERATIONAL", false);
    ResetC1AuditSnapshot(gC1Audit);
    return;
  }

 // Rota do snapshot/telemetria. Com rota válida = a rota que vai armar.
 // Sem rota válida: CONT se a POC da CONT está FOLLOWING e o C5_CONT
 // está aprovado (ou ABS desabilitada); senão ABS.
 // Com Glass=false, idêntico à v2.40 (c5ContOk == true).
 bool useCont;
 if(absValid)     useCont = false;
 else if(contValid) useCont = true;
 else          useCont = Inp_EnableContinuation &&
                   (!Inp_EnableAbsorption ||
                    (c5ContOk && contPocState == "POC_FOLLOWING"));

 // Estado da POC / moveDir / piso da rota selecionada (TELEMETRIA)
```

```
string pocStateSel = useCont ? contPocState : pocState;
int moveDirSel = useCont ? contMoveDir         : moveDir;
double minDisplSel = useCont ? contMinDispl_Dyn : minDispl_Dyn;

// Cirurgia 5 — call site 4: preenche snapshot após TODAS as subcondições
gC1Audit.valid           = true;
gC1Audit.candleBody          = candleBody;
gC1Audit.maxDisplThreshold = minDisplSel;
gC1Audit.priceMove          = priceMove;
gC1Audit.pocMove            = pocMove;
gC1Audit.moveDir           = moveDirSel;
gC1Audit.waveDir           = waveDir;
gC1Audit.pocState          = pocStateSel;
gC1Audit.volNow           = vA;
gC1Audit.volPrev          = vB;
gC1Audit.pocNow             = pocNow;
gC1Audit.pocPrev           = pocPrev;
gC1Audit.lastPriceNow        = lastPriceNow;
gC1Audit.lastPricePrev = lastPricePrev;
gC1Audit.anchored           = anchored ? 1 : 0;

if(!useCont)
{
   // Semântica ABS v2.41/v2.43: poc_ok = absPocContextOk; demais campos REAIS.
   gC1Audit.effort_ok   = c1_effort_ok     ? 1 : 0;
   gC1Audit.body_ok      = c1_body_ok       ? 1 : 0;
  gC1Audit.displacement_ok = c1_displacement_ok ? 1 : 0;
  gC1Audit.poc_ok       = c1_poc_ok       ? 1 : 0;
  gC1Audit.movedir_ok     = c1_movedir_ok     ? 1 : 0;
  gC1Audit.wavedir_ok    = c1_wavedir_ok     ? 1 : 0;
  gC1Audit.c23        = c23 ? 1 : 0;
  gC1Audit.scoreEff    = scoreEff;
}
else
{
  // Semântica CONT: body_ok = vela na direção da operação
  gC1Audit.effort_ok   = cont_effort_ok    ? 1 : 0;
  gC1Audit.body_ok      = cont_candle_dir_ok ? 1 : 0;
  gC1Audit.displacement_ok = cont_displacement_ok ? 1 : 0;
  gC1Audit.poc_ok       = cont_poc_ok       ? 1 : 0;
  gC1Audit.movedir_ok     = cont_movedir_ok     ? 1 : 0;
  gC1Audit.wavedir_ok    = cont_wavedir_ok      ? 1 : 0;
  gC1Audit.c23        = c23Cont ? 1 : 0;
  gC1Audit.scoreEff    = scoreContEff;
}

// [v2.41] Registro auditável do extremo observado pela ABS (1x por M5)
if(glass && Inp_EnableAbsorption)
```

```
{
   WriteAudit("ABS_GLASS_OBSERVE", DirStr(dir), "NONE", price, posRel, distAbs, effort,
         pocNow, pocState, vwapVig,
         (int)c1, (int)c2, (int)c3, (int)c4, (int)c5AbsOk, 0, 0);
}

 if(Inp_EnableAbsorption)
 {
    PrintFormat(
      "[REMORA2.0][MATRIZ_ABS] Dir=%s | C5_ABS=%s | C1(MESTRE)=%d CTX=%s "
      "[eff=%d ctx=%d | TELEMETRIA: body=%d displ=%d mDir=%d wDir=%d] | "
      "C23=%d(C2=%d C3=%d) C4=%d | SCORE_EFF=%d/3 | "
      "WW(eff=%.2f wDir=%d) | Corpo=%.2f<=%.2f | "
      "MinDispl=%.2f | PriceMove=%.2f PocMove=%.2f moveDir=%d | "
      "POC=%.2f/%.2f Estado=%s Anc=%s | Vwin=%.0f/%.0f",
      DirStr(dir),
      c5AbsOk ? (glass ? "PORTAO_OK_GLASS_OBSERVE" : "PORTAO_OK") :
"SEM_DIR",
      (int)c1, absCtxTag,
      (int)c1_effort_ok,(int)c1_poc_ok,
      (int)c1_body_ok,(int)c1_displacement_ok,(int)c1_movedir_ok,(int)c1_wavedir_ok,
      (int)c23,(int)c2,(int)c3,(int)c4, scoreEff,
      effort, waveDir,
      candleBody, maxBody_Dyn,
      minDispl_Dyn,
      priceMove, pocMove, moveDir,
       pocNow, pocPrev, pocState, anchored?"SIM":"NAO", vA, vB
  );

   // [v2.43] Telemetria de fluxo — SOMENTE diagnóstico (NÃO vota)
   PrintFormat("[REMORA2.0][MATRIZ_ABS_FLOW] FLOW=%s | REACTION=%d
EXTENSION=%d | VolRatio=%.4f PriceEff=%.4f PocResp=%.4f | DIAGNÓSTICO (sem
threshold).",
          s.flowRegime,
          (int)s.absFollowingReaction, (int)s.absFollowingExtension,
          s.volRatio, s.priceEfficiency, s.pocResponse);
 }

 if(Inp_EnableContinuation)
 {
   // [FIX-2] Telemetria CONT: MinDispl = contMinDispl_Dyn | POC_STATE = contPocState
   PrintFormat(
      "[REMORA2.0][MATRIZ_CONT] Dir=%s | C5_CONT=%s | C1_CONT(MESTRE)=%d "
      "[eff=%d displ=%d poc=%d mDir=%d wDir=%d candle=%d] | "
      "C2_CONT=%d C3_CONT=%d C4_CONT=%d | SCORE_EFF=%d/3 | %s | "
      "Effort=%.2f MinEffort=%.2f | PriceMove=%.2f MinDispl=%.2f PocMove=%.2f | "
      "POC_STATE=%s moveDir=%d waveDir=%d candleDir=%d | "
      "Corpo=%.2f (%.3f sigma) | POC=%.2f/%.2f Anc=%s | Vwin=%.0f/%.0f",
```

```
      DirStr(dirCont),
      c5ContOk ? "PORTAO_OK" : "GLASS_BLOCK",
      (int)c1Cont,
      (int)cont_effort_ok,(int)cont_displacement_ok,(int)cont_poc_ok,
      (int)cont_movedir_ok,(int)cont_wavedir_ok,(int)cont_candle_dir_ok,
      (int)c2Cont,(int)c3Cont,(int)c4Cont, scoreContEff,
      turboCont ? "TURBO" : (baseCont ? "BASE" : "SEM_CONF"),
      effort, Inp_Cont_Vol_Effort,
      priceMove, contMinDispl_Dyn, pocMove,
      contPocState, contMoveDir, waveDir, candleDir,
      candleBody, (SigmaD1>0.0 ? candleBody/SigmaD1 : 0.0),
      pocNow, pocPrev, anchored?"SIM":"NAO", vA, vB
   );
}

//--- Variáveis da rota SELECIONADA ---------------------------------
int dirSel = useCont ? dirCont      : dir;
int c1Sel = useCont ? (int)c1Cont     : (int)c1;
int c2Sel = useCont ? (int)c2Cont     : (int)c2;
int c3Sel = useCont ? (int)c3Cont     : (int)c3;
int c4Sel = useCont ? (int)c4Cont     : (int)c4;
int c5Sel = useCont ? (int)c5ContOk : (int)c5AbsOk;
int scoreSel = useCont ? scoreContEff      : scoreEff;
bool turboSel = useCont ? turboCont       : turbo;

if(!absValid && !contValid)
  {
   PrintFormat("[REMORA2.0][SEM_CONFLUENCIA] ABS(%s): SCORE_EFF=%d/3
C1=%d C23=%d C4=%d | CONT(%s): SCORE_EFF=%d/3 C1=%d C23=%d C4=%d | Sem
BASE (C1+1) nem TURBO (3/3) em rota habilitada. Sinal descartado.",
           Inp_EnableAbsorption ? "ON" : "OFF", scoreEff, (int)c1, (int)c23, (int)c4,
           Inp_EnableContinuation ? "ON" : "OFF", scoreContEff, (int)c1Cont, (int)c23Cont,
(int)c4Cont);
   WriteAudit(useCont ? "NO_CONFLUENCE_CONT" : "NO_CONFLUENCE",
DirStr(dirSel),
          "NONE", price, posRel, distAbs, effort,
          pocNow, pocStateSel,
          vwapVig, c1Sel, c2Sel, c3Sel, c4Sel, c5Sel, 0, 0);
   ResetC1AuditSnapshot(gC1Audit); // limpar APÓS gravar
   return;
  }

  PrintFormat(
   "[REMORA2.0][GEOMETRIA C2/C3 FIXA] Rota=%s | Familia=%s | C2_Req>=%.2f pts |
"
      "C3_Req=[%.2f a %.2f] pts | DistAtual=%.2f | SigmaD1=%.4f | "
      "MinDespl(C1)=%.2f | MaxCorpo(C1 ABS)=%.2f",
      useCont ? "CONT" : "ABS",
```

```
        gIsIndexFamily ? "WIN/IND" : "WDO/DOL",
        distC2_Fixa, distC3_Min_Fixa, distC3_Max_Fixa,
        distAbs, SigmaD1, minDisplSel, maxBody_Dyn);

    //--- BASE x TURBO DINÂMICO (SL com teto financeiro, TP multiplicador) ----
    // Rota CONT marca o modo com sufixo _CONT (leva a rota p/ CSV, ordem e deals).
    string mode;
    if(useCont) mode = turboSel ? "TURBO_CONT" : "BASE_CONT";
    else     mode = turboSel ? "TURBO"    : "BASE";

    // Stop Dinâmico protegido pelo Teto Financeiro para não destruir sua conta
    double activeSL = SigmaRisk() * Inp_Mult_SL;

  // Take Profit Dinâmico Livre (busca o alvo completo da volatilidade)
  double activeTP = turboSel ? (SigmaD1 * Inp_Mult_TP_Turbo) : (SigmaD1 *
Inp_Mult_TP_Base);

    int lots = FullSizeLot();
    if(lots==0)
    {
      WriteAudit("BAL_INSUFICIENTE", DirStr(dirSel), mode, price, posRel, distAbs, effort,
            pocNow, pocStateSel, vwapVig, c1Sel, c2Sel, c3Sel, c4Sel, c5Sel, 0, 0);
      ResetC1AuditSnapshot(gC1Audit); // limpar APÓS gravar
      return;
    }
//=================================================================
// MAPEIA A REGIÃO DE ANOMALIA (Cluster) E ARMA O GATILHO
// Varre 1..Inp_Region_Candles => Máxima Absoluta e Mínima Absoluta
//=================================================================
double regHigh = -DBL_MAX;
double regLow = DBL_MAX;

MqlRates regionRates[];
int regionCopied = CopyRates(
   Symbol(),
   Inp_Timeframe,
   1,          // começa na última barra FECHADA
   Inp_Region_Candles,
   regionRates
);

bool regionDataOk = (regionCopied == Inp_Region_Candles);

if(regionDataOk)
{
   for(int i = 0; i < regionCopied; i++)
   {
     double hi = regionRates[i].high;
```

```
    double lo = regionRates[i].low;

      if(hi <= 0.0 ||
         lo <= 0.0 ||
         hi < lo)
      {
         regionDataOk = false;
         break;
      }

      if(hi > regHigh) regHigh = hi;
      if(lo < regLow ) regLow = lo;
   }
}

if(!regionDataOk ||
  regHigh <= -DBL_MAX ||
  regLow >= DBL_MAX ||
  regHigh <= regLow)
{
  WriteAudit("REGION_INVALID", DirStr(dirSel), mode, price, posRel, distAbs, effort,
        pocNow, pocStateSel, vwapVig,
        c1Sel, c2Sel, c3Sel, c4Sel, c5Sel, 0, 0);
  ResetC1AuditSnapshot(gC1Audit);
  return;
 }

 armed.active       = true;
 armed.dir        = dirSel;
 armed.regionHigh = regHigh;
 armed.regionLow = regLow;
 armed.regionEndTime = iTime(Symbol(), Inp_Timeframe, 1); // ABERTURA da última barra
fechada da região
 armed.activeSL      = activeSL;
 armed.activeTP       = activeTP;
 armed.mode          = mode;
 armed.lots       = lots;
 armed.route        = useCont ? "CONT" : "ABS";
 armed.breakoutEffortMin = useCont ? Inp_Cont_Breakout_Effort : Inp_Breakout_Effort;
 armed.sameSideGlassLogged = false;
 armed.posRel        = posRel;
 armed.distD1       = distAbs;
 armed.effort      = effort;
 armed.vwapVig        = vwapVig;
 armed.c1=c1Sel; armed.c2=c2Sel; armed.c3=c3Sel; armed.c4=c4Sel; armed.c5=c5Sel;

 // Cirurgia 5 — call site 6: congela snapshot no sinal armado
 armed.audit      = gC1Audit;
```

```
 armed.audit.valid = true;

 double armTrigger = ComputeTrigger(dirSel, regHigh, regLow); // normalizado ceil/floor por
tick
 string ctxSel = useCont ? "CONT_CONTEXT_FOLLOWING" : absCtxTag;

  PrintFormat("[REMORA2.0][ARMED_%s] %s %s | %d lotes | SCORE_EFF=%d/3 (C1
Mestre OK) | SL=%.1f | Região[%d velas] High=%.2f Low=%.2f | Gatilho: %s trigger=%.2f |
Janela=%d velas | OndaViva>=%.2f | PosRel=%.3f DistD1=%.2f WW=%.2f | CTX=%s |
Glass=%s",
        armed.route, mode, DirStr(dirSel), lots, scoreSel, activeSL, Inp_Region_Candles,
        regHigh, regLow, (dirSel==-1?"last<=":"last>="), armTrigger,
        Inp_Breakout_Max_Bars, armed.breakoutEffortMin, posRel, distAbs, effort,
        ctxSel, glass ? "SIM" : "NAO");

 WriteAudit("ARMED", DirStr(dirSel), mode, price, posRel, distAbs, effort, armTrigger,
pocStateSel,
       vwapVig, c1Sel, c2Sel, c3Sel, c4Sel, c5Sel, lots, 0);

  ResetC1AuditSnapshot(gC1Audit); // gC1Audit limpo DEPOIS do ARMED (snapshot vive
em armed.audit)
}

//+------------------------------------------------------------------+
//| AUDITORIA CSV — 44 colunas (22 originais + 22 snapshot C1)       |
//| Assinatura INALTERADA. Snapshot: armed.audit > gC1Audit > neutro.|
//| [E] Novos status usam os campos existentes; nenhuma coluna nova. |
//| v2.40: a rota viaja no campo MODE (BASE/TURBO x BASE_CONT/       |
//| TURBO_CONT). v2.41: status ABS_GLASS_OBSERVE e                   |
//| ABS_LATE_GLASS_SAMESIDE_ALLOW (mesmas 44 colunas).               |
//| v2.43: arquivo _Audit_v243.csv; telemetria nova vai para o CSV |
//| auxiliar _Shadow_v243.csv (este CSV permanece com 44 colunas). |
//+------------------------------------------------------------------+
void WriteAudit(string status, string dirStr, string mode,
           double price, double posRel, double distD1, double effort,
           double poc, string pocState, double vwapVig,
           int c1,int c2,int c3,int c4,int c5,
           int lots, int retCode)
{
  if(csvHandle==INVALID_HANDLE) return;

 C1Audit src;
 if(armed.audit.valid) src = armed.audit;
 else if(gC1Audit.valid) src = gC1Audit;
 else            ResetC1AuditSnapshot(src);

 FileWrite(csvHandle,
  TimeToString(TimeTradeServer(), TIME_DATE|TIME_SECONDS),
```

```
   status, dirStr, mode,
   DoubleToString(price, 2),
   DoubleToString(posRel, 4),
   DoubleToString(distD1, 2),
   DoubleToString(effort, 2),
   DoubleToString(poc, 2),
   pocState,
   c1, c2, c3, c4, c5,
   lots, retCode,
   DoubleToString(vwapVig, 2),
   DoubleToString(VWAP_D1, 2),
   DoubleToString(SigmaD1, 4),
   DoubleToString(H1, 2),
   DoubleToString(L1, 2),
   // --- Snapshot C1 (Cirurgia 5) ---
   src.c23,
   src.scoreEff,
   src.effort_ok,
   src.body_ok,
   src.displacement_ok,
   src.poc_ok,
   src.movedir_ok,
   src.wavedir_ok,
   DoubleToString(src.candleBody, 2),
   DoubleToString(src.maxDisplThreshold, 2),
      DoubleToString(src.priceMove, 2),
      DoubleToString(src.pocMove, 2),
      src.moveDir,
      src.waveDir,
      src.pocState,
      DoubleToString(src.volNow, 0),
      DoubleToString(src.volPrev, 0),
      DoubleToString(src.pocNow, 2),
      DoubleToString(src.pocPrev, 2),
      DoubleToString(src.lastPriceNow, 2),
      DoubleToString(src.lastPricePrev, 2),
      src.anchored);
    FileFlush(csvHandle);
}

//+------------------------------------------------------------------+
double OnTester() { return 0.0; }
//+------------------------------------------------------------------+
```